Hybrid Adaptive Consensus: A Sovereign Finite-State-Machine Fallback for Modular Blockchains

Tien Cuong Dang[1] Binh Minh Nguyen[1,*]

[1]Hanoi University of Science and Technology, Hanoi, Vietnam

cuong.dt220020@sis.hust.edu.vn, minh.nguyenbinh@hust.edu.vn

*Corresponding author

Abstract

Modular blockchains obtain settlement and data availability (DA) from independently operated layers. A chain whose block validity requires Bitcoin settlement receipts and DA receipts inherits the liveness of those layers: when either stalls, no proposal is valid and the chain halts although its validators are connected and correct. Dropping the requirement restores progress but suspends the guarantees it provided. We present *Engram*, a protocol layer on a lightly modified CometBFT core that makes this suspension an agreed, deterministic state transition. Validators sign height-bound observations of Bitcoin, DA and peer health as vote extensions; a *health certificate* aggregates them so that every metric stays within the range of the honest observations (Lemma 1), and a four-state finite state machine evaluates it. In the degraded modes the chain keeps producing blocks, relaxes the receipt requirement of the failed layer and excludes withdrawal-marked transactions. It returns to ANCHORED only after hysteresis, DA backfill of the degraded interval and a validator-verified recovery proof over all but a bounded unproven tail; on seven machines in three regions this recovery completed, in two runs each, with the last boundary block committed in round 1 and in round 2. A layered TLA+ specification passes bounded model checking at two layers (2.13 million distinct states in the largest run); the refinement of the composed layer remains open. On a seven-validator testbed with Bitcoin regtest and Celestia, Engram kept 96–100% of its healthy block rate under a checkpoint pause, a DA outage and both, where a receipt-gated baseline that reads receipts at each validator kept 20–61% and one that reads them through vote extensions stopped committing 44–121 s after the fault began and did not resume. A composite fault repeated five times, on one host and on seven machines in three regions, ended in ANCHORED with complete AppHash agreement. The block interval grows by 1.8% on one host (95% interval 0.4 to 3.1%) and, at 100 tx/s across three regions, is 1.9 times vanilla's when the validators read the sensors inside `ExtendVote` and 1.01 times with background sensor reads. The price is the detection delay: 17–58 withdrawal-marked transactions committed before the lock (40–125 s on the testbed, 47–62 min when converted to mainnet Bitcoin cadence by a model, not measured on mainnet), and none after it.

Keywords: blockchain consensus, Byzantine fault tolerance, modular blockchain, finite state machine, formal verification, TLA+, zero-knowledge proofs

1. Introduction

Modular blockchains separate execution from services such as settlement and data availability (DA), which independently operated networks provide. Consider a chain whose validity predicate requires fresh evidence from such layers: a block is valid only if it passes the BFT checks and carries both a Bitcoin settlement receipt and a receipt from a DA network such as Celestia [1], each attesting a height within a tolerance of the current Bitcoin tip (τ_{BTC}) or of the block (τ_{DA}). Formally,

$$valid(B) = BFTvalid(B) \wedge BTCreceipt(B) \wedge DAreceipt(B).$$

If Bitcoin settlement is congested or censored, or the DA network is unavailable, no proposal satisfies the predicate. The BFT consensus layer [2, 3] built on top stalls although the validator set is fully connected and correct. The chain inherits the liveness of the layers it depends on.

Dropping the requirement restores progress but suspends the guarantees that motivated the modular design: without a settlement checkpoint the chain cannot claim externally anchored settlement, and without DA confirmation it cannot claim that its recent data is externally available. The problem is therefore not only to detect an outage. The protocol must make all validators agree on when an external guarantee may be suspended, restrict the operations whose safety depended on it, and define the evidence required before the guarantee is restored.

The gap. Existing work addresses parts of this problem under other failure models. Protocols that trade finality for availability adapt to validator participation or consensus-network conditions, not to external infrastructure [4,5]. Rollup mechanisms provide alternative paths when sequencing or data publication fails, but the decision rests with users, a sequencer, a committee, or a contract on the parent chain [6–8]. Oracle designs and ABCI++ vote extensions let validators bring externally observed values into replicated state [9,10], but the value goes to the application. None of them covers a native chain whose validity depends on independently operated external layers that different validators observe differently. There, validators must agree on the observation and on the change in safety policy that follows from it (Section 2).

Why the decision belongs in consensus. The relaxation of receipt requirements and the withdrawal circuit breaker change two things that every validator must agree on: whether a block is valid, and whether a withdrawal executes. Bitcoin, DA, and peripheral-network status are observed locally and asynchronously, so two honest validators may see different conditions at the same instant. Suppose each validator applied a local policy to its own view. Validators would disagree on validity, so proposals would collect nil votes and rounds would repeat, or they would execute withdrawals differently and produce diverging `AppHash` values. A local, non-deterministic observation cannot drive replicated state directly. It must first become a deterministic input that every validator evaluates identically.

Design approach. Engram keeps CometBFT's proposer selection, prevote/precommit protocol, quorum intersection, and deterministic finality as upstream's apart from the changes of the fork (Section 6.1), and modifies the object and the validity policy that this pipeline carries. At height $h-1$, each validator signs its sensor snapshot as a vote extension; the sensors are read only there. The proposer of height h includes the signed snapshots of a quorum (more than two thirds of the voting power) in a *health certificate* C^h. Every validator verifies C^h, recomputes a deterministic weighted-median/majority aggregate $\mathsf{Agg}(C^h)$, and accepts a proposal only if its target state satisfies $s^h = \delta(s^{h-1}, \mathsf{Agg}(C^h))$, where δ is the transition function of a four-state finite-state machine (FSM) with states Anchored, Suspicious, Sovereign, and Recovering. FSM state is written only from a certificate that CometBFT has already agreed on (Section 4.3). "Hybrid Adaptive Consensus" denotes this adaptive layer inside the consensus pipeline, not a replacement for the agreement protocol.

The FSM makes the chain's security mode explicit. In Anchored, blocks require the Bitcoin and DA receipts. Sustained degradation moves the chain through Suspicious into Sovereign, where the quorum keeps ordering blocks, the receipt requirement of each unhealthy layer is relaxed, and cross-chain withdrawals are locked. Return to Anchored requires recovery hysteresis, DA backfill of the Sovereign interval, and a validator-verified zero-knowledge proof over that interval except a bounded unproven tail (Sections 4.4 and 4.7; the prototype's proof gate is qualified in Section 8.3).

Scope. Engram handles degradation of Bitcoin, Celestia, and the peripheral P2P interfaces while the CometBFT quorum remains live. It does not address loss of the core quorum: like

any BFT protocol, it needs at least a quorum (more than two thirds of the voting power) to communicate after global stabilization time. "P2P degradation" in this paper always means partial, peripheral degradation (Section 3.1).

To our knowledge, no published design combines the following for a modular chain whose validity depends on external settlement or DA layers: (i) a quorum-backed set of validator observations from which every validator deterministically derives external-layer health; (ii) deterministic adaptation of both block-validity requirements and withdrawal admissibility from that health while the native quorum keeps ordering blocks; and (iii) a recovery condition that requires a proof over the degraded interval and its DA backfill. Engram targets this combination.

Contributions.

- **Quorum-backed external-health agreement.** A height-bound certificate of signed observations from a quorum, carried in vote extensions, and a deterministic weighted-median/majority aggregation rule. When Byzantine validators hold less than one third of the voting power, every numeric aggregate stays within the range of the honest observations in a well-formed certificate, whichever observations a Byzantine proposer includes or omits (Section 4.3).
- **Dependency-aware adaptive execution.** A four-state FSM on the certificate, with asymmetric hysteresis (quick to degrade, slow to restore), that changes the Bitcoin and DA receipt requirements without modifying CometBFT's agreement core. A down threshold of 2 flapped under wide-area noise and a threshold of 4 did not, at about 3 s more detection delay, while an adaptive threshold did not beat the static one (Section 7.3). A withdrawal circuit breaker [11] excludes withdrawal-marked transactions while the chain runs without external corroboration (Sections 4.4–4.5; Withdrawal Admission Safety, a hand proof under assumptions M1–M3 that constrains which blocks may contain a withdrawal and does not imply asset safety, in Section 5.1).
- **Proof- and backfill-gated recovery.** A Noir/UltraHonk circuit that proves, for up to 256 SOVEREIGN-interval headers at constant proving and verification cost, header-chain continuity, that every recorded state is SOVEREIGN or RECOVERING with the lock flag set, the binding of the interval to the last anchored root and to the new root, and the quorum signatures of the two boundary blocks, linked to chain state by the public inputs that the validators check. It does not check transitions against the transition function or show that no withdrawal executed. Longer intervals chain several proofs (Appendix D). The exit condition also requires that the interval's data was published to the DA layer. Validators verify each proof inside block execution (Section 4.7).
- **Layered formal specification.** A four-layer TLA+ specification. Bounded model checking of the Tendermint-style layer (safety) and the FSM layer (safety and liveness) found no violation within their bounds (the Tendermint-style layer at $N \leq 4$ and $f = 1$; the FSM layer at the thresholds of Table 20, not the deployed ones); the largest run explored 2.13 million distinct states. In an ablation study of the protocol's safety mechanisms, removing each mechanism makes the invariant that guards it fail, which shows that the invariants detect the removal and not that the mechanisms are sufficient. The refinement of the composed layer to the abstract consensus object is specified but open (Section 5).
- **Live evaluation.** A 7-validator Cosmos SDK testnet against live Bitcoin regtest and Celestia networks with network- and application-level fault injection, comparing Engram against two strict receipt-gated baselines (receipts read through vote extensions or at each validator), a baseline without the withdrawal lock, and a vanilla CometBFT control (Sections 6–7).

Table 1: Research questions

RQ	Question
RQ1	**Safety.** Under Byzantine voting power below one third (two of the seven testbed validators), do honest validators stay in agreement on the FSM state and the certified health, and is no withdrawal-marked transaction committed while the FSM is locked? Safety here is consensus safety; asset-level safety is outside the prototype's scope (Section 8).
RQ2	**Liveness.** When Bitcoin settlement or DA becomes unavailable, does Engram keep committing blocks where strict receipt-gated baselines slow to a fraction of their rate or stop resuming, and what does it give up: how many withdrawals commit before its circuit breaker locks, compared with a baseline that never locks? A degraded peripheral P2P view does not affect a baseline gated on receipts, so it is examined under RQ1.
RQ3	**Hysteresis and recovery.** How much does the FSM's hysteresis reduce flapping, and at what cost in detection delay? Once peripherals recover, do the DA backfill and the recovery proof return the chain to ANCHORED?
RQ4	**Cost.** What overhead do the extended consensus object, certificate validation, and recovery proof add, and where does it come from: proposal size, validation time, or additional consensus rounds?

Headline results. Under a Bitcoin checkpoint pause or a DA outage, Engram kept 96–100% of its healthy block rate, where a strict receipt-gated baseline that reads receipts at each validator kept 20–61% and one that reads them through vote extensions stopped committing 44–121 s after the fault began and did not resume, and 17–58 withdrawal-marked transactions committed before its circuit breaker locked. A composite fault repeated five times ended in a full recovery with identical `AppHash` at all seven validators each time. Against seven vanilla validators on the same hosts, Engram's block interval grows by 1.8% (95% interval 0.4 to 3.1%). Section 7 gives the evidence and its limits, and Table 1 the questions it answers.

The remainder of this paper is organized as follows. Section 2 situates Engram in the literature. Section 3 states the system model, the threat model, and the security goals. Section 4 defines the protocol: sensors, the health certificate, the FSM, proposal validity, and recovery. Section 5 states the security properties and reports the formal specification and bounded model checking. Section 6 describes the implementation, testbed, baselines, and methodology, and Section 7 answers RQ1–RQ4. Section 8 discusses the security envelope, limitations, and future work, and Section 9 concludes.

2. Related Work

A chain whose block validity depends on external layers raises five questions, and each has been studied by a separate community. Fig. 1 organizes the literature by these questions: how consensus behaves under degraded conditions (Section 2.1), how local observations of an external layer become data that all validators see and agree on (Section 2.2), how modular designs cope with the failure of Bitcoin, DA, or settlement layers (Section 2.3), how a chain returns to the anchored state (Section 2.4), and how such designs are validated (Section 2.5). Several building blocks of Engram exist in prior work: vote extensions that carry signed external data, weighted-median aggregation, and protocol-defined fallback for DA and sequencing failures. Engram differs in what the aggregated observation controls. Table 2 shows what the failure of a dependency stops in each design, and Appendix A compares representative systems by trigger, decision authority, behavior under failure, recovery, assumption and reported assurance; Section 2.6 states the combination left open.

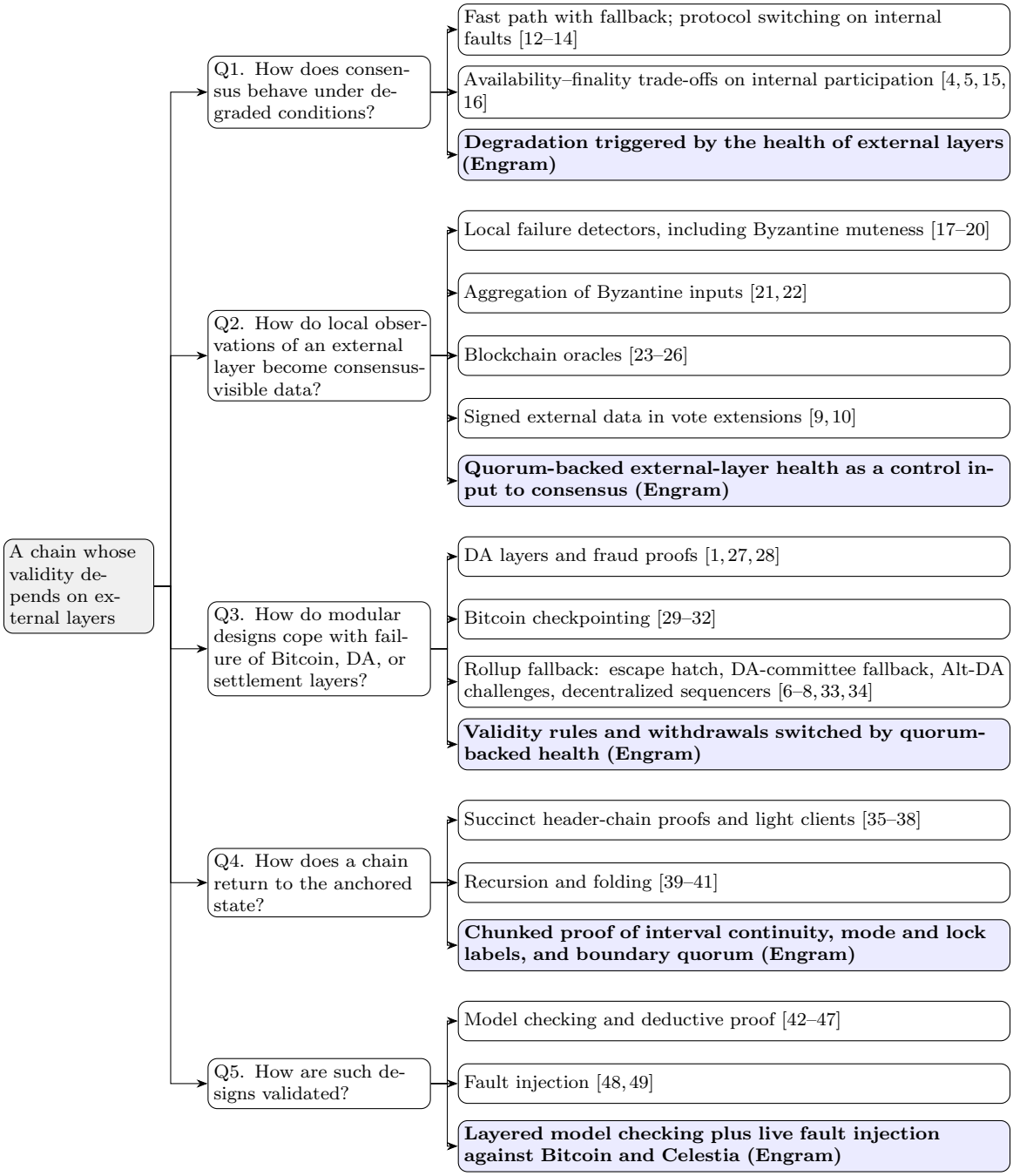


Figure 1: Taxonomy of related work, organized by the question each line of work answers. Shaded leaves mark where Engram sits. Vote-extension oracles supply signed external data to consensus (Q2) but attach no fallback semantics to it.

2.1. Consensus Under Degraded Conditions

Agreement core. PBFT [50] and the partial-synchrony model of Dwork, Lynch, and Stockmeyer [51] underlie Tendermint and its CometBFT successor [2, 3], which provide deterministic finality with a rotating proposer. HotStuff [52] achieves responsiveness and linear communication under a correct leader within the same leader-based lineage. Narwhal/Tusk [53] separate transaction dissemination from ordering, and Bullshark [54] orders transactions on top of the Narwhal DAG. These protocols agree on the content of a proposal; whether a proposal is acceptable is decided by an application predicate that every correct validator must evaluate identically. Tendermint's consensus problem includes such a `valid()` predicate [3], and Cachin et al. [55] introduced the *external validity* condition for multi-valued Byzantine agreement. Engram relies on the same structure: its proposal-validation predicate depends on the health certificate, so the predicate stays deterministic only if the certificate is itself agreed on. BFT forensics [56] studies how many malicious replicas can be identified after a safety violation. AdoB [57] mechanizes benign and Byzantine consensus in one model, and Adore [58] adds certified reconfiguration. Engram inherits CometBFT's evidence pool for accountability and keeps the agreement rules of the core as upstream's, apart from the changes of its fork (Section 6.1).

Fast paths and protocol switching. A second group changes the protocol's behavior in response to conditions that the replicas themselves observe. Abstract [13] lets a replicated state machine abort a client request when something goes wrong, which supports composing and reconfiguring BFT protocols. Thunderella [12] combines an optimistic fast path with a synchronous fallback, and Autobahn [14] targets seamless recovery from network blips. The trigger in each case is a fault of the replicas or of the network among them.

Availability–finality trade-offs. The closest conceptual relatives give up finality to keep the chain growing. The sleepy model [15] lets players be online or offline and gives a consensus protocol that is resilient as long as a majority of the online players are honest. Ebb-and-flow protocols [4] expose both a dynamically available ledger and a finalized prefix, so the chain keeps growing when finality stalls and the finalized ledger catches up on recovery. Casper FFG [16] proves accountable safety and plausible liveness, and Gasper [5], which combines it with the LMD GHOST fork-choice rule, proves safety, plausible liveness and probabilistic liveness. For the case in which more than a third of the validators crash, Casper FFG proposes an inactivity leak that drains the deposits of non-voting validators until the voting validators again form a supermajority, and leaves the exact formula outside its scope, noting that the leak can let conflicting checkpoints finalize without slashing. These designs share the "degrade, do not halt" goal and the tension described by the CAP theorem [59]. They differ from Engram in two respects: the trigger is *internal* participation, and the response changes how finality is reached, not which blocks and withdrawals are valid.

2.2. From Local Observations to Consensus-Visible External Data

Failure detectors. Unreliable failure detectors [17], their QoS metrics [18], and accrual detectors [19] give each process a local, eventually accurate suspicion of other processes. Muteness failure detectors [20] extend the abstraction to Byzantine processes that stop sending the messages an algorithm expects while possibly continuing to send others. Engram's sensors play the same role for peripheral layers, with two differences: a local suspicion has no effect until a quorum of signed observations backs it, and the adversary controls up to f of the reporters.

Aggregating Byzantine observations. Approximate agreement [21] requires non-faulty processes to decide on values that are close together rather than identical. Median validity [22] requires the decision to be close to the median of the non-Byzantine inputs and, at maximal fault tolerance, within the range of those inputs; the authors give a deterministic algorithm for $n \geq 3t + 1$. The Engram health certificate satisfies an analogous range-validity property

for each weighted-median metric. It does not run a second approximate-agreement protocol: validators sign individual height-bound sensor snapshots carried as vote extensions, CometBFT agrees on the proposal that carries them, and every validator then recomputes the aggregate deterministically (Section 4.3).

Blockchain oracles. Oracle research addresses how replicated applications import external information. Town Crier authenticates web data with trusted hardware [23]. A systematization of oracle designs classifies their trust assumptions and manipulation risks [24]. DecenTruth combines multi-source truth discovery with decentralized consensus to tolerate dishonest reports [25], and PSCBO proposes a provably secure consensus-based oracle [26]. In all of these, the oracle supplies a value to the application layer.

Vote extensions. The closest mechanism to Engram's certificate is the ABCI++ vote extension. Each validator extends its precommit with application data, which may be obtained non-deterministically, and the next proposer includes the signed extensions in its proposal. The Cosmos SDK design record requires signatures from at least two thirds of the voting power and uses an oracle price fetcher as its example [9]; Slinky applies the pattern to off-chain price data and aggregates the extensions with a stake-weighted median [10]. Engram uses the same transport: sensor snapshots taken at height H reach the proposer at $H+1$, where their deterministic aggregation forms the health certificate. The difference lies in what the aggregate controls. A price oracle supplies a value to the application; the health certificate describes the infrastructure on which block validity depends and determines the receipt requirements, the withdrawal policy, and the entry into and exit from a degraded mode.

Peripheral P2P attacks. Sybil attacks [60], in which one entity presents multiple identities, and eclipse attacks on Bitcoin's P2P layer [61, 62], which monopolize a node's connections, let an adversary control what a node sees of an external network. They motivate the multi-signal peer-health profile that Engram's P2P sensor uses and are one reason a single local reading is not trusted.

2.3. Fallback for Modular-Chain Dependencies

Data availability. LazyLedger/Celestia [1] decouples DA from execution. Fraud and data availability proofs [27] let light clients detect withheld data, and coded distributed arrays target the peer-to-peer layer that distributes data availability sampling fragments [28]. This line of work makes DA *verifiable*, but a chain that requires a DA inclusion proof before accepting a block still stops when DA stops. Engram adds a consensus-level response to DA unavailability and a backfill requirement before the chain returns to ANCHORED (Section 4.7).

Bitcoin anchoring. Pegged sidechains [63] introduced two-way pegs to Bitcoin. Karakostas and Kiayias [29] study checkpointing as a defense of proof-of-work ledgers against 51% attacks and report a liveness attack, "block lead", against existing checkpointing designs. For proof-of-stake chains, Babylon [30, 32] and Pikachu [31] checkpoint onto Bitcoin to prevent long-range attacks. Tas et al. [32] show that slashable safety is impossible without an external source of trust and state their guarantees under the assumption that Bitcoin is safe and live. Their design also defines a fallback in which Bitcoin orders the PoS blocks after censorship, which strengthens the PoS chain's dependence on Bitcoin. Babylon also specifies a fallback for censored checkpoints and an emergency break, but to our knowledge none of these publications specifies how the chain operates while the anchor itself is unreachable. Engram treats unreachability as an input backed by a quorum of signed observations, keeps ordering blocks, and locks withdrawals while it persists. Accountability comes from CometBFT's equivocation evidence; Engram implements no Bitcoin-enforced slashing.

Rollup fallback and its decision locus. When a rollup operator stops building blocks

or proposing state roots, escape hatches [6] let users withdraw on the L1 by proving their assets against an L2 state root. Arbitrum [64] resolves disputes about contract behavior with a challenge-based protocol on the L1. Surveys of layer-two protocols [65] treat the ability of honest users to withdraw as a core security property, and list among the open weaknesses that layer-two designs assume the underlying blockchain eventually accepts transactions. Capretto et al. [33] give fraud-proof games, mechanized in Lean 4, with which an honest agent can prove on the L1 that a sequencer-and-DAC "arranger" violated a safety property even if all of its replicas are compromised; a liveness violation that preserves safety cannot be proved on the L1. A follow-up decentralizes the sequencer and the DA committee with set consensus [34].

Two designs define a protocol-level response to a DA failure. AnyTrust normally relies on a data-availability committee. If the sequencer cannot collect enough committee signatures within a few minutes, it posts the full data to the parent chain as a non-AnyTrust rollup would do [7]. Withdrawals then incur the delay of a standard rollup, and the chain switches back to the cheaper mode when the committee resumes [66]. The sequencer makes this decision, and the failure it reacts to is a committee failure. OP Stack Alt-DA keeps data with an external DA provider and lets anyone challenge a commitment on the L1. If the data is not posted before the resolve window elapses, derivation reorgs the L2 blocks that depended on it [8]. This design answers the same question as Engram, what happens when an external DA input fails, by resolving the failure on the L1 and discarding the affected blocks.

These designs show that protocol-defined fallback for DA and sequencing failures exists. A rollup that settles its state root on a base layer gives users an exit that does not depend on the operators; Engram instead keeps its own validator set sovereign and trades that exit for liveness, and we do not claim that the trade is preferable for asset custody. They differ from Engram in three respects:

- *Who decides.* A sequencer, a committee, users, or an L1 contract; Engram decides inside the validator set's own consensus instance.
- *What is observed.* A committee or data-posting failure, not the health of Bitcoin settlement or of an external DA network; Engram uses signed observations of all three peripheral layers.
- *What the response changes.* It publishes more data or reorganizes the chain; Engram changes the validity predicate and the withdrawal policy and commits the FSM state in the same block.

Withdrawal risk. Lee et al. [67] break a bridge into its components, identify the risk each introduces, and analyze past exploits that targeted bridges. They report that over USD 1 billion in digital assets was stolen, incorrectly minted, or locked in bridges in the year before their study. Withdrawals are the path through which a chain without external corroboration can lose assets, which is why Engram locks them while the chain runs in SOVEREIGN mode.

2.4. Recovery and Re-Anchoring

Succinct proofs are the standard tool for convincing a verifier of a chain's history without re-executing it. They differ in how a long history is covered. Mina [37] keeps a constant-size proof of the whole chain by recursively composing SNARKs, which Plumo's authors report imposes a large overhead on the protocol being proved [36]. zkBridge [35] verifies source-chain consensus inside a two-layer recursive proof system. Plumo instead syncs a client through a chain of sequential SNARK transition proofs, each covering four months of blockchain history [36]. Recursive composition [39, 40] and folding [41] are the general techniques behind the recursive designs. FlyClient [38] takes a different route for proof-of-work chains: a light client downloads only a logarithmic number of block headers, selected by probabilistic block sampling. All of these

Table 2: What the failure of an external dependency stops, by design (entries from the sources of Table 16, except the two receipt-gated baselines and Engram).

Gate	What stops	Design
Block validity	Every proposal, while the validity predicate cannot be satisfied	In-band and out-of-band gates (this work)
Finality	The finalized prefix lags; the ledger keeps growing	Ebb-and-flow
Client acceptance	A client stops adding blocks whose data behind a signed checkpoint is unavailable	Babylon emergency break
L2 derivation	Derivation stalls until a challenge resolves the data or the blocks are reorged	OP Alt-DA
Oracle input	Nothing: an empty vote extension is valid and the chain keeps running	Vote-extension oracle
An operation	Withdrawals, and the return to the anchored mode; block production continues	Engram

prove that a header chain, or a validator-set transition, is valid. A chain that has run without external corroboration must also show that the interval followed the protocol. The long-range attack literature [68] reviews how an attacker can rewrite the history of a proof-of-stake chain, which is the exposure of an interval secured only by the chain's own validators. Engram's recovery proof is a fixed-capacity PLONK-family [69] proof over one interval chunk, and chunk proofs are chained through an on-chain root, as Plumo chains its transition proofs. The statement differs: besides header continuity it covers the recorded mode and lock-flag label of every header and the quorum signatures of the two boundary blocks, and it does not check transitions against the transition function (Section 4.7).

2.5. Validation Methodology

TLA+ [70] with TLC and Apalache [42] has been used to specify and model-check Tendermint's blockchain synchronization protocol [43]. Deductive approaches establish properties by machine-checked proof instead of by exploring a bounded instance. Ivy [44] verifies safety of infinite-state systems interactively, and Lotan and Shoham [71] automate liveness proofs in first-order logic over finite but unbounded domains. IronFleet [45] and Verdi [46] verify distributed-system implementations, the latter in Coq, and LiDO [47] gives mechanized safety and liveness proofs in Coq for unpipelined and pipelined Jolteon. A recent preprint places these approaches on a maturity scale [72]. Engram sits in the model-checking family, with a per-layer result and an open refinement obligation (Section 5). On the empirical side, Twins [48] tests BFT implementations by duplicating validator identities, and chaos engineering [49] injects faults into production systems. Engram's evaluation combines network-level and application-level fault injection against live Bitcoin and Celestia networks.

2.6. Research Gap

Prior work supplies consensus that adapts to internal participation faults, oracle systems that aggregate external observations, vote extensions that deliver signed external data to the validator pipeline, rollup fallbacks for sequencing and DA failures, and succinct proofs of chain histories. When an external layer fails, a chain that gates validity on it halts, and a chain that drops the dependency gives up the guarantees that motivated it. The fallbacks surveyed above place the decision with users, watchers, a sequencer, a committee, or an L1 contract.

What a response gates. The designs differ in what the failed dependency stops (Table 2). Only the strict receipt-gated baselines stop block commitment. The other designs stop finality, client acceptance, L2 derivation or nothing, or leave the chain running and gate an operation.

Table 3: Notation used in Sections 3–5.

Symbol	Meaning
V, w_i, W	validator set, voting power of validator i, total voting power
$\mathrm{wt}(S)$	total voting power of the validators in S
B, H	Byzantine and honest validators
h, r	consensus height and round
m_i, o_i	local metric vector and signed observation of validator i
C^h, Agg	health certificate of height h, deterministic aggregation
s^h, δ	FSM state committed at height h, transition function
τ_{BTC}, τ_{DA}	receipt tolerance of the Bitcoin receipt (Bitcoin blocks) and of the DA receipt (Engram blocks)

To the best of our knowledge, no published design combines (i) a quorum-backed set of height-bound validator observations from which every validator deterministically derives the health of external layers, (ii) use of that health inside the validator consensus pipeline to change both proposal-validity requirements and withdrawal admissibility while the native validator quorum keeps ordering blocks, and (iii) a recovery condition that requires a proof over the degraded interval and the backfill of that interval to the DA layer before the chain returns to the externally anchored mode. Engram provides these three (with the proof-gate limitation of Section 8.3), retains CometBFT as the agreement protocol, and Sections 3–7 evaluate the combination.

3. System Model and Security Goals

This section fixes who participates, what may fail, what is trusted, and which properties the protocol is meant to give in each FSM state.

3.1. Validators, Network, and Assumptions

Validators and network. We assume partial synchrony: message delay is unbounded before an unknown global stabilization time (GST) and bounded after it [51]. Let V be the validator set, $w_i > 0$ the voting power of $i \in V$, $W = \sum_{i \in V} w_i$ the total voting power, and $\mathrm{wt}(S) = \sum_{i \in S} w_i$ the weight of $S \subseteq V$ (Table 3). The adversary controls a set $B \subseteq V$ of Byzantine validators, which may deviate arbitrarily and collude; the other validators $H = V \setminus B$ are honest. As in CometBFT, a *quorum* is a set whose weight exceeds two thirds of the total:

$$\mathrm{wt}(B) < \frac{W}{3}, \qquad \mathrm{wt}(S) > \frac{2W}{3} \ \text{ for every quorum } S. \tag{1}$$

The seven-validator testbed gives every validator the same power, so Eq. (1) allows $f{=}2$ Byzantine validators and a quorum is any five validators. The adversary cannot forge signatures, find hash collisions, or subvert the UltraHonk backend's KZG setup. After GST, the *core consensus network* connects a quorum, as CometBFT itself requires.

Validator set. The validator set is the genesis set and stays fixed throughout a Sovereign or Recovering interval. Certificate signatures verify against the genesis validator keys, and the compiled recovery circuit fixes the set size at seven. Validator-set changes inside a degraded interval would require proving the set transitions and are outside this paper.

Peripheral layers. Bitcoin, Celestia, and the peripheral P2P interfaces may be slow, unavailable, partitioned from some validators, or adversarially degraded (delayed, censored, eclipsed) for arbitrarily long. They are not assumed to be Byzantine-fault-tolerant from the validator set's perspective, but Bitcoin reorganizations deeper than a configurable depth $K_{\mathrm{DEEP_FINALITY}}$ are excluded, as in every Bitcoin-anchoring design (Section 2). Honest validators' peripheral observations may differ at any instant, and identical views are not assumed. The health certificate

Table 4: Properties by FSM state. P_{agree} holds in every state under $\text{wt}(B) < W/3$. † holds only if the checkpoint commits the chain through its height and the recovery exit gate includes DA backfill (Section 4.7). The prototype implements both; the proof condition of the exit gate is qualified in Section 8.3.

State	P_{agree}	P_{settle}	P_{avail}	**Withdrawals**
ANCHORED	Yes	Yes†	Yes†	Permitted
SUSPICIOUS	Yes	Yes†, delayed	Yes†	Permitted
SOVEREIGN	Yes	No	No (validator block stores only)	Locked
RECOVERING	Yes	Pending recovery proof	Backfilling	Locked

(Section 4.3) gives every validator the same aggregate under arbitrary honest disagreement; whether a sensor reading matches the external layer is a separate question (Section 4.2).

Parameters. FSM thresholds, hysteresis limits, and $K_{\text{DEEP_FINALITY}}$ are genesis parameters shared by every validator, like the block time or quorum size. A validator running different parameters simply fails to validate honest proposals; this affects only its own participation, not safety.

Layered adversary. The adversary may degrade the peripheral connectivity of *every* honest validator at once (delay Bitcoin headers, cut the DA bridge, eclipse the peripheral P2P interface), but it cannot partition the core consensus network below a quorum. Degrading the periphery changes the FSM state; it does not by itself stop block production.

Byzantine power of one third or more. Beyond $W/3$, no BFT protocol retains agreement. In ANCHORED and SUSPICIOUS, a checkpoint that carries the CometBFT block hash lets any Bitcoin observer detect a fork, because two conflicting histories cannot both match a Bitcoin-timestamped hash; this is accountability, not prevention. In SOVEREIGN nothing outside the validator set constrains the chain, which is why withdrawals are locked in that state (Section 3.2).

3.2. Security Goals by Mode

We state what each FSM state guarantees, using five properties of the committed chain $B_1, \ldots, B_h$. A *prefix* $B_1, \ldots, B_c$ is the chain up to height $c \leq h$.

- P_{agree}: no two conflicting blocks are committed at the same height. Assumption: $\text{wt}(B) < W/3$.
- $P_{\text{settle}}(c)$: the prefix up to c cannot be rewritten without a Bitcoin reorganization deeper than $K_{\text{DEEP_FINALITY}}$. Assumptions: such reorganizations are excluded (Section 3.1), and the Bitcoin checkpoint that buries height c commits the chain through c (assumption M2 of Theorem 1, Section 5.1).
- $P_{\text{avail}}(c)$: the block data of every height up to c is retrievable from the DA layer. Assumption: the DA layer is honest and the data was published.
- P_{admit}: no committed block in SOVEREIGN or RECOVERING contains a withdrawal-marked transaction. It is enforced by the circuit breaker.
- P_{assure}: when a withdrawal commits at height h, there are heights c_s and c_a with $P_{\text{settle}}(c_s)$ and $P_{\text{avail}}(c_a)$ (receipt tolerances below). It concludes only for the prefix up to $\min(c_s, c_a)$. It says nothing about state after that prefix, and the prototype does not record which heights a withdrawal depends on.

Table 4 lists which properties hold in each state.

Receipt tolerance. The two receipts are measured on different clocks, so each has its own

tolerance, fixed by the round in which the block commits (Section 4.6). τ_{BTC} bounds, in Bitcoin blocks, how far the checkpoint's Bitcoin height may lag the certified Bitcoin tip b_h. τ_{DA} bounds, in Engram blocks, how far the DA receipt's height may lag h. In ANCHORED and SUSPICIOUS, a valid block at height h whose layers are reported healthy carries a Bitcoin receipt that settles Engram height c_s through a checkpoint at Bitcoin height at least $b_h - \tau_{\mathrm{BTC}}$, and a DA receipt that attests availability up to Engram height $c_a \geq h - \tau_{\mathrm{DA}}$.

Theorem 1 (Section 5.1) states the conditions under which a committed withdrawal satisfies P_{admit} and P_{assure}. Section 4 defines the mechanisms, and Section 5 states and checks the properties.

4. Engram Protocol

The mechanisms follow the order of the data: the architecture, the sensors that produce observations, the certificate that aggregates them, the state machine that consumes the aggregate, the validity predicate that enforces it, and the recovery path out of SOVEREIGN.

4.1. Architecture Overview

Engram is a Cosmos SDK application [73] running on a CometBFT core. Three peripheral layers are monitored: (i) Bitcoin, used as a settlement and finality anchor via periodic checkpoints; (ii) Celestia [1], used as the DA layer; and (iii) the peer-to-peer interfaces through which validators reach the Engram, Bitcoin, and Celestia networks. Engram follows one rule throughout: *sensors observe, the quorum certifies, the state machine decides.* Each validator's sensors read the peripheral layers locally and asynchronously, so two honest validators may observe different values at the same instant. Those readings never enter replicated state directly. Instead, each validator signs its reading as a CometBFT vote extension; the next proposer aggregates a quorum of them into a *health certificate* (Section 4.3); and the FSM transition is a deterministic function of that certificate and the previously committed FSM state (Fig. 2).

4.2. Network Sensors

Sensors are the only non-deterministic component; their outputs enter consensus only through the certificate (Section 4.3).

The definitions of the sensor gaps, their units and the P2P health sensor are in Appendix L.

Agreement versus correctness of health. Lemma 1 (Section 4.3) bounds what Byzantine validators can do to the aggregate; it does not make an honest sensor accurate. The six P2P bounds are a heuristic detector: an eclipse that keeps peer count, subnet diversity, tenure, and latency within bounds while controlling a validator's view is not detected by them. The TLA+ model assumes that an eclipsed honest validator reports a degraded reading and never a healthier one (Section 5); that is a sensor-soundness assumption of the model, not a property derived from the six bounds. A detector that is evaded at every honest validator yields a healthy aggregate and no transition; it does not break agreement.

4.3. Health Certificates

The health certificate turns asynchronous, possibly inconsistent local observations into a single value that every validator can recompute.

Observation. At height $h-1$ and round r, validator i produces an observation $o_i = (i, m_i, \sigma_i)$. Here m_i is its local metric vector (Section 4.2) – Bitcoin gap, anchored and current Bitcoin heights, anchor re-verification result, DA gap and verified height, DA sampling and attestation

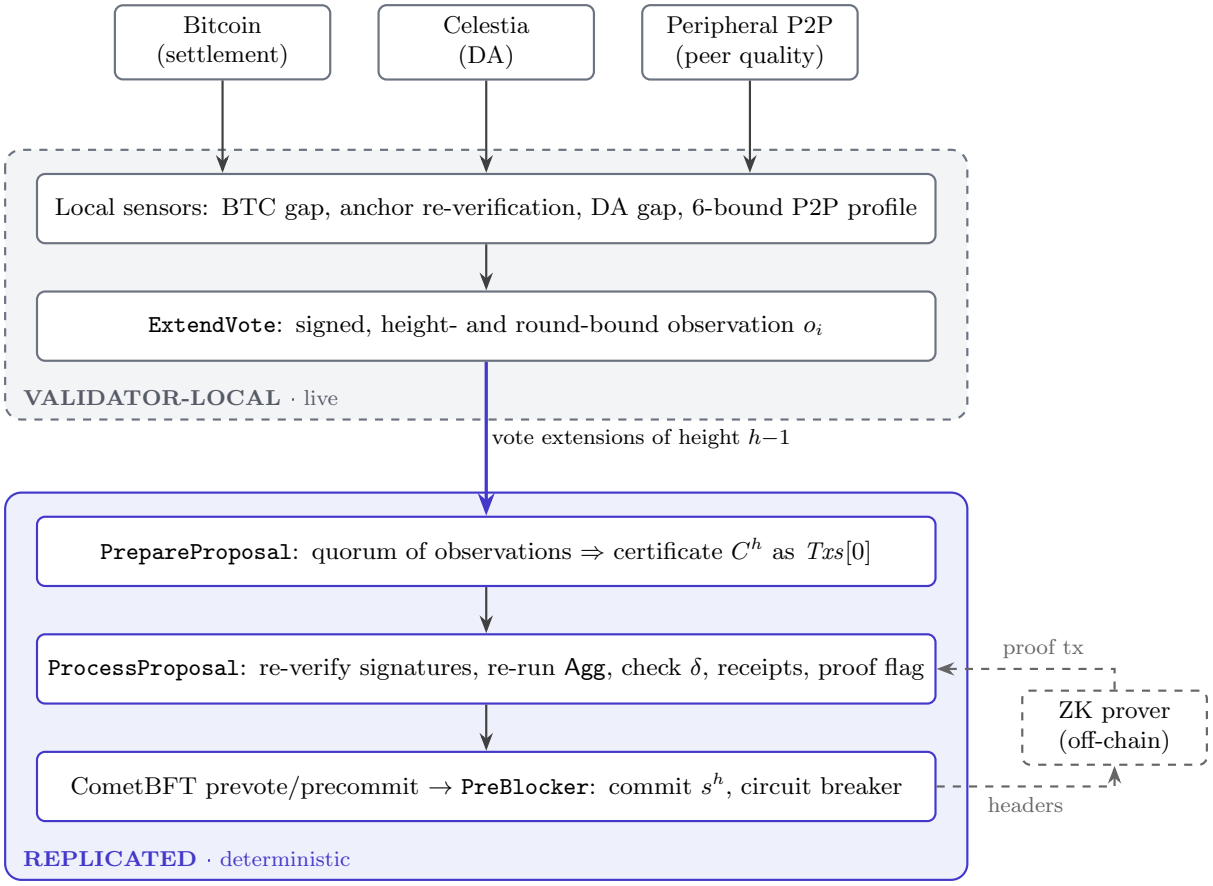


Figure 2: Engram architecture. Live sensor reads stay in the validator-local zone and enter consensus only as signed vote extensions (bold arrow, the single crossing). The replicated zone, from certificate aggregation downward, is a pure function of data every validator holds, so the committed FSM state s^h is deterministic. The dashed path is the recovery proof (Section 4.7).

flags, and the six P2P-quality bounds – and σ_i is i's signature over CometBFT's vote-extension sign bytes $(chainID, h-1, r, m_i)$. The extension is produced in `ExtendVote`, the only ABCI++ hook permitted to read live sensors.

Sensor reads off the critical path. `ExtendVote` does not call the Bitcoin or Celestia services. Two background loops, started at the first extension and run every 0.5 s, read them: one queries the Bitcoin node (reachability, current height, re-verification of the anchored checkpoint) and the Celestia node (probe), the other drives the checkpoint submitter and the DA publisher and collects their state. Each loop stores its latest reading with the local time at which the reading completed, and takes as input the height and the anchored Bitcoin height of the latest `ExtendVote`. `ExtendVote` composes m_i from the stored readings and the local P2P state and signs it, so its duration depends on local work only. A reading that failed, that is older than the freshness window (5 s by default) or that does not exist yet is *unknown*: it is reported as the worst case that an error of the same call already produces (the maximal Bitcoin gap, the DA layer failed), never as the last healthy value, and the validator logs the observation as degraded. The window is a local parameter of the observation and not part of any agreed state. The loops only produce observations; the certificate is aggregated from the signed extensions and the FSM transition is computed from the certificate, and `ProcessProposal` and the aggregation never read a sensor. The earlier build, which reads the services inside `ExtendVote`, remains available for comparison as a run-time mode (`ENGRAM_SENSOR_MODE=sync`), as does a mode that signs a constant healthy observation without any read (`empty`).

Certificate. Let $r^\star$ be the round in which height $h-1$ committed. The proposer of height h

receives the extended commit of height $h-1$ and places

$$C^h = \big(h-1,\ r^\star,\ \{o_i\}_{i\in S}\big)$$

as the first transaction of its proposal. C^h is *well-formed* iff (a) every σ_i verifies against the genesis key of validator i for $(h-1, r^\star)$; (b) each validator contributes at most one observation; (c) every $i \in S$ has a commit precommit in round $r^\star$ of height $h-1$; and (d) $\mathrm{wt}(S) > 2W/3$, so S is a quorum.

Aggregation. For a numeric metric j, the aggregate is the smallest value whose cumulative weight exceeds half of the certificate weight; a Boolean metric is true iff the weight backing true exceeds half of it:

$$\begin{aligned} \widehat{m}_j &= \min\big\{x : \mathrm{wt}(\{i \in S : m_{i,j} \le x\}) > \tfrac{1}{2}\,\mathrm{wt}(S)\big\}, \\ \widehat{b}_j &= \mathbf{1}\big[\mathrm{wt}(\{i \in S : b_{i,j} = 1\}) > \tfrac{1}{2}\,\mathrm{wt}(S)\big]. \end{aligned} \tag{2}$$

$\mathsf{Agg}(C^h)$ collects these values. The Boolean metrics are failure flags (failed anchor re-verification, failed DA sampling, failed DA attestation), so a tie leaves the flag unset. Agg is a pure function: every validator that receives C^h computes the same value without contacting any peripheral network. The TLA+ model gives every validator the same voting power (Section 5).

Lemma 1 (honest-bounded aggregate). *Let C^h be well-formed and $\mathrm{wt}(B) < W/3$, where B is the set of Byzantine validators and H the set of the others. For every numeric metric j,*

$$\min_{i\in S\cap H} m_{i,j} \ \le\ \widehat{m}_j \ \le\ \max_{i\in S\cap H} m_{i,j},$$

and every Boolean metric on which all members of $S \cap H$ agree takes that value.

Proof. Write $w = \mathrm{wt}(S)$. By (d), $w > 2W/3$, and $\mathrm{wt}(S \cap B) \le \mathrm{wt}(B) < W/3 < w/2$. Hence $\mathrm{wt}(S \cap H) = w - \mathrm{wt}(S \cap B) > w/2$, and $S \cap H$ is not empty. Let a and b be the smallest and the largest value of metric j among the members of $S \cap H$, and let $F(x) = \mathrm{wt}(\{i \in S : m_{i,j} \le x\})$, so that $\widehat{m}_j$ is the least x with $F(x) > w/2$ (Eq. (2)); such an x exists because F reaches w at the largest reported value. Every member of $S \cap H$ reports at most b, so $F(b) \ge \mathrm{wt}(S \cap H) > w/2$ and $\widehat{m}_j \le b$. For $x < a$ no member of $S \cap H$ reports at most x, so $F(x) \le \mathrm{wt}(S \cap B) < w/2$; no $x < a$ satisfies the condition and $\widehat{m}_j \ge a$. For a Boolean metric on which all members of $S \cap H$ report v: if $v = 1$, the weight backing true is at least $\mathrm{wt}(S \cap H) > w/2$; if $v = 0$, it is at most $\mathrm{wt}(S \cap B) < w/2$. In both cases $\widehat{b}_j = v$. □

This is the range form of median validity [22] for weighted observations. The bound on $\mathrm{wt}(B)$ is the largest the quorum rule supports: the proposer chooses S, and once it omits enough honest observations that w is just above $2W/3$, a coalition of weight just above $W/3$ exceeds $w/2$ and the aggregate can leave the honest range. The unequal-power test of Section 7.1 found such coalitions at 34 or 35 out of 100.

Corollary 1 (threshold decisions). *Under the hypotheses of Lemma 1, let θ be a threshold on metric j. If every member of $S \cap H$ reports $m_{i,j} > \theta$, then $\widehat{m}_j > \theta$; if every member reports $m_{i,j} \le \theta$, then $\widehat{m}_j \le \theta$.* Both follow from $a \le \widehat{m}_j \le b$. A degradation that every honest validator in S observes therefore moves the certified aggregate across the threshold, and a weight below $W/3$ cannot move it across when no honest validator in S observes the degradation.

The limits of the lemma, the adversarial cases and the binding of the certificate to height and round are in Appendix L.

Table 5: FSM states and their validity requirements. The exit gate is defined in Section 4.7.

State	Validity requires	Withdrawals	Forced-tx admission	BTC settlement
ANCHORED	Quorum; BTC and DA receipts	Permitted	Full	Yes (K_{DEEP} conf.)
SUSPICIOUS	Quorum; receipt of each layer reported healthy	Permitted	Rate-limited	Yes, delayed
SOVEREIGN	Quorum; receipt of each layer reported healthy	Locked	Full	No
RECOVERING	Quorum; BTC and DA receipts; exit gate	Locked	Full	Pending proof

CometBFT's deterministic finality is unaffected by the FSM state; the measured commit interval on our testbed is about 1–2 s in every state. Bitcoin settlement is probabilistic and follows Bitcoin's own confirmation time (K_{DEEP} blocks, ∼10 min per block on mainnet); it is not measured in this paper.

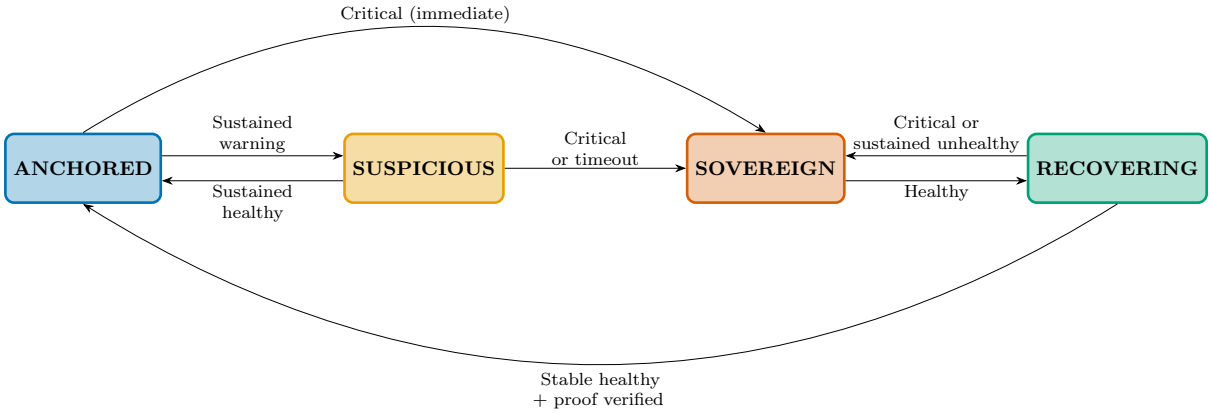


Figure 3: FSM transition function δ. *Sustained* means the condition accumulates over several blocks (hysteresis; up-counters tolerate isolated non-healthy blocks); critical conditions skip the wait and act immediately. The waits are given in Table 46.

4.4. Finite State Machine

The FSM has four states (Table 5). ANCHORED is normal operation: blocks are valid only with a Bitcoin receipt and a DA receipt within tolerance, and withdrawals are permitted. SUSPICIOUS is an early-warning state reached after a sustained warning; withdrawals remain permitted, but admission of forced transactions is rate-limited by a bounded queue. SOVEREIGN is the fallback: the Bitcoin and DA receipt checks are relaxed for each layer that the certificate does not report healthy, blocks continue under the validator quorum, and the circuit breaker locks all cross-chain withdrawals. RECOVERING is entered once the certificate reports healthy peripherals; withdrawals stay locked until the exit gate of Section 4.7 is satisfied.

Fig. 3 shows the transition function δ. A proposal carries a target state s^h; honest validators prevote for it only if

$$s^h = \delta\big(s^{h-1}, \mathsf{Agg}(C^h)\big), \tag{3}$$

where δ also reads the hysteresis counters committed with s^{h-1} (Section 4.5). Since Agg and δ are deterministic and the state and counters are committed, every honest validator reaches the same verdict on a given proposal, and the committed state inherits CometBFT's agreement guarantee. δ evaluates the conditions in a fixed priority: a critical condition first, then a warning (in RECOVERING, any non-healthy reading), then a healthy reading. The first condition that applies decides, so a critical condition overrides every hysteresis counter.

The wait parameters of the FSM (Table 46) are in Appendix L.

4.5. Transitions and Hysteresis

A *warning* (suspicious Bitcoin gap, unhealthy DA, or unhealthy P2P quality) demotes ANCHORED to SUSPICIOUS only after $k_{\downarrow}$ consecutive non-healthy blocks. A *critical* condition (excessive Bitcoin gap, failed anchor re-verification, complete anchor-peer isolation, or a SUSPICIOUS timeout of T_{sus} blocks) moves the chain to SOVEREIGN immediately and bypasses every counter. Recovery is deliberately slower than degradation: SUSPICIOUS→ANCHORED requires $k_{\uparrow}$ consecutive healthy blocks, and RECOVERING→ANCHORED requires the safe-block counter to reach H plus the exit gate of Section 4.7. SOVEREIGN→RECOVERING needs only a healthy certificate.

The down threshold $k_{\downarrow}$ has an optional backoff against an adversary who repeatedly toggles a peripheral. It is an extension that we evaluate in Section 7.3 and do not claim as a contribution: it did not outperform a static threshold of 4 in our live runs. With the cap equal to the initial value $k_{\downarrow}$ is static. With backoff, it starts at a genesis value and doubles per flap level, up to a genesis cap, and both demotion edges (ANCHORED→SUSPICIOUS and the non-critical RECOVERING→SOVEREIGN) use it. The flap level rises by one for each such demotion until the cap is reached, falls by one after T_{stable} healthy ANCHORED blocks, and is not reset by a successful recovery.

The counters follow the committed state. The unhealthy streak counts non-healthy blocks absorbed in ANCHORED or RECOVERING; it resets on any transition and on a healthy block. The safe-block counter in RECOVERING rises on healthy blocks up to H and falls by one on an absorbed non-healthy block; the corresponding SUSPICIOUS counter behaves the same way up to $k_{\uparrow}$. Any other transition resets both. Section 7.3 evaluates all three hysteresis-gated edges.

4.6. Extended Proposal and Validity Predicate

A proposal P_h at height h carries, in $Txs[0]$, the certificate C^h, the target state s^h, a health flag, a DA receipt (last DA-verified height and attestation flag), a Bitcoin receipt (checkpoint height and block hash), and, when applicable, a recovery-proof reference.

Definition 1 (Engram proposal validity). `ProcessProposal` accepts a proposal P_h iff

$$\mathsf{Valid}_h(P_h) \iff \mathsf{WellFormed}(C^h) \wedge \mathsf{StateOK} \wedge \mathsf{ReceiptsOK} \wedge \mathsf{WithdrawalsOK} \wedge \mathsf{RecoveryOK}, \quad (4)$$

where

- $\mathsf{WellFormed}(C^h)$: C^h satisfies conditions (a)–(d) of Section 4.3;
- $\mathsf{StateOK}$: $s^h = \delta(s^{h-1}, \mathsf{Agg}(C^h))$ and the health flag matches $\mathsf{Agg}(C^h)$;
- $\mathsf{ReceiptsOK}$: in ANCHORED and RECOVERING, and for any layer the certificate reports healthy, the DA and Bitcoin receipts are consistent with $\mathsf{Agg}(C^h)$ within the round-dependent tolerances τ_{BTC} and τ_{DA} defined below;
- $\mathsf{WithdrawalsOK}$: no withdrawal is included while s^h is SOVEREIGN or RECOVERING;
- $\mathsf{RecoveryOK}$: the proposal carries a recovery-proof reference with a fresh DA attestation exactly when it targets RECOVERING with the safe-block counter at H, and the exit RECOVERING→ANCHORED is taken only on a proof the chain has verified (Section 4.7).

`ProcessProposal` never reads a live sensor. The FSM state is written to replicated state only by `PreBlocker` after commit, from the certificate already agreed upon.

The tolerances are deterministic functions of the round r and genesis parameters, so every validator computes the same bounds. Each is a base threshold plus a widening w_r that is capped. A DA receipt may lag the Engram height by $\tau_{\text{DA}} = \Theta_{\text{DA}} + w_r^{\text{DA}}$ Engram blocks, where Θ_{DA} is the DA threshold and $w_r^{\text{DA}} = 0$ for $r \leq 1$, 2 for $r = 2$, and 4 for $r \geq 3$. A Bitcoin

receipt's checkpoint height may lag the Bitcoin tip by $\tau_{\mathrm{BTC}} = K_{\mathrm{DEEP_FINALITY}} + 2 + w_r^{\mathrm{BTC}}$ Bitcoin blocks, where $w_r^{\mathrm{BTC}} = 0$ for $r \leq 2$ and 1 for $r \geq 3$. The base $K_{\mathrm{DEEP_FINALITY}} + 2$ is the depth a confirmed checkpoint already lags the tip plus two blocks of propagation slack. Both widenings are capped, so $\tau_{\mathrm{BTC}} \leq K_{\mathrm{DEEP_FINALITY}} + 3$ Bitcoin blocks and $\tau_{\mathrm{DA}} \leq \Theta_{\mathrm{DA}} + 4$ Engram blocks at every round: a height that skips many rounds does not admit a staler receipt than one that skips three. Theorem 1 takes both tolerances at the round in which B_h committed. The widenings never decrease with the round, so a block that satisfies the receipt conditions at round r satisfies them at every later round. The round dependence therefore departs from Tendermint's round-independent validity predicate only in liveness and cannot make two honest validators commit different values; we do not re-prove Tendermint's liveness for it.

ReceiptsOK is exactly what the strict receipt-gated baseline (Section 6.2) enforces unconditionally. Engram's only liveness-relevant change is to make it conditional on the certified state: when the certificate reports a layer unhealthy outside ANCHORED and RECOVERING, that layer's receipt is no longer required, and the circuit breaker (WithdrawalsOK) removes the operations whose safety depended on it.

4.7. Recovery Proof and Re-anchoring

Normative protocol. The exit defined here assumes proof-only recovery: it requires a recovery proof that the chain has verified. The prototype also contains an optional engineering heuristic that can satisfy the proof condition; Section 8.3 identifies it.

Statement. On leaving SOVEREIGN, a prover builds a Noir circuit proof (Barretenberg UltraHonk backend [69,74]) over a contiguous range of $n \leq N_{\max} = 256$ headers starting from the last anchored root rt_{last}. A header is the tuple $(\mathit{prev_hash}, \mathit{fsm_state}, \mathit{withdrawal_locked}, \mathit{state_root})$, where *state_root* is CometBFT's application hash for that block. The circuit checks five conditions. (a) Each header's *prev_hash* is the Poseidon2 hash of its predecessor. (b) Every header records *fsm_state* in {SOVEREIGN, RECOVERING}. (c) Every header records *withdrawal_locked* set. (d) The first header links to rt_{last}, and the hash of the last header equals the claimed rt_{new}. (e) The first and the last block of the range carry ed25519 precommit signatures (in the round in which that block committed, a private input) from at least two thirds of the voting power of a validator set whose hash is a public input (the compiled circuit fixes the set size at seven). Unused slots are padding, so proving time, verification time, and proof size are constant in n (Appendix C). In short, the circuit proves header-chain continuity and root binding (a, d), mode-label and lock-flag consistency (b, c), and boundary quorum certification (e). It does not check each transition against δ, and the lock flag is the one the chain recorded in the header.

The intended verifier of the recovery proof is described in Appendix L.

Which party verifies what in the proof message is described in Appendix L.

Proof timing. The proof message executes in block h and latches the height of the new checkpoint. The `PreBlocker` of block $h{+}1$ recomputes the proof-validity flag from that height, and `ProcessProposal` reads this committed flag when it evaluates the exit edge, so RECOVERING→ANCHORED cannot occur before block $h{+}2$. `ProcessProposal` does not verify the proof itself.

Order of the checks in the message handler. The handler of a recovery proof compares the public inputs with the chain's own record before it verifies anything: the length of the public inputs, rt_{last} against the tracked checkpoint root, rt_{new} against the accumulator of a recorded header, the count against the difference of heights, the first height against the last anchored height plus one, the hashes of the first and the last boundary block and the validator-set hash against the recorded values. Only a proof that passes them reaches the UltraHonk verification (about 20 ms), and at most eight verifications are made per block; further proofs of that block

are refused without being verified. The transaction carries no fee or signature, so this limit, and not a price, bounds the verification work that a sender can cause. The validator logs which check refused a proof.

What the recovery proof establishes. The recovery proof shows, to a verifier that also checks its public inputs against its own record, as the validators' message handler does, that (i) the recorded headers of the SOVEREIGN interval form a single hash chain whose end points are two boundary blocks signed by at least two thirds of the voting power, and (ii) every recorded state is SOVEREIGN or RECOVERING with the lock flag set. Because the lock flag is the chain's own record, (ii) does not prove that no withdrawal executed. The proof does not make the interval's *ordering* secure against a Byzantine supermajority: during SOVEREIGN, safety rests on the validator quorum alone, which is precisely why withdrawals are locked. Re-anchoring restores Bitcoin timestamping from the recovery point forward.

Four points bound how much the proof contributes. (1) Nothing in the circuit is secret: the headers are public and the proof is used as a succinct argument, not for zero knowledge. (2) The round in which each boundary block committed is a private input of the circuit and enters the signed precommit bytes, so a block that committed in round 1 or 2 is witnessable (commit `6e9f7a5`; the earlier circuit fixed round 0). The circuit accepts synthetic precommits signed in rounds 0, 1 and 2 and rejects a witness that claims another round, a tampered boundary hash, a tampered validator-set hash and a signing power below two thirds (eight cases of a script and ten `nargo` tests, all as expected). On Testbed B recoveries whose last boundary block had committed in round 1, and in round 2, were accepted by the chain in two runs each (Section 7.3). (3) The boundary check shows that validators holding more than two thirds of the genesis power signed the block, so it is as sound as the assumption of at most $f{=}2$ faulty validators; the circuit is fixed to the seven-validator genesis set, to voting power below 128 per validator and to one compiled chain identifier, and any change of set or identifier needs a new circuit and verification key. (4) The circuit does not relate the headers to the signed boundary blocks beyond the boundary hashes; the comparison of the public inputs with the chain's own record, made by every validator, supplies that link. For the validators the proof is therefore a consistency check on a record they already hold; its security contribution would materialize for a verifier that does not replay the chain, such as a light client or a bridge, which we neither build nor evaluate. Soundness also assumes that nobody knows the trapdoor of the Aztec KZG setup over BN254 and the random-oracle model for Fiat–Shamir; a forged proof could satisfy the proof gate but not the native comparison of public inputs with chain state. The message handler compares the public inputs with the chain's record first and verifies the proof (about 20 ms) only afterwards, and verifies at most eight proofs per block; the prototype has no sender authentication or fee on proof messages, and we did not measure this channel.

Data availability of the Sovereign interval. A recovery proof shows the interval is recorded consistently, not that its data is *available*. The exit condition is therefore stronger than the proof: RECOVERING→ANCHORED requires (1) a recovery proof that the chain has verified and (2) a DA backfill certificate – a statement, backed by the quorum of signed observations in the health certificate, that the block data for every height of the SOVEREIGN interval has been published to the DA layer and is retrievable. During SOVEREIGN, each validator keeps the interval's blocks in its own store (they are ordinary committed CometBFT blocks) and queues them for publication; on recovery the queue is drained in height order. Each validator reports a contiguous height: the highest height such that every height up to it has been confirmed retrievable. The certificate takes the weighted median $\widehat{d}$ of these reports. Let e be the height at which RECOVERING was entered, and let safe^{h-1} and proof^{h-1} be the committed safe-block counter and proof-validity flag. The exit edge is taken at height h iff

$$\text{healthy}\big(\mathsf{Agg}(C^h)\big) \;\wedge\; \text{safe}^{h-1} = H \;\wedge\; \text{proof}^{h-1} \;\wedge\; \widehat{d} \geq e - 1. \tag{5}$$

The DA gap sensor keeps measuring head liveness, so the DA layer stays healthy while the backfill drains. The specification abstracts the backfill condition as a Boolean, `da_backfill_valid`. The prototype implements the queue and the gate; Section 8.4 describes the remaining live validation.

5. Security and Formal Analysis

This section states the properties that follow from the mechanisms of Section 4 under the assumptions of Section 3, and reports what bounded model checking establishes about them.

5.1. Analytical Properties

Security of mode switching. A transition happens only when a well-formed health certificate implies it (Lemma 1, Section 4.3). Byzantine validators below $W/3$ cannot move a numeric aggregate outside the range of the honest observations in a well-formed certificate, and cannot overturn a Boolean value on which all represented honest validators agree. When honest observations straddle an FSM threshold, the proposer's choice of quorum subset can still influence which side of the threshold the aggregate falls on; hysteresis limits this influence but does not remove it. The certificate therefore guarantees honest-range validity. It is not an unbiased estimate of the state of the peripheral layer, and it carries no information about that state beyond what the honest validators observed. The second attack surface is griefing by an adversary that degrades the peripheral view of honest validators (Section 8).

Theorem 1 (Withdrawal Admission Safety). *Assume* $\mathrm{wt}(B) < W/3$*, no Bitcoin reorganization deeper than* $K_{DEEP_FINALITY}$*, and an honest DA layer. Assume further that (M1) honest validators reject a block that contains a withdrawal while the state is* Sovereign *or* Recovering*; (M2) each Bitcoin checkpoint commits the chain through its height, and with it the replicated state; and (M3)* Recovering→Anchored *requires a recovery proof that the chain has verified, covering the* Sovereign *interval except at most* T *trailing blocks (the unproven tail,* `MaxUnprovenTailBlocks`*; the heuristic that can stand in for the proof is disabled), and a DA backfill certificate for the whole interval. If a withdrawal commits in block* B_h*, then* P_{agree} *and* P_{admit} *hold (the state at* h *is* Anchored *or* Suspicious*), and* P_{assure} *holds: there are Engram heights* c_s *and* $c_a \geq h - \tau_{DA}$ *such that* $P_{settle}(c_s)$ *and* $P_{avail}(c_a)$ *hold, where the checkpoint that settles* c_s *lags the certified Bitcoin tip by at most* τ_{BTC} *Bitcoin blocks. Every height up to* $\min(c_s, c_a)$ *is settled and available. A* Sovereign *interval is covered only to the extent that this prefix reaches it.*

Proof. P_{agree} follows from $\mathrm{wt}(B) < W/3$. By (M1), P_{admit} holds and the state at h is Anchored or Suspicious. In the transition function δ, Suspicious is entered only from Anchored, and Sovereign is left only through Recovering; so every Sovereign interval before h ended with a Recovering→Anchored transition, which by (M3) required a verified recovery proof and a DA backfill certificate. Hence the interval is hash-linked and available, and all but at most T of its trailing blocks are covered by the proof; those last blocks are certified by the validator quorum alone (Section 8.3). In the Anchored and Suspicious blocks since then, every block carries receipts within τ_{BTC} and τ_{DA}. The Bitcoin receipt in B_h attests a checkpoint at depth at least $K_{\text{DEEP_FINALITY}}$ for some c_s; by (M2) it commits the chain through c_s, so rewriting the prefix up to c_s requires a deeper reorganization, which is excluded. The DA receipt in B_h attests data availability up to some $c_a \geq h - \tau_{\text{DA}}$, and the DA layer is honest, which gives P_{assure}. □

Scope of Theorem 1. The theorem is a safety property of the committed history: it constrains which blocks may contain a withdrawal. It does not bound what a withdrawal depends on. Settlement is guaranteed only through c_s and availability only through c_a, whereas a withdrawal may depend on state created after them, including state inside a Sovereign interval that the

Table 6: Per-layer, non-refinement verification results (TLC)

Layer / property	States gen.	Distinct	Depth	Time	Workers	Viol.
Tendermint (safety)	3,668,099	521,958	25	1 m 13 s	10	0
FSM (safety)	642,091,376	2,126,591	16	4 m 50 s	10	0
FSM (liveness)	17,977,073	31,105	25	2 m 09 s	10	0

Each row is one completed TLC run of that layer's driver. Depth is the number of breadth-first levels at which the search terminated with every state satisfying the state constraint explored, so it is the diameter of the constrained state graph and not a cap. Bounds: N=4, f=1, two rounds and heights below 10 for Tendermint; the thresholds of Appendix E for the FSM. Hardware: Apple M1 Pro, 10 cores, 32 GB RAM.

checkpoint does not yet cover. The prototype records no dependency height for a withdrawal, so the theorem separates admission (P_{admit}) from assurance about a prefix (P_{assure}); it is not a guarantee of asset safety. Asset-level safety would add the condition that every withdrawal depends only on state at heights up to $\min(c_s, c_a)$; the theorem does not establish it. Liveness comes from relaxing the receipt checks in Sovereign; the property a bridge would need comes from tying withdrawals to the settled and available prefix.

The prototype realizes the three assumptions as follows:

- (M1) is enforced in the validity predicate and the ante handler.
- (M2) holds because each checkpoint payload carries the CometBFT block hash at its height c. That hash commits the header of B_c, hence the chain of blocks through c by the hash links, and the header carries the application hash of the state before B_c. The checkpoint therefore commits that state and the transactions of B_c, and, since execution is deterministic, the state after B_c.
- (M3) is enforced on the exit edge by the backfill condition, which the FSM safety run checks as an action property, and by the proof condition (qualified in Section 8.3).

The theorem is a pen-and-paper argument over these assumptions, not a mechanized statement about the implementation.

5.2. Formal Model

Model-checking consensus, FSM, and sensors together is intractable at useful scale. We therefore specify the protocol in TLA+ [70] as four layers – an abstract consensus object, a Tendermint-style consensus layer, the FSM layer, and a server layer composing them – and check each layer with TLC. Apalache [42] is used for the certificate lemma, whose observation vector is symbolic; every row of the ablation study is checked with TLC. The target property is: under partial synchrony and an f-bounded adversary, no two honest validators commit conflicting blocks or conflicting FSM states, and any such conflict implies double-signing evidence.

The separation of safety from liveness in the formal model is explained in Appendix L.

5.3. Model-Checked Properties and Attack Resilience

The model-checked properties and attack resilience are described in Appendix L.

5.4. Bounded Results and Mechanism Ablation

Table 6 reports the per-layer results. Each row is a run of the layer's own driver at the configuration of Table 20, and none found a violation in its bounded state space. The FSM safety run, repeated with a different number of workers, gave identical counts. The runs cover the Tendermint-style and FSM layers. The abstract layer is exercised only through the open

Table 7: Ablation study: removing one mechanism at a time

Removed	Targeted threat	Tool, spec	Depth	Time	Violated property
Circuit breaker	Withdrawal while locked	TLC, current spec	5	2 m 03 s	`Sanity_NeverAttempt WithdrawalWhileLocked`
Hysteresis	Premature re-anchoring	TLC, current spec	9	21 s	`Sanity_NoIllegal HysteresisExit`
P2P health gate	False recovery under eclipse	TLC, current spec	5	1 s	`Sanity_NoIllegal P2PRecovery`
DA consistency	Data withholding	TLC, current spec	4	4 s	`Sanity_Never ProposeWithheldData`
DA backfill gate	Exit from RECOVERING before the interval's data is republished	TLC, current spec	10	49 s	`Sanity_NoIllegal BackfillExit`

refinement obligation, and the server layer through that obligation, the certificate lemma and two ablations. The ablation study spans two layers: the hysteresis, P2P health gate and DA backfill gate belong to the FSM layer, and the circuit breaker and DA consistency to the server layer, so each result holds within its own layer's configuration.

Table 7 reports the ablation study. Each row removes one mechanism from the specification of its layer and checks the sanity property that the mechanism protects. TLC finds a violation in every case, and the counterexample is the shortest trace to it, since the search is breadth-first. Without the circuit breaker a withdrawal can be proposed in a locked state; without hysteresis the chain re-anchors prematurely; without the P2P gate an eclipsed validator forces an illegal recovery; without DA consistency a withholding proposal is accepted; and without the backfill gate the chain leaves RECOVERING before the interval's data is republished. Two further ablations are not part of this safety table: removing the $f+1$ fast-forward is a liveness ablation and has not completed, and removing the Byzantine bound of Lemma 1 is reported with that lemma (Section L). Single-factor ablation does not test joint removal, so it does not establish that the set of mechanisms is sufficient.

5.5. Cross-Layer Refinement and Verification Scope

The evidence is per-layer bounded model checking, not an end-to-end proof. The Tendermint-style layer (safety) and the FSM layer (safety and liveness) are each checked standalone, exhaustively within a stated bounded configuration (Table 6); the ablation study removes one mechanism at a time (Table 7); and the certificate lemma is checked symbolically with Apalache. The abstract and server layers are not checked standalone. What would transfer the target property to the composed system is the obligation that the server layer refines the abstract consensus object. It is specified at three scales (C1: N=4, f=1; C2: deeper rounds; C3: N=7, f=2) but has not converged at any of them, so the composed system is *not* formally verified end-to-end; its whole-system evidence in this paper is empirical (Section 7).

Why the refinement of the composed layer is not claimed is explained in Appendix L.

What is and is not claimed. We claim that the Tendermint-style and FSM layers satisfy the properties of Table 6 within the stated bounds, that each mechanism of Table 7 is needed for the property it protects when removed on its own, and that the certificate lemma holds at N=4, f=1. We do not claim refinement of the server layer to the abstract object, safety of the composed system, consensus liveness after GST, or that the set of mechanisms is sufficient.

6. Implementation and Evaluation Methodology

6.1. Prototype and Testbed

The reference implementation is a Cosmos SDK application, `engramd`, on a forked CometBFT core. The application mounts no `x/auth`, `x/bank`, or `x/staking`: transactions are unsigned envelopes and the validator set comes from the genesis file. Three modules hold the protocol logic: `x/sovereignty` (the FSM, the sensors, the circuit breaker, and the ABCI++ hooks), `x/da` (the Celestia receipt and its verification), and `x/anchor` (the Bitcoin settlement receipt and its re-verification). The ABCI++ hooks follow the pipeline of Section 4.1: only `ExtendVote` reads live sensors, and `PreBlocker` commits the FSM transition from the agreed certificate.

The fork differs from upstream CometBFT in four ways: an $f+1$ timeout round-skip, per-peer round-trip-time measurement and a peer health monitor that feed the P2P sensor, per-peer rate limiting and bounded round acceptance for timeout messages, and the proposal round passed to the application through `PrepareProposal` and `ProcessProposal`.

Bitcoin and Celestia are live dependencies, deployed as described in Section 6.1. The recovery proof is a Noir circuit proved with Barretenberg (Section 4.7); a prover process watches for a SOVEREIGN backlog, builds the proof from committed headers, and submits it as a transaction that every validator verifies during block execution. Section 6.1 lists the versions.

Testbed.

The testbed runs the reference implementation of Section 6.1 as seven validators with equal voting power (n=7, f=2), each in its own container. Validator gossip uses a dedicated full mesh of pairwise links, separate from the network that carries Bitcoin, Celestia, and RPC traffic; this separation lets us degrade peripheral connectivity without partitioning the core consensus network. The peripheral layers are real software, not simulators: a two-node Bitcoin regtest network mined continuously at roughly one block per 20 s (thirty times faster than Bitcoin's 600 s average; the thresholds are in Bitcoin blocks, so detection times measured here convert to mainnet as in Section 8), and a Celestia network with seven bridge nodes, one per validator, so that each validator forms its own DA view. The single-actor attacks of Appendix H and the f=2 collusion suite of Section 7.1 ran on the seven-validator cluster.

The Celestia setup is described in Appendix L.

Environment and versions.

Most results come from Testbed A: the seven-validator Docker deployment above on a single host. Unless stated otherwise, the live tables and figures of Section 7 are Testbed A measurements; the in-process simulations, the microbenchmarks and the offline prover measurements of Appendix D are labelled as such. A second environment, Testbed B, serves the experiments that depend on real wide-area conditions: seven validators on seven virtual machines in three regions (us-east-2, eu-west-1 and ap-southeast-1), the Bitcoin regtest node and `celestia-app` alone on a machine in a fourth region (us-west-2), and the prover host with its worker machines in a fifth (us-east-1). Round trips between regions are 46–207 ms, and the peer-latency bound of the health sensors is raised to 500 ms (Table 22). Results from the two environments are never pooled into one number. The load comparison, the composite faults, the hung-RPC, freshness-window and added-delay tests and the recoveries at boundary rounds 0, 1 and 2 ran on Testbed B, on a freshly deployed chain for each run or build, with proof-only recovery and a tail bound of 128; the load generator was the operator's computer outside the cloud, and Appendix F gives the commit and the number of runs of each. Table 21 in Appendix F lists the configuration and the versions.

6.2. Baselines

Strict receipt-gated baseline (primary, RQ2). The same binary with the FSM pinned to ANCHORED (the command-line mode is called `hard-gated`). Condition (3) of the validity predicate (Section 4.6) is enforced at every height, so a proposal is valid only if it satisfies the agreement checks, carries a Bitcoin receipt within tolerance, and carries a DA receipt within tolerance: $V_h(B) = V_{\mathrm{BFT}}(B) \wedge R_{\mathrm{BTC}}(B,h) \wedge R_{\mathrm{DA}}(B,h)$, with no other validity path while a receipt predicate cannot be satisfied. It is the behavior of a chain that makes settlement and DA inclusion part of block validity, it is a construct of this paper and not a published system, and it isolates the effect of Engram's adaptive relaxation. It is a runtime mode of the same binary (`ENGRAM_MODE=hard-gated`); Section 7.2 runs the RQ2 fault schedule against it. A chain in this mode cannot start from genesis, because the first block needs a DA receipt that does not exist yet; every trial therefore starts the cluster as Engram, waits until all validators report ANCHORED with healthy sensors, waits until the Bitcoin height gap is at most three, and restarts the validators in the receipt-gated mode with their state kept, with the Bitcoin miner paused during the restart.

The definition of the out-of-band baseline is in Appendix L.

Soft baseline (RQ2). The real FSM runs, but withdrawals are never locked in SOVEREIGN or RECOVERING (`ENGRAM_MODE=soft`). It keeps liveness and drops the circuit breaker, which isolates what the withdrawal lock contributes.

The vanilla control and the ablation baselines are defined in Appendix L.

6.3. Fault Model, Schedules and Parameters

The fault schedules and the fault model are in Appendix L.

FSM parameters.

The parameters are fixed in the genesis file, so every validator holds identical values; Table 23 in Appendix F lists them. The live experiments use a Bitcoin gap of 5 blocks for SUSPICIOUS and 8 for SOVEREIGN, a DA gap of 30 blocks, a finality depth $K_{\mathrm{DEEP_FINALITY}}$=2, hysteresis waits of 2 blocks, a down threshold $k_{\downarrow}$=2, a SUSPICIOUS escalation bound of 24 blocks, and a recovery gate that accepts only a verified proof with an unproven tail of at most 128 blocks, unless a section states an override. Sensors are sampled once per block, when a validator signs its vote extension; the measurement scripts poll each validator's RPC every 3 s (2 s during the packet-loss cycles of C4). Blocks arrive about once per second.

6.4. Workloads, Metrics and Statistics

The synthetic workloads are described in Appendix L.

The terms, metrics and statistical procedures are defined in Appendix L; confidence intervals are 95% Student-t intervals unless stated otherwise.

7. Evaluation

Each research question is answered by the live experiments that bear on it most directly; the remaining attack scenarios, the prover-backend comparison, and the Bitcoin-reorg boundary test appear in Appendices H, C, D and I. Unless stated otherwise, all numbers are live measurements on Testbed A, the seven-validator testbed of Section 6; its configuration and the FSM parameters are in Appendix F (Tables 21 and 23), and metrics follow Section 6.4. Table 25 in Appendix F

states for each result whether it is live, in-process, model-based or offline, what one replicate is, and which recovery mode the runs used.

7.1. RQ1: Safety Under Byzantine Behavior

By safety we mean agreement among honest validators (identical FSM state and AppHash, aggregate within the honest range, forged or stale receipts rejected) and the lock invariant on committed blocks: no withdrawal-marked transaction commits while the state is SOVEREIGN or RECOVERING. We do not claim that cross-chain assets are protected; the lock acts only once the FSM has reached a locked state (Section 7.2), and the prototype has no bridge (Section 8).

Single Byzantine proposer. The most direct attack on the extended consensus object is a proposer that claims an FSM state not implied by the certificate. One of the seven validators (n=7, 14.3% of the voting power) was configured to do so. Every honest validator recomputed $\delta(s^{h-1}, \mathtt{Agg}(C^h))$, rejected the forged claim in `ProcessProposal`, and the chain committed only honest proposals, with zero AppHash divergence. The same outcome held for the nine further single-actor attacks of Appendix H (eclipse, Sybil, data withholding, forged Bitcoin receipt, withdrawal in SOVEREIGN, censorship, a combined attack, double-signing, and timeout flooding).

The analysis of colluding Byzantine validators (f=2) is in Appendix L; the honest validators agreed on the AppHash at every height in all cases.

The function-level checks of the aggregation are in Appendix L.

7.2. RQ2: Liveness Under Peripheral Degradation

Engram versus the strict receipt-gated baselines. The two receipt-gated baselines (Section 6.2) enforce Bitcoin and DA receipts at every height, one reading them through the vote extensions (in-band) and one directly at each validator (out-of-band observation); the soft baseline keeps Engram's FSM but never locks withdrawals. We ran each system through three faults: Bitcoin checkpoint submission suspended, all seven Celestia bridges stopped, and both. A trial is 60 s healthy, 180 s of fault and 120 s of recovery, with a withdrawal-marked transaction sent every 2 s; each cell of Table 8 has three trials. Both receipt-gated baselines are redeployed for every trial, because the in-band one does not recover on its own in our implementation (below). Engram and the soft baseline recover through proof-only recovery with an unproven-tail bound of 128 blocks (Section 4.7); the receipt-gated baselines have no recovery mode, since their FSM stays ANCHORED.

Two observations should be kept apart. *During the fault*, the receipt-gated baselines commit a fraction of their blocks or stop, which the validity predicate predicts and the experiment measures. The out-of-band baseline commits 20% (29–30 blocks) of its healthy rate under a DA outage, 20% (27–33 blocks) under the combined fault and 61% (89 blocks) under checkpoint suspension, with a longest gap of 6.6–6.8 s and rounds up to 21 (12 under checkpoint suspension), against 96–100% for Engram. The in-band baseline does not slow but halts: it commits 79–82 blocks under checkpoint suspension, 23–28 under a DA outage and 24–28 under the combined fault, all before it stops, 120–121 s after a checkpoint fault begins and 44–52 s after a DA or combined fault begins, and it commits no block again until the end of the trial (the open gap is at least 181 s and 259 s). The fault-window rate in the table, 56–78%, 22–25% and 16–18% of the healthy rate, is therefore the count of blocks before the halt divided by 180 s; proposals reach round 13 under checkpoint suspension and 21–22 otherwise. Its healthy rate differs between trials (0.55 to 0.82 blocks/s), which is why that rate range is wider than the block counts. In the four trials with the lowest healthy rate (0.55 to 0.60) the 60 s healthy window contains three or four blocks decided after round 0, each preceded by a gap of about 6.5 s; the other five in-band trials and all Engram trials contain none. The Bitcoin height gap of a healthy chain reaches

Table 8: Liveness and withdrawals under peripheral degradation (180 s fault, three runs per cell, mean and range). n is the number of trials; Blocks is the count committed in the fault window, Max round the highest round at which a proposal was decided and Longest gap the longest interval between consecutive blocks in the window. Rate is the fault-window block rate as a share of the same trial's healthy rate; "No resume" counts trials in which the chain committed no block in the 120 s after the fault ended. A gap entered as $>x$ is a chain that never resumed: x is the time from its last block to the end of the trial, a lower bound of a gap that never closed. The in-band gate is the receipt-gated baseline whose receipts come through the vote extensions, the out-of-band gate the receipt-gated one in which each validator reads the layers itself; Soft is Engram without the withdrawal lock. The last two columns count withdrawals committed after and before the moment Engram's circuit breaker locks, 125 s (checkpoint pause), 40 s (DA outage) and 40 s (both) after the fault begins; the same window is applied to every system. Engram and the soft baseline recover through proof-only recovery with an unproven-tail bound of 128 blocks.

			Liveness in the fault					Withdrawals committed	
Fault	**System**	n	Blocks	Rate	Longest gap (s)	Max round	No resume	After lock	Before lock
Checkpoint pause	In-band gate	3	81 (79–82)	64 (56–78)%	>181	13	3/3	0	59
	Out-of-band gate	3	89	61 (60–63)%	6.7 (6.6–6.8)	12	0/3	3	59 (58–59)
	Engram	3	142 (139–145)	96 (94–98)%	6.3 (6.2–6.4)	2 (2–3)	0/3	0	58 (55–59)
	Soft	3	143 (142–144)	97 (96–98)%	6.4 (6.3–6.4)	2 (2–3)	0/3	28 (27–29)	60 (58–61)
DA outage	In-band gate	3	26 (23–28)	24 (22–25)%	>264 (261–267)	21 (21–22)	3/3	5 (3–7)	18 (17–19)
	Out-of-band gate	3	29 (29–30)	20 (19–21)%	6.7 (6.6–6.8)	21 (20–21)	0/3	8 (7–8)	19 (18–19)
	Engram	3	150 (149–151)	100 (99–103)%	1.4	0	0/3	0	18 (18–19)
	Soft	3	149 (146–152)	98 (96–100)%	1.4 (1.3–1.4)	0	0/3	72 (72–73)	19
Both	In-band gate	3	27 (24–28)	17 (16–18)%	>262 (259–266)	21	3/3	6 (3–7)	19 (18–19)
	Out-of-band gate	3	30 (27–33)	20 (17–22)%	6.7 (6.7–6.8)	20 (20–21)	0/3	9 (7–10)	18 (18–19)
	Engram	3	149 (148–151)	97 (95–98)%	1.4 (1.3–1.4)	0	0/3	0	17 (15–18)
	Soft	3	150 (149–151)	97 (96–97)%	1.3	0	0/3	72 (72–73)	19

the receipt tolerance once per checkpoint cycle (Appendix K), which is consistent with this, but we did not record the rejected proposals (Section 8.3). Because the rate of Table 8 divides by each trial's own healthy rate, we also divided the fault-window rate of every trial by one reference, the median healthy rate of the 36 trials (0.80 blocks/s). The in-band baseline then has 55–57% under checkpoint suspension, 15–18% under a DA outage and 16–18% under both; the out-of-band baseline 62%, 19–20% and 18–22%; Engram 96–100% and the soft baseline 96–100% under every fault. The comparison does not depend on the normalization. One further in-band DA trial is not in the table: its pre-fault window did not run healthy (4 blocks in 60 s, with proposals already at round 3), so it measures nothing about the fault, and it was repeated; the criterion is the pre-fault window alone, and the excluded trial is kept in the repository. As a sensitivity check, an earlier set of in-band trials, taken before the Bitcoin receipt tolerance was capped, committed 53–57% of the healthy rate under checkpoint suspension and 14–19% under the DA and combined faults (three trials per cell). The cap bears on the Bitcoin receipt only, so the DA and combined rates are not expected to depend on it; the difference to the 17–25% above is of the size of the variation between trials of one baseline, and we do not attribute it to the cap.

After the fault, the in-band baseline commits nothing in the 120 s after the layer returns in all three DA trials, in the combined trials and in the checkpoint trials. In one earlier trial observed beyond the recovery window, the chain was still stuck after twelve minutes, with proposals rejected at round 57 and above. The predicate is satisfiable again once the layer returns, so the stall comes from where this baseline reads the receipts. The checkpoint submitter, the DA publisher and the sensor reads run inside `ExtendVote`, which ABCI++ calls only when a validator is about to precommit, and `ProcessProposal` receives no vote extensions, so the certificate of a proposal is built from the last committed block. Once proposals are rejected, no validator precommits and no new receipt is produced, so the baseline has no path back without an external restart. The out-of-band baseline reads the layers at each validator whether or not a block commits, and it resumes in every one of its nine trials: 48–57 s after the fault ended under

checkpoint suspension, 21–25 s under a DA outage and 51–74 s under the combined fault. We did not separate the two dependencies from each other. The price of the out-of-band design is that two honest validators can read different values and disagree on a proposal, which shows in the rounds above and in the longest gap of about 6.7 s. All seven validators read the same Bitcoin node and the same Celestia chain on one host, so their readings are far more alike here than they would be across independent nodes. Engram commits 139–151 blocks (94–103% of its healthy rate) under every fault, with a longest gap of 1.4 s under DA faults and about 6 s under checkpoint suspension (rounds 2–3), and resumes within two seconds of the fault ending. The 6 s gap occurs once in each of the three checkpoint trials, at round 0 and 35–38 s before the FSM first reaches SOVEREIGN; the soft baseline, which runs the same FSM, shows it in its three trials as well (Section 8.3).

The withdrawal columns separate the systems on withdrawal exclusion. After the circuit breaker has locked, Engram committed none; this count is a consistency check of the breaker, because the window opens when Engram's own FSM reaches SOVEREIGN. The receipt-gated baselines, which have no breaker, committed 3–7 withdrawals after that moment under DA faults (in-band) or 7–10 (out-of-band) and none (in-band) or 3 (out-of-band) under checkpoint suspension, few only because they commit few blocks; both admit withdrawal-marked transactions while degraded, as the soft baseline does, so the low count after the lock moment reflects how few blocks they commit, not a safeguard. Engram's cost shows in the last column: it committed 15–19 withdrawals before locking under DA faults and 55–59 under checkpoint suspension, because it needs about 40 s and 125 s to leave ANCHORED for SOVEREIGN. These counts follow from the workload (one withdrawal-marked transaction every 2 s) and from Bitcoin running thirty times faster than on mainnet; they are not a production exposure. The workload-independent measures and a bound are in Section 8.

The soft baseline shows the other side of the trade-off. It runs Engram's FSM and keeps Engram's liveness, 96–100% of the healthy block rate with a longest gap of 1.4 s under DA faults and about 6 s under checkpoint suspension, but it never locks, so it committed 27–29 withdrawals after the lock moment under checkpoint suspension and 72–73 under DA faults, against none for Engram. Counted by the FSM state at each height, 71–97 of its 175 withdrawals under checkpoint suspension and 131–133 of about 180 under DA faults were committed while the state was SOVEREIGN or RECOVERING. Only Engram combines the liveness of the soft chain with the zero count of a chain that locks.

Composite fault. Engram was then run through a continuous, scripted composite-fault sequence under a standing wide-area latency profile (Table 24): a Bitcoin fault, then a full DA outage (all seven bridges stopped), then three cycles of packet loss on the validator P2P network, healed in turn. The packet loss degrades peer quality without removing quorum connectivity. Fig. 9 shows the resulting timeline.

The composite-fault timeline (Fig. 9) is in Appendix L.

In the first run, made with the heuristic fallback enabled, over 341 seconds (742 samples, 126 polling ticks), the height advanced continuously from 905 to 1093 with no stalled height, and all seven validators agreed on the AppHash at every tick (zero divergent heights, zero RPC errors). The chain entered SOVEREIGN under the combined fault and, after the faults were healed, the recovery proof was verified at t=319 s, the chain entered RECOVERING at t=321 s and ANCHORED at t=324 s. Withdrawals stayed locked throughout SOVEREIGN and RECOVERING, as the circuit-breaker invariant requires. These timings come from a single trial.

We then repeated the sequence five times, each on a freshly deployed chain, with proof-only recovery and an unproven-tail bound of 128 blocks, so that recovery needed a verified proof. In all five runs the height advanced throughout (from 373–387 to 747–760), the validators agreed

on the AppHash at every sampled height (zero divergent heights; one RPC error in one run), and the chain ended in ANCHORED. Times are those at which the median (fourth) validator first reported the state, from the first sample. In two of the five runs some validators were briefly in SUSPICIOUS, ordinary sensor noise at a threshold of two, before the first fault. The chain reached SOVEREIGN 38–53 s after the start (mean 44.9 s, 95% CI ±7.2 s over five runs, Student t), four of the five passing through SUSPICIOUS first (16–22 s). RECOVERING began 311–324 s in, after the faults were healed, and lasted 233–257 s (mean 240.6 s, 95% CI ±12.0 s), against 3 s in the first run; most of that time is the cost of the recovery proof. In one run two of the seven validators fell back from RECOVERING to SOVEREIGN for 2 s before returning. We then ran the sequence five times on the cloud testbed (Testbed B, Appendix F): seven validators in three regions, proof-only recovery with a tail bound of 128 and two proving workers, each run on a freshly deployed chain, at commit `3fe656dce021`, in which the sensor reads run in background loops and `ExtendVote` composes the latest result (Section 4), with the times defined as above.

All five runs started with every validator in ANCHORED and none showed SUSPICIOUS. The height advanced throughout (from 218–228 to 461–494), the validators agreed on the AppHash at every sampled height (zero divergent heights, zero RPC errors), and the chain ended in ANCHORED. The median validator left ANCHORED 33–77 s after the start (mean 45.2 s, 95% CI ±22.6 s), against a Bitcoin delay injected at 20 s and the bridges stopped at 94–96 s; this is the time of the first departure from ANCHORED because the passage through SOVEREIGN to RECOVERING can be shorter than the sampling tick of about 3 s. In every run six or seven validators fell back from RECOVERING to SOVEREIGN (the first fall-back of the median validator at 62–104 s), between 6 and 13 times in total per run. The Bitcoin delay was removed at 297–315 s and the bridges were restarted at 301–319 s. The median validator entered RECOVERING for the last time at 314–331 s, and every validator reported ANCHORED 23–62 s later (mean 40.5 s, ±21.8 s), at 345–385 s after the start. The `ExtendVote` calls of the five runs (25,526 calls) had a median of 0.1 ms in each of the three regions and a maximum of 10.6 ms. Each of the five runs logged two proofs rejected at submission by the chain, one within 40 s of the start and one 368–418 s after the start, when the chain had already returned to ANCHORED; recovery completed through the other segments (see Section 7.3 for a logged case). All 44 segments the prover logged had both boundary blocks in round 0, with no retry. Five runs per measure, so the intervals are descriptive. The seven-scenario sweep (Fig. 7) is in Appendix L.

7.3. RQ3: Hysteresis and Recovery

Hysteresis and flapping. Hysteresis should suppress oscillation ("flapping") without unduly delaying legitimate transitions. We swept the threshold on each of the three gated edges – RECOVERING→ANCHORED, ANCHORED→SUSPICIOUS, and SUSPICIOUS→ANCHORED – in-process under fixed-seed synthetic noise and live on the seven-validator cluster under real network noise (Fig. 8).

The hysteresis figure and table (Fig. 8, Table 45) and their discussion are in Appendix L.

Recovery: what ends Recovering. The state exposes whether the proof condition holds, not which path set it, so we ran one 120 s DA outage per setting and read the prover's log. With the heuristic fallback enabled, the chain left RECOVERING after 36 s while the prover's single proof was rejected as invalid (the interval had grown while it was built): the fallback, not the proof, ended recovery. With proof-only recovery and the default bound of 8 unproven tail blocks, the chain never left RECOVERING, not even after genesis (observed for over thirty minutes in two deployments), although the chain accepted the prover's proofs (seven in one deployment). A 256-header proof takes about two minutes to build and submit, more than 90 blocks at 1.2 s, so the tail was always longer than the bound. With the bound raised to 128 blocks the chain left RECOVERING 341–356 s after entering it in each of five runs on fresh chains (mean 347.2 s, 95%

Table 9: Recoveries on Testbed B with the last boundary block committed in round 0, 1 and 2; each cell gives the first run, then the second (commit rounds read from the chain's `/commit` for the first and last block of the proof). The recovery with round 0 is the control, with no validator stopped. "Not informative": the repeated transaction was byte-identical to the prover's and the mempool answered it.

	Round 0	**Round 1**	**Round 2**
Heights covered by the held proof	235–251 / 231–246	416–457 / 431–464	590–617 / 603–638
Commit round of first / last block	0 / 0 in both	0 / 1 in both	0 / 2 in both
Audit-trail pointer on Celestia (height)	none / 62	92 / 92	121 / 121
Held proof logged as accepted by a validator	not logged / yes	not logged / yes	not logged / yes
Seven validators ANCHORED afterwards	yes / yes	yes / yes	yes / yes
Mutated proofs rejected (of 7)	7 / 7	7 / 7	7 / 7
Same proof submitted again	not informative / rejected	rejected / rejected	rejected / rejected

CI ±8.1 s), the first valid proof being accepted 341–353 s in (recorded in three of the five). The time is mostly the wait for the prover's batch of 256 headers and its build time, not the 68–73 blocks of the outage interval. The bound must therefore exceed the proving latency measured in blocks, and the blocks of that tail are protected by consensus alone. No block of these intervals was committed in a round other than 0. The RQ2 trials above and the repeated composite runs on Testbed A use proof-only recovery with the bound of 128, as do all runs on Testbed B (the composite, load, hung-RPC, window, delay and boundary-round runs); the first composite run, the seven-scenario runs and the data-availability runs used the fallback.

Recovery with a boundary block committed after round 0. On Testbed B (proof-only recovery, tail bound 128, seven validators in three regions) we ran three recoveries from SOVEREIGN on one chain, and repeated the three on a second, freshly deployed chain (Table 9; first run at commit `fe419f406c46`, second at `ee271a1c6b2b`, which adds a log line for an accepted proof). Each froze the Bitcoin daemon and stopped the seven Celestia bridges until every validator reported SOVEREIGN, then restored both until every validator reported RECOVERING. In the second and third recovery two validators adjacent in the proposer rotation were stopped; five of seven keep the quorum, and a height whose first proposer was one of them commits in round 1 or 2. A control file made the prover end a segment only at a block that committed in the wanted round, and hold the finished proof; the prover dropped five headers from the end and retried before it found such a block. While the proof was held we submitted mutated copies of it, then released it.

The chain accepted each held proof and all seven validators returned to ANCHORED; the chain stayed in RECOVERING while the proof was held. In the first run the acceptance follows from the prover's count of accepted proofs, from the absence of a failure logged for the segment and from the rejection of the repeated proof; in the second a validator logged the acceptance with the heights of the proof.

Seven mutations of each proof were rejected: a changed rt_{last}, first-block hash, last-block hash, validator-set hash, count or first height, each by its own check before the UltraHonk verification (the validator log names the check), and a changed proof byte by the verification itself. Submitting an accepted proof again was rejected in the chain's handler, at the rt_{last} check where the validator log named it, in every recovery except the first control, where the repeated transaction was byte-identical to the prover's and the mempool answered it, so that case does not test the chain.

A burst of 24 proofs that pass the binding checks but fail verification, sent in the control recovery, was met by the limit of eight verifications per block: in the first run 13 were refused by the limit and 11 were verified and rejected, in the second 8 and 16 (the number verified depends on how many blocks the burst spans).

A wrong claimed round and a signing power below two thirds cannot be tested on the chain, because no proof exists for them: the circuit does not satisfy its constraints, and the tests of the circuit cover them (Section 4).

There were two recoveries per round, on two chains. An earlier attempt of the same procedure (two recoveries, rounds 1 and 2, commit `3fe656dce021`) is not reported: in both, the proof that the prover held was refused when it was released, and in the first the chain had fallen back to SOVEREIGN during the hold. Its harness mutated the newest instead of the oldest held proof, could not tell a repeated transaction from one that reached the handler, and recorded no check names, so the cause of the refusals was not established; the two runs reported here followed it, with the harness and the log corrected.

A segment queued behind the accepted one was rejected after the chain returned to ANCHORED (in the first recovery, the log of the validators ends with a rejection by the rt_{new} check), which is consistent with the pruning of the interval's headers on the return and with the two proofs rejected in each composite run above; we did not isolate it.

The prover on Testbed B. The proving experiment was repeated on the cloud testbed (Appendix F) with the paper's parameters (Table 23): proof-only recovery and a tail bound of 128. The prover host handed each proof to one of N worker machines, each building one proof at a time, for $N{=}1, 2, 3, 4$. A run held the chain in SOVEREIGN for 15 minutes by suspending checkpoint submission and released it; it is one run per setting. The block rate was 0.41 blocks/s, and a proof took 76–77 s from its start to its accepted submission (12, 22, 23 and 23 proofs accepted). The unproven tail had a 95th percentile of 61, 47, 47 and 46 blocks for one to four workers (maxima 63, 49, 52 and 51). The gate was open for 100% of the samples at the bound of 128 and at 64 in every run; at 32 it was open for 4–12% and at 8 for none; the bound of 64 passed with a margin of one block (maximum 63) and is not a safe setting. The percentiles come from 76–77 autocorrelated samples of one hold per setting. The chain returned to ANCHORED on a verified proof 89–100 s after the release. With one and two workers the figures follow $rL(1+1/N)$ (Section 8), which gives 63 and 47 blocks for the measured rate and latency; with three and four they stay at the value for two, as Section 8 explains. They are not comparable in blocks with the Testbed A runs above, whose block rate was about twice as high.

The data-availability backfill runs and controls (Tables 40 and 41) and the exploratory adaptive down threshold are in Appendix L.

7.4. RQ4: Cost

The certificate-scaling measurements (Table 42) and the analysis of where the cost is and is not are in Appendix L.

The load sweep up to 500 tx/s on one host (Table 43) is in Appendix L; on one host Engram's latency grows with load where vanilla's does not.

Load on separate machines. On Testbed B, where each validator has its own machine, we offered 100 tx/s for 60 s to four configurations on three freshly deployed chains each (Table 11), with the load generator on the operator's machine, outside the cloud: vanilla CometBFT, which does not run the vote-extension code; vanilla's code with an empty vote extension on every vote and a constant healthy observation (empty); Engram with the sensor reads inside `ExtendVote` (sync); and Engram with background sensor reads (snapshot). The client's round trip to the validators and a queueing term that grows with the block interval enter the latency, so only the block interval is independent of the client. Every configuration committed every offered transaction in every deployment, with one round per height.

The block interval was 1.433 ± 0.027 s for vanilla, 1.447 ± 0.023 s for empty, 1.453 ± 0.030 s for

Table 10: Overhead of the extended consensus object against vanilla CometBFT on Testbed A (seven validators on one host except the one-process column; 20 tx/s, 60 s per repeat; mean ± 95% CI over the repeats). "ext. on" is vanilla CometBFT with vote extensions enabled.

	Vanilla			Engram
Metric	1 process	7 val., no ext.	7 val., ext. on	7 val.
Repeats (runs)	5	5	3	5
Rounds per height	1.000	1.000	1.000	1.000
Proposals rejected / height	0	0	0	0
Nil precommits / height	0	0	0	0
Block interval, mean (s)	1.034 ± 0.001	1.233 ± 0.015	1.229 ± 0.023	1.262 ± 0.020
Block interval p50 / p95 (s)	1.032 / 1.042	1.220 / 1.313	1.217 / 1.300	1.241 / 1.335
Blocks/s	0.967	0.811	0.814	0.793
Committed tx/s	20.00	20.00	20.00	20.00
Tx commit latency p50 / p95 (s)	0.67 / 1.16	0.99 / 1.57	0.99 / 1.56	1.03 / 1.63
Mempool backlog, mean / max	8.2 / 24	14.5 / 31	14.6 / 31	15.5 / 44
Tx per block (mean)	17.6	21.1	21.1	21.5

Per-height values come from scanning *every* committed height (commit round, nil precommits, block time). Nil prevotes are not stored on chain; rejected proposals (distinct height/round/reason in the validators' logs) are the proxy for why a validator would prevote nil. "ext. on" keeps vote extensions enabled in the genesis with empty extensions; "no ext." is a fresh genesis without them. The Engram arm is a fresh deployment; a first deployment gave 1.242 ± 0.005 s (three runs). Transaction counts exclude the health certificate in `Txs[0]`. Latency runs from submission to the first 0.25 s poll that sees the transaction in a block. The 1-process arm is a local single node.

Table 11: Load of 100 tx/s for 60 s on Testbed B (mean ± 95% Student-t interval over three freshly deployed chains per column, one measurement each). Vanilla ran at commit `6e010cd` on 12 October; the other three columns at commit `3fe656d` on 16–17 October. "Empty ext." signs a constant healthy observation without reading any sensor; "sync" reads the Bitcoin and Celestia nodes inside `ExtendVote`; "snapshot" composes the latest background reading. The share in ANCHORED is the fraction of the samples of one validator's FSM state during the measurement.

	Vanilla	**Empty ext.**	**Sync**	**Snapshot**
Block interval (s)	1.433 ± 0.027	1.447 ± 0.023	2.773 ± 0.209	1.453 ± 0.030
Latency p50 (s)	2.70 ± 0.03	3.09 ± 1.51	3.74 ± 0.32	2.70 ± 0.11
Latency p95 (s)	3.61 ± 0.02	5.82 ± 9.30	5.49 ± 0.80	3.63 ± 0.10
Latency p99 (s)	3.85 ± 0.04	6.14 ± 9.71	6.44 ± 1.15	3.88 ± 0.16
`ExtendVote`, mean (ms)	n/a	0.01 ± 0.00	924.25 ± 30.82	0.12 ± 0.01
Rounds per height	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
Share of samples in ANCHORED	n/a	1.000 ± 0.000	0.231 ± 0.597	0.923 ± 0.191
Busiest validator CPU (%)	23.8 ± 5.7	33.8 ± 34.1	20.5 ± 17.6	34.9 ± 32.5
Committed / offered	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000

Table 12: Hung Bitcoin RPC on Testbed B (daemon frozen for 240 s, one run per build, no transaction load). Block intervals from the sampled heights; states are shares of the samples of all seven validators.

Build	Phase	Blocks	Interval (s)	`ExtendVote` mean / p95 / max (ms)	States (share of samples)
Inline reads	60 s healthy	23	2.39	896.66 / 1221.9 / 2285.6	Anchored 100%
	240 s hung	80	2.98	1613.85 / 1719.3 / 1779.2	Sovereign 99%, Anchored 1%
	until ANCHORED	12	2.13	977.63 / 1613.1 / 2290.6	Recovering 45%, Anchored 29%, Sovereign 26%
Background reads	60 s healthy	44	1.36	0.07 / 0.1 / 5.0	Anchored 84%, Suspicious 16%
	240 s hung	167	1.42	0.06 / 0.1 / 0.6	Sovereign 99%, Anchored 1%
	until ANCHORED	19	1.21	0.06 / 0.1 / 0.3	Recovering 60%, Anchored 36%, Sovereign 5%

snapshot and 2.773 ± 0.209 s for sync. The mean `ExtendVote` time was 924 ± 31 ms for sync and 0.12 ± 0.01 ms for snapshot (0.01 ms for empty). The 95th-percentile latency was 3.61 ± 0.02 s for vanilla, 3.63 ± 0.10 s for snapshot and 5.49 ± 0.80 s for sync. For empty it was 3.65 and 3.67 s in two deployments and 10.14 s in the first, whose block interval (1.446 s), rounds per height and committed rate were those of the others; we did not find the cause of that one deployment, and its busiest validator used 50% of its CPU against 26% in the other two. With three deployments per column the comparison is descriptive.

The FSM did not stay in ANCHORED in every deployment. With sync the share of samples in ANCHORED was 0.50, 0.15 and 0.04 (SUSPICIOUS in all three, and RECOVERING and SOVEREIGN in the third); with snapshot it was 0.85, 1.00 and 0.92 (SUSPICIOUS in two); with empty it was 1.00 by construction. The latency of the sync column is therefore the latency of a chain that spent most of the measurement outside ANCHORED, and a run at this load does not say how long a chain stays in ANCHORED. The busiest validator used 13–50% of its CPU in all four configurations, with no relation to the configuration.

Two readings follow. First, the empty configuration has the block interval of vanilla, so the extra vote-extension round does not account for the interval of sync on this network. Second, the interval of snapshot is that of vanilla as well, which is consistent with the sensor reads inside `ExtendVote` being the cause: a precommit quorum of five of seven cannot form without a validator of ap-southeast-1, and the `ExtendVote` of sync has to cross to the Bitcoin and Celestia nodes in us-west-2. The runs do not report `ExtendVote` per region and do not isolate that effect.

Slow and hung Bitcoin RPC. We froze the Bitcoin daemon with `docker pause` for 240 s on a chain in ANCHORED, so that every validator's connection opened and every request waited. Table 12 gives the two builds. With inline reads, `ExtendVote` took 1.61 s on average during the freeze (maximum 1.78 s), twice the 800 ms bound of one RPC call, which fits a read that chains two calls, and the block interval rose from 2.39 to 2.98 s. With background reads it stayed at 0.06 ms (maximum 0.6 ms) and the interval was 1.42 s, that of the healthy phase. Both chains kept committing, left ANCHORED for SOVEREIGN and returned to ANCHORED 30.8 s (inline) and 27.6 s (background) after the daemon resumed. The background build reported a Bitcoin gap of 8 (the worst case, the sovereign threshold) in all 1,106 snapshot lines after the freeze took hold, and none with a gap of 0: a frozen layer was not reported as healthy. The freshness flag was never raised in this test, because each call returns within its 800 ms bound and the background reader therefore always delivered a fresh reading of *unreachable*; the branch for a reading older than the window is covered by a unit test and by the window sweep below. One run per build.

`ExtendVote` against injected delay. To vary the latency of the Bitcoin RPC instead of freezing it, `tc netem` added a fixed delay to every response that left the Bitcoin host, for 90 s per value (0, 100, 300 and 500 ms; Table 13), on a freshly deployed chain of each build and with no transaction load. With inline reads the mean `ExtendVote` time rose from 0.87 s to 1.64, 3.09 and 4.65 s, about 7.5 ms for each millisecond of delay, and the block interval from 2.54 to 3.32, 4.43 and 5.82 s. With background reads the mean was 0.06–0.07 ms at every value (maxima 0.3–1.9 ms) and the block interval 1.36–1.65 s. At 500 ms the added delay and the round trip to the Bitcoin host exceeded the 800 ms bound of one call, and the background build reported

Table 13: `ExtendVote` time and block interval against the delay added to every Bitcoin RPC response (Testbed B, 90 s per value, one chain per build, no transaction load). The interval is the span of the highest sampled height over the window after the first 15 s.

Build	Added delay (ms)	Blocks	Interval (s)	`ExtendVote` mean / p95 / max (ms)	States (share of samples)
Inline reads	0	28	2.54	865.89 / 1200.1 / 2230.2	Anchored 73%, Suspicious 27%
	100	22	3.32	1644.33 / 2833.4 / 3538.8	Anchored 100%
	300	14	4.43	3087.39 / 4584.3 / 6134.2	Anchored 100%
	500	11	5.82	4646.79 / 8507.8 / 8924.8	Suspicious 61%, Anchored 39%
Background reads	0	47	1.45	0.07 / 0.1 / 0.3	Anchored 86%, Suspicious 14%
	100	50	1.36	0.06 / 0.1 / 0.3	Anchored 100%
	300	40	1.65	0.06 / 0.1 / 1.9	Anchored 100%
	500	50	1.42	0.06 / 0.1 / 0.4	Recovering 79%, Anchored 15%, Sovereign 6%

Table 14: Hung Bitcoin RPC with the freshness window of the snapshot rule set to 1, 2, 5 and 10 s (Testbed B, one freshly deployed chain per window, daemon frozen for 240 s). "Degraded" counts the `sensor_snapshot` lines that carry the flag for a missing or stale reading; "healthy reads" counts the lines of the frozen phase that report a Bitcoin gap of 0.

Window (s)	Degraded, healthy	Degraded, frozen	Healthy reads, frozen	`ExtendVote` mean / max (ms)	Back to Anchored (s)
1	4 / 301	9 / 1148	0	0.06 / 0.4	42.1
2	0 / 308	0 / 1141	0	0.06 / 0.7	29.0
5	0 / 301	0 / 1127	0	0.06 / 2.0	38.8
10	0 / 294	0 / 1092	0	0.06 / 0.4	27.6

the layer as failed: its validators left Anchored for Sovereign and Recovering (79% of the samples in Recovering), while the validators with inline reads were in Suspicious for 61% of the samples. One 90 s window per value and build, with 11–50 blocks per window; the intervals are descriptive, and samples that failed are not counted in the states.

Freshness window. We repeated the hung-RPC test with the window of the snapshot rule set to 1, 2, 5 and 10 s (`ENGRAM_SENSOR_MAX_AGE_MS`; Table 14), each on a freshly deployed chain, with 60 s healthy and 240 s frozen. In none of the four runs did a validator report a Bitcoin gap of 0 while the daemon was frozen (0 of 4,508 snapshot lines), and `ExtendVote` took 0.06 ms on average in each. The flag that marks a stale or missing reading was raised in the frozen phase only with the window of 1 s (9 of 1,148 lines) and in the healthy phase only with that window (4 of 301 lines); with 2, 5 and 10 s it was never raised. The reader loop runs every 0.5 s and one call may wait 0.8 s, so a window of 1 s can be shorter than the age of the latest reading, and from 2 s on it was not in these runs. The time to return to Anchored after the daemon resumed was 42.1, 29.0, 38.8 and 27.6 s, with no order in the window. In the healthy phase the FSM was in Suspicious for 21, 17, 19 and 0% of the samples, which the flag does not explain (it was raised in none of the lines of the 2 and 5 s runs); we did not isolate the cause. One run per window. The run with 5 s repeats the test of the previous paragraphs with the same settings (27.6 s; flag never raised).

8. Discussion and Limitations

8.1. Security Envelope and Trade-offs

The fallback trades cross-chain withdrawals for in-chain liveness. We discuss the trade along three axes: liveness inside the chain, what the lock establishes, and the cost of griefing.

Liveness inside the chain. In Sovereign, block production, in-chain transactions, application logic, and the forced-transaction queue continue. Cross-chain withdrawals stop. A strict receipt-gated chain under the same degradation commits only a fraction of its blocks, and it resumes after the layer returns only if it observes the layer outside the vote extensions (Section 7.2). The reason is that observations in the in-band design are produced when validators precommit, so the loop that reads the periphery closes only while blocks commit. Relaxing the receipt of the failed layer in Sovereign keeps blocks committing and the sensors reading, which is how the

chain sees the layer return.

What the lock establishes.

- *Shown:* the decision to admit or reject a withdrawal-marked transaction is agreed by all honest validators and applied identically once the FSM is SOVEREIGN or RECOVERING (Section 7.1).
- *Not shown:* that cross-chain assets are protected. The prototype has no bridge and no real assets: it mounts neither `x/auth` nor `x/bank`, transactions are unsigned envelopes, and a withdrawal is recognized by the substring `TX_WITHDRAWAL` in the transaction bytes.
- *Before the lock:* nothing is excluded until the FSM reaches SOVEREIGN; the exposure in that window is measured in Table 47.
- *What asset safety would need:* a bridge that releases funds only against evidence that the Engram height is committed in an anchored state, and a history that a coalition above the fault bound cannot rewrite while the chain is in SOVEREIGN mode (Section 4).

The analysis of the cost of griefing is in Appendix L.

Exposure before the circuit breaker locks. Withdrawals lock only after the detection conditions hold and the FSM reaches SOVEREIGN; the lock does not start at the onset of the fault. The exposure is the time from the start of a fault to that point, and the number of withdrawals that can commit in it is at most the number of Engram blocks in the window times what one block can hold. The window is set by the thresholds, which count Bitcoin blocks for a Bitcoin fault and Engram blocks for a DA fault. On the testbed Bitcoin mines thirty times faster than on mainnet and Engram does not, so the measured times convert as in Table 47. The 55–59 withdrawals of Section 7.2 follow from the workload and the acceleration: the same one withdrawal every 2 s over the mainnet window would be about 1,900, and nothing but block capacity limits a faster sender.

The exposure table (Table 47) is in Appendix L.

The 47–62 minute figures are a model and a scaling of testbed times to mainnet block cadence; no deployment on Bitcoin mainnet was run. The testbed conversion is a scaling of the measured time. A model of the checkpoint cycle, validated against the testbed, puts the lock at 47–50 minutes at mainnet cadence, below the 62 minutes of the plain conversion, so the two bracket it. The same model shows that the healthy gap already touches the SUSPICIOUS threshold every checkpoint cycle, so the threshold cannot be lowered, and that `MaxSuspiciousTime` must follow the Bitcoin cadence of the deployment; a live check at a 600 s mining interval (three runs) reached SOVEREIGN after 50.6–60.7 min, mean 54.1 min, against the model's 47–50 min (Appendix K). We do not implement any of the ways of bounding the exposure that the appendix lists.

The trade-off of the down threshold between noise and concealment is in Appendix L.

Proposer bias within the honest range. When honest observations straddle a threshold, a proposer can choose which honest value the certificate selects (Section 4.3). The proposer, not the validators, holds the vote extensions: the application receives the extensions of the previous height only in the proposer's `PrepareProposal`, and every other validator sees in `ProcessProposal` the commit information and the certificate that the proposer built, not the extensions that were left out. A validator can therefore check that each extension in the certificate is validly signed and bound to the commit, but cannot check that the proposer used all of them, and CometBFT cannot be made to require the whole extended commit without a change to the agreement protocol. The certificate thus guarantees honest-range validity, not an unbiased estimate of the external health. A sequence of Byzantine proposers can therefore bias the timing of transitions, and, because the lemma is per metric, can produce an aggregate vector that no

single honest validator reported, although each of its metrics lies within the honest range of that metric. A Byzantine proposer can also emit a healthy aggregate and reset the unhealthy streak whenever one honest validator in the chosen subset reports healthy, so only faults that every honest validator observes are guaranteed to be detected. A persistent split straddling an FSM threshold (for example three against four validators) has not been tested live; the RQ4 scan shows no extra rounds in the unfaulted case.

The divergence of history at re-anchoring and the data availability of the SOVEREIGN interval are discussed in Appendix L.

The unproven tail: liveness against trust. A recovery proof is built while the chain keeps producing blocks, so the gate accepts it only if the unproven tail is at most `MaxUnprovenTailBlocks`, which must be at least the proving latency times the block rate. A small bound keeps almost all of the SOVEREIGN interval under proof but may never be satisfied if the proof takes longer to build; a larger bound lets slower proofs through, at the price that the last blocks before the return to ANCHORED are certified by the validator quorum alone, the same residual exposure as above, now bounded by the tail. The prototype discards the tracked headers when the chain returns to ANCHORED, so no later proof covers the tail. The experiments used 128 blocks (the code default is 8). We chose 128 as roughly twice the tail that the model below gives for one prover at the block rate of the wide-area testbed ($rL \approx 63$ blocks), and as half of the 256 headers one proof can cover. It was fixed before the tails of the multi-host runs were measured, so it is a bound wide enough for the measured configurations, not an optimum. In the multi-host tail runs (Section 7.3) the gate was open for 100% of the SOVEREIGN hold at bounds of 64 and 128 blocks in every run, for 4–12% at 32 and for none at 8, the default. Measured proving latencies, and why they depend on the load of the host, are in Appendix K. **How the prover keeps up.** Throughput is not what limits recovery: one proof covers up to 256 headers every 77 s on Testbed B, more than three blocks per second against a block rate of 0.41. Latency is. When a proof lands, its tail already holds rL blocks (r the block rate, L the time from the start of the proof to its accepted submission), and with one proof in flight the tail then grows for another L seconds, to $2rL$. Starting a proof every L/N seconds on N machines, each building one proof, shortens the interval between two landings to L/N and bounds the tail by $rL(1 + 1/N)$. This is allowed by the chain's check at submission: a proof binds its first header to the previous checkpoint, so segments can be proven at the same time and are accepted only in order. On Testbed B (r=0.41, L≈77 s) the formula gives 63 and 47 blocks for one and two workers and the measured 95th percentiles are 61 and 47 (maxima 63 and 49), against a bound of 128; the bound must exceed $rL(1 + 1/N)$ with some margin, and holds only for the block rate it was measured at, as the number of blocks of a tail scales with r. The hardware sets L: the same proving step takes 65 s on a `c7a.2xlarge` and 111 s on a `c5.2xlarge` (Table 36), and about 110 s inside a container on the shared Testbed A host, where a single prover left the gate open for 36–42% of the hold. The formula holds for one and two workers and not beyond. Measured with three and four workers, the 95th percentile of the tail was 47 and 46 blocks (maxima 52 and 51), not the 42 and 39 that the formula gives. The prover does not start a segment until it holds a minimum batch of B=16 headers (a setting of the prover, not of the protocol), which takes $B/r \approx 39$ s at this block rate, and the logs show segments starting about 39 s apart for two and for four workers alike. The interval between two landings is then $\max(L/N, B/r)$ and the tail is bounded by $rL + \max(rL/N, B)$, which gives 63, 48, 48 and 48 blocks for one to four workers, against the measured 61, 47, 47 and 46. This reading was drawn from the logs after the measurements and was not tested separately: a run with B lowered was not made. Under it, workers beyond $\lceil rL/B \rceil$=2 add nothing at this block rate, and the floor is rL, about 32 blocks.

Table 15: Claims, evidence and scope

Claim	Evidence	Scope and limit
Certified metrics stay within the honest range	Lemma 1 (proof); Apalache check at N=4; weighted-median tests on random subsets; live threshold sweep	Equal power live; unequal power tested as a function only
Honest validators agree on state and aggregate under f=2	Collusion suite: 33 repeats, offline replay of every committed height	Seven validators, one host; repeats share a chain
Withdrawal Admission Safety (Theorem 1)	Pen-and-paper proof under M1–M3; lock invariant checked on committed blocks	Admission, not asset safety; no bridge in the prototype
Blocks keep committing under peripheral faults	96–100% of the healthy rate, against 20–61% for the receipt-gated baseline that reads at each validator, and a halt after 44–121 s for the one that reads through vote extensions; composite fault repeated five times on one host and five times on seven machines in three regions	Three runs per cell; the baselines are constructs of this paper
Return to ANCHORED needs proof and DA backfill	Proof-only recovery with an unproven tail of 128 blocks in five runs; backfill checked against Celestia and withholding controls	Depends on the tail bound and on proving speed; the tail of at most the bound is certified by the quorum alone and never proven afterwards; the exit gate is qualified in Section 8.3
The cost is about 2% of the block interval	Seven-validator vanilla arms, 20 paired deployments on the same host, the interval excludes zero; at 100 tx/s on seven machines in three regions, three deployments per configuration, the interval is 1.9 times vanilla's with the sensor reads inside `ExtendVote` and 1.01 times with background reads	One host for the 2%; on separate machines the inline build costs more, and the background build does not at this load; Engram's latency grows with load above 100 tx/s on one host
Tendermint-style and FSM layers have no violation within bounds	Bounded model checking per layer; safety ablation of the mechanisms (Table 7)	two of the four layers have no completed standalone check, the composition (C1) and liveness after GST are open, and no check covers N=7, f=2; no end-to-end proof
A static down threshold of 4 or more suppresses flapping	Live three-phase and stationary sweeps	Few runs per cell; the adaptive backoff did not beat it (Appendix J)

8.2. Interpretation of Results

The formal and empirical results cover complementary ground. TLC establishes, per layer and within bounded configurations, that the FSM's safety invariants hold and that removing each mechanism of the ablation study reproduces the violation it targets; Lemma 1 bounds the certified health input by honest observations. The live evaluation shows the composed implementation behaving as specified under real peripheral failures, and because the cross-layer refinement is open, the whole-system claim rests on it. Single-trial live results show that correct behavior occurs without characterizing its distribution; the repeated experiments (the composite repeats, the hysteresis sweeps, the overhead benchmark) do.

Table 15 states each claim of the paper with the evidence behind it and its scope, so that no claim is read as stronger than its evidence.

The three results that need a causal account are discussed in Appendix L.

8.3. Limitations

- *Equal power.* The live runs and the TLA+ model use equal voting power. The weighted aggregation is tested on unequal power only as a function (Section 7.1), not on a live network.
- *Proof system.* The recovery proof is an UltraHonk proof over the BN254 reference string of the Aztec trusted setup that Barretenberg uses, a universal setup that we did not generate or verify; the circuit, its compilation and the embedded WASM verifier have not been audited. The proof shows that the recorded labels are consistent and anchored at two signed boundaries (Section 4.7); it does not show that the interval followed the transition function.
- *Specification and code.* The Go implementation ports the named operators of the specification and cites them, but no conformance or trace-validation check relates the two, so the model-checked results are about the model. The model checks use $N{=}4$ and $f{=}1$, whereas the testbed has seven validators and $f{=}2$.
- *Baselines.* Both strict baselines stop when a receipt is missing. No baseline relaxes the receipt check after a local timeout without the FSM, so the live runs do not separate the contribution of the quorum-backed state machine from that of dropping the requirement. The in-band baseline reads receipts inside block voting and so has no path back once proposals are rejected; a baseline whose checkpoint submission did not depend on block voting might resume, and we did not build one. We report the baseline as built and did not tune it.
- *Behavior not explained.* Several observations in the live runs have no isolated cause. (i) After S4, one of seven seven-scenario runs stayed in SOVEREIGN through the remaining 2,400 s or so of the run, with all validators agreeing and blocks committing; it kept no sensor log, and the two later runs that did, did not repeat it. (ii) Every recovery in the seven-scenario and relay-latency runs fell back from RECOVERING to SOVEREIGN between one and six times before returning to ANCHORED. (iii) Under checkpoint suspension, Engram and the soft baseline each had one gap of about 6 s, at round 0, 35–38 s before the FSM first reached SOVEREIGN. (iv) The healthy rate of the in-band baseline varied between 0.55 and 0.82 blocks/s; in the four slowest trials the healthy window contained three or four blocks decided after round 0, which is consistent with the Bitcoin height gap reaching the receipt tolerance once per checkpoint cycle (Appendix K), but the rejected proposals were not recorded. None of the four changes the ordering of the systems in Table 8, and (i) and (ii) bear on how reliably the chain returns to ANCHORED under the heuristic fallback. On Testbed B behaviour (ii) recurred under proof-only recovery: in each of the five composite runs six or seven validators fell back from RECOVERING to SOVEREIGN, between 6 and 13 times in total, before the return. The other three were not looked for there, since the seven-scenario, suspension and in-band runs were not repeated. Three further observations come from Testbed B: (v) with background sensor reads the FSM was in SUSPICIOUS in the healthy phase of four of the five hung-RPC runs (16–21% of the samples; none with the window of 10 s), in the first window of the delay test (14%) and in two of the three load deployments (8 and 15%), a share the freshness flag does not explain; (vi) one of three deployments with an empty vote extension had a 95th-percentile latency of 10.1 s against 3.7 s in the other two, with the same block interval; (vii) each composite run logged two proofs rejected by the chain, one of them after the return to ANCHORED, and the check named in the validators' log for a rejected proof queued behind an accepted one was the rt_{new} lookup.
- *Testbed.* The multi-host runs (Testbed B: seven validators in three regions, with the prover,

the Bitcoin node and the Celestia node in two others) are one run per prover setting, five composite runs, three deployments per configuration at one load level, two recoveries per boundary round, and one run per build or per value in the hung-RPC, freshness-window and added-delay tests; the vanilla load column was measured at another commit four days before the other columns, and the load generator ran on the operator's machine outside the cloud. The Testbed A results are not pooled with them. The other results come from one host with accelerated Bitcoin (thirty times mainnet; the conversion to mainnet cadence is model-based, see the exposure paragraph above) and single-bridge DA trust, and the backfill gate rests on a quorum-reported contiguous height, not a proof of retrievability (Section 8.4).

- *Unproven tail.* The blocks after the last accepted proof, at most the bound of 128, are certified by the validator quorum alone, and the prototype discards them when the chain returns to ANCHORED, so they are never proven later. The prover's ability to keep the tail short is measured at one to four workers on one testbed, one run each (Section 7.3); it depends on the block rate, which differs between the testbeds.
- *Proof gate.* The recovery-validity flag holds if a verified proof was submitted within the permitted unproven tail of the tracked tip, or if a heuristic holds: the last submitted Bitcoin checkpoint has reached the anchored depth. A protocol parameter disables the heuristic, so that only a verified proof satisfies the gate. Section 7.3 states which runs used which mode; the runs made with the heuristic enabled (the first composite run, the seven-scenario runs and the data-availability runs) do not isolate the cost of the recovery proof in their time in RECOVERING. Recovery through a proof also depends on the unproven-tail bound and on the testbed's block rate and proving speed, as discussed under the unproven tail above.
- *Byzantine scale.* The single-actor attacks involve one Byzantine actor at a time; the $f{=}2$ collusion suite (Section 7.1) addresses colluding validators.
- *Fixed validator set.* The prototype assumes one validator set throughout a degraded interval (Section 3.1).
- *Formal scope.* The cross-layer refinement is open, and attempts to find a configuration in which it completes did not succeed; consensus liveness after GST is specified, but its check did not complete. The health certificate is modelled with Boolean observations in the server layer; numeric aggregation is covered by Lemma 1 only (Section 5).
- *Receipt-gated baselines.* Both are constructs of this paper, not published systems. The out-of-band one lets every validator read Bitcoin and Celestia itself, which is not deterministic: honest validators can disagree on the validity of a proposal, as the rounds and the longest gaps in Table 8 show. All seven validators read the same Bitcoin node and the same Celestia chain on one host, so their readings agree far more often than they would across independent nodes, and we did not inject disagreement between them. Neither baseline can run while a layer is down, by construction, and neither locks withdrawals.
- *Sensor units.* The Bitcoin gap counts Bitcoin blocks and the DA gap Engram heights (Section 4.2), so their thresholds do not compare with each other, and a time bound that covers a Bitcoin fault must follow the Bitcoin cadence of the deployment.
- *Receipt parameters.* The base threshold $K_{\text{DEEP_FINALITY}} + 2$ and the widening caps are fixed design parameters, and the evaluation does not vary them.
- *Prototype robustness.* A validator that restarts republishes its DA height from zero, so the certified DA health stays failed until it catches up (more than half an hour at a chain of about 7,400 heights); persisting the height would remove this, and the prototype does not (Appendix K).

- *Vote-extension liveness and equivocation.* With inline sensor reads (the sync configuration) a slow read delays that validator's precommit; the certificate and the quorum tolerate this only while two thirds of the voting power supply their extensions in time. With background reads (commit `3fe656dce021`) a missing or stale reading counts as the worst case, `ExtendVote` took 0.06 ms on average while the Bitcoin daemon was frozen, and the chain kept committing (Section 7.4); the bound on the time of `ExtendVote` is a property of the code path, measured with a frozen Bitcoin daemon and with delays of 0 to 500 ms added to every response (Section 7.4). A validator can also sign two different extensions for the same height and round. CometBFT does not treat this as evidence, and it gives the adversary nothing beyond signing an arbitrary observation, whose effect Lemma 1 already bounds; accountability in this paper covers equivocation of votes only.
- *Source of the wide-area overhead.* At 100 tx/s on Testbed B the block interval with inline sensor reads is 1.9 times vanilla's, with an empty vote extension it is 1.01 times, and with background reads 1.01 times (Table 11); three deployments per configuration, and the vanilla column was measured four days before the others. The reads inside `ExtendVote` account for the interval in the sense that removing them restores vanilla's; the split by region was not measured. In the load runs the FSM left ANCHORED in the sync and, less often, in the snapshot deployments (Section 7.4), and the cause of those departures was not isolated.
- *Load range.* Load was offered at up to 500 tx/s on one host, and at 100 tx/s alone on seven machines in three regions (Section 7.4). Above 100 tx/s Engram's latency grows and its FSM can leave ANCHORED; neither system saturated, and no throughput ceiling or production scalability is claimed.

8.4. Threats to Validity

Internal validity: host sharing can confound the cost measurements on Testbed A; the controls are in Appendix L.

External validity. Results at seven validators on one host, and the fewer runs on seven machines in three regions, need not extrapolate to larger validator sets or other networks; they are evidence of correct behavior and of the source of the overhead, not production performance figures.

Construct validity. AppHash agreement and height progression are the conventional operational proxies for safety and liveness in CometBFT-family systems. AppHash agreement alone cannot detect an illegal transition agreed on by all validators; the semantic replay check of Section 7.1 addresses this gap.

8.5. Future Work

- *Formal scope.* Closing the cross-layer refinement and extending the certificate model from Boolean to numeric observations would extend the formal results to the composed system; a compositional argument for the interleaving of environment and consensus steps would extend the partial results of Section 5.5. Deductive methods [44, 71] complement bounded model checking where it does not scale.
- *Prover.* Proofs of segments built at the same time on one to four workers keep the tail within the bound on the cloud testbed (Section 7.3); the aggregation of chunks proven in parallel, and longer intervals, are not measured. Incremental recursion would also make the recovery artifact constant-size for intervals of any length.

- *Closing the tail.* Keeping the headers of the unproven tail after the return to ANCHORED and accepting proofs for them while the chain is ANCHORED would make every SOVEREIGN and RECOVERING block eventually covered by a proof; it does not touch the transition function, so the specification of the FSM would not change. A single pending tail is implemented and unit-tested in a development branch of the repository, not evaluated end to end: a tail still open when the next interval begins cannot be chained to it (the ANCHORED blocks between them are not tracked), so that variant drops it and counts the dropped blocks. A list of pending tails, or tracking headers while the chain is ANCHORED, would remove the drop at a cost in state and in the analysis of the proof gate.
- *Data availability.* Replacing single-bridge trust with sampling across independent peers would let the DA sensor detect partial withholding rather than only outages.
- *Baselines and verifiers.* A receipt-gated baseline whose proposals carry verifiable evidence (a Bitcoin header chain with a Merkle proof, and a proof that the blob was posted) would remove the disagreement between validators of the out-of-band baseline, but it would still stall while a layer is down. An external verifier (Section 4), a light client or bridge on another chain that accepts a re-anchoring checkpoint only with a valid proof, is also unbuilt, so its verification cost is unmeasured.
- *The proposer's influence on the certificate.* Requiring one observation in the certificate for every commit vote of `LastCommit` is a single added condition in `ProcessProposal`; it does not affect an honest proposer and makes an omission visible and attributable. The proposer would still choose which precommits `LastCommit` records, because CometBFT accepts any `LastCommit` with more than two thirds of the voting power; removing that choice changes the agreement protocol, needs a new safety and liveness analysis and would require every experiment to be run again. The quantified influence of Section 4.3 stays the guarantee of this paper.
- *Asset-level safety.* The lock is a necessary condition for protecting bridged assets, not a sufficient one. It needs (i) a bridge on the counterparty chain that releases funds only against a proof that the Engram height is committed in an anchored state, with escrow accounting and replay protection; (ii) a treatment of history rewriting by a coalition above the fault bound while the chain is in SOVEREIGN mode (Section 4); and (iii) a bound on the exposure before the lock, for example by a withdrawal quota per Bitcoin block. We evaluate none of these.
- *A time budget for `ExtendVote`.* The sensor reads of `ExtendVote` have no deadline of their own, so a slow Bitcoin or Celestia read delays the precommit of that validator (Section 7.4). A fixed budget, with the last good reading carried with its age and a maximally degraded vector signed when the budget is exceeded, would bound that delay; we have not implemented or measured it.
- *Unequal voting power.* Every live and model-checked result uses equal power. A deployment with unequal power, which the weighted aggregation and the lemma cover, would test the weighted median and the quorum thresholds on a live network.

9. Conclusion

A chain whose block validity requires settlement and DA receipts halts when those layers fail, although its validators are connected and correct: an external dependency can cause a liveness failure without a BFT failure. Engram makes suspending such a dependency an agreed validity and safety mode. A quorum-backed health certificate drives a four-state FSM above a lightly modified CometBFT core. In the degraded modes the corresponding receipt requirement is

relaxed and withdrawal-marked transactions are excluded, and the return to ANCHORED requires recovery hysteresis, DA backfill of the SOVEREIGN interval, and a validator-verified recovery proof.

The evidence supports the mechanism within the stated assumptions. It does not show that cross-chain assets are protected: that needs a bridge that honors the lock, which the prototype does not implement. Bounded model checking of the Tendermint-style and FSM layers found no violation (the largest run explored 2.13 million distinct states), and each of five ablations makes the invariant that guards the removed mechanism fail; the cross-layer refinement remains open. A seven-validator testbed with Bitcoin regtest and Celestia networks kept committing blocks under a composite fault and completed a full recovery cycle with complete AppHash agreement in each of five runs. Against strict receipt-gated baselines under the same faults, Engram kept 96–100% of its block rate; the baseline that reads receipts at each validator kept 20–61% and resumed, and the one that reads them through vote extensions stopped committing 44–121 s after the fault began and did not resume. Neither locks withdrawals, so Engram's cost is the detection delay before its lock. Against seven vanilla validators on the same hosts, Engram lengthens the block interval by 1.8% (95% interval 0.4 to 3.1%; 1.246 s against 1.225 s over 20 paired deployments).

Artifact Availability

The implementation, TLA+ specifications, and experiment scripts are in three repositories:

- `engram-sovereign-fsm`: the Cosmos SDK application, the TLA+ specifications, the recovery circuits, the experiment scripts, and this manuscript. `https://github.com/cuongct924/engram-sovereign-fsm`
- `engram-consensus-core`: the CometBFT fork used as the agreement core (Section 6.1 lists its four differences from upstream). `https://github.com/cuongct924/engram-consensus-core`
- `engram-barretenberg-wasm`: the build of the UltraHonk verifier embedded in the application; it is provenance only, not a build dependency. `https://github.com/cuongct924/engram-barretenberg-wasm`

The first two repositories are released under the Apache License 2.0. The prover and tail runs on the multi-host testbed used images built from commit `1c34d994e7ad` of the first repository, which pins commit `493a20121` of the second and commit `43a63abbb` of the third; the digests of the two images are `sha256:b66ce901...` (`engram-node`) and `sha256:c7915434...` (`engram-reanchoring-prover`). The composite, load and hung-RPC runs used commit `3fe656dce021`, with the digests `sha256:7155a49a...` and `sha256:81ad486b...`; the vanilla load column was measured at commit `6e010cd`, which does not run the vote-extension code. The first set of recoveries with a boundary block committed in round 1 and 2 used commit `fe419f406c46`, which differs from `3fe656dce021` by one log line in the proof handler (digests `sha256:3651ae15...` and `sha256:d722079e...`); the second set used commit `ee271a1c6b2b`, which adds a log line for an accepted proof (digests `sha256:09fdb54c...` and `sha256:e7aa8e2a...`). `docs/REPRODUCE.md` lists the digests in full.

References

[1] M. Al-Bassam, "LazyLedger: A Distributed Data Availability Ledger With Client-Side Smart Contracts," arXiv:1905.09274, 2019, https://arxiv.org/abs/1905.09274.

[2] E. Buchman, "Tendermint: Byzantine Fault Tolerance in the Age of Blockchains," Master's thesis, University of Guelph, 2016, https://atrium.lib.uoguelph.ca/xmlui/bitstream/handle/10214/9769/Buchman_Ethan_201606_MAsc.pdf.

[3] E. Buchman, J. Kwon, and Z. Milosevic, "The Latest Gossip on BFT Consensus," arXiv:1807.04938, 2018, https://arxiv.org/abs/1807.04938.

[4] J. Neu, E. N. Tas, and D. Tse, "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma," in *Proceedings of the 2021 IEEE Symposium on Security and Privacy (S&P)*, 2021, pp. 446–465.

[5] V. Buterin, D. Hernandez, T. Kamphefner, K. Pham, Z. Qiao, D. Ryan, J. Sin, Y. Wang, and Y. X. Zhang, "Combining GHOST and Casper," arXiv:2003.03052, 2020.

[6] F. Gomes Figueira, M. Derka, C. L. Chiu, and J. Gorzny, "A Practical Rollup Escape Hatch Design," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2025)*, 2025, https://arxiv.org/abs/2503.23986.

[7] Offchain Labs, "Inside AnyTrust," Arbitrum documentation, https://docs.arbitrum.io/inside-anytrust, accessed 2026-10-02.

[8] Optimism contributors, "Alt-DA Mode Specification," OP Stack experimental specification, https://github.com/ethereum-optimism/specs/blob/main/specs/experimental/alt-da.md, accessed 2026-10-03.

[9] Cosmos SDK contributors, "ADR 064: ABCI 2.0 Integration (Phase II)," Cosmos SDK architecture decision record, https://github.com/cosmos/cosmos-sdk/blob/main/docs/architecture/adr-064-abci-2.0.md, accessed 2026-10-03.

[10] Skip Protocol and dYdX contributors, "Slinky: A General-Purpose Oracle Leveraging ABCI++," Software repository (continued as skip-mev/connect), https://github.com/dydxprotocol/slinky, accessed 2026-10-03.

[11] N. C. Mendonça, C. M. Aderaldo, J. Cámara, and D. Garlan, "Model-Based Analysis of Microservice Resiliency Patterns," in *Proceedings of the 2020 IEEE International Conference on Software Architecture (ICSA)*, 2020, pp. 114–124.

[12] R. Pass and E. Shi, "Thunderella: Blockchains with Optimistic Instant Confirmation," in *Advances in Cryptology – EUROCRYPT 2018*. Springer, 2018.

[13] P.-L. Aublin, R. Guerraoui, N. Knežević, V. Quéma, and M. Vukolić, "The Next 700 BFT Protocols," *ACM Transactions on Computer Systems*, vol. 32, no. 4, pp. 1–45, 2015.

[14] N. Giridharan, F. Suri-Payer, I. Abraham, L. Alvisi, and N. Crooks, "Autobahn: Seamless High Speed BFT," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP 2024)*, 2024, pp. 1–23.

[15] R. Pass and E. Shi, "The Sleepy Model of Consensus," in *Advances in Cryptology – ASIACRYPT 2017*, ser. Lecture Notes in Computer Science, vol. 10625, 2017, pp. 380–409.

[16] V. Buterin and V. Griffith, "Casper the Friendly Finality Gadget," arXiv:1710.09437, 2017.

[17] T. D. Chandra and S. Toueg, "Unreliable Failure Detectors for Reliable Distributed Systems," *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, 1996.

[18] W. Chen, S. Toueg, and M. K. Aguilera, "On the Quality of Service of Failure Detectors," *IEEE Transactions on Computers*, vol. 51, no. 5, pp. 561–580, 2002.

[19] N. Hayashibara, X. Défago, R. Yared, and T. Katayama, "The φ Accrual Failure Detector," in *Proceedings of the 23rd IEEE International Symposium on Reliable Distributed Systems (SRDS)*, 2004, pp. 66–78.

[20] A. Doudou, B. Garbinato, R. Guerraoui, and A. Schiper, "Muteness Failure Detectors: Specification and Implementation," in *Dependable Computing – EDCC-3*, 1999, pp. 71–87.

[21] D. Dolev, N. A. Lynch, S. S. Pinter, E. W. Stark, and W. E. Weihl, "Reaching Approximate Agreement in the Presence of Faults," *Journal of the ACM*, vol. 33, no. 3, pp. 499–516, 1986.

[22] D. Stolz and R. Wattenhofer, "Byzantine Agreement with Median Validity," in *19th International Conference on Principles of Distributed Systems (OPODIS 2015)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 46, 2015, pp. 22:1–22:14.

[23] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi, "Town Crier: An Authenticated Data Feed for Smart Contracts," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 270–282.

[24] S. Eskandari, M. Salehi, W. C. Gu, and J. Clark, "SoK: Oracles from the Ground Truth to Market Manipulation," in *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies (AFT '21)*, 2021, pp. 127–141.

[25] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A Decentralized Truth Discovery Approach to the Blockchain Oracle Problem," in *Proceedings of IEEE INFOCOM 2023*, 2023, pp. 1–10.

[26] Y. Yao, Y. Shi, G. Tian, M. Miao, and W. Susilo, "PSCBO: A Provably Secure Consensus-Based Blockchain Oracle," *Computer Standards & Interfaces*, vol. 91, 2025.

[27] M. Al-Bassam, A. Sonnino, V. Buterin, and I. Khoffi, "Fraud and Data Availability Proofs: Detecting Invalid Blocks in Light Clients," in *Financial Cryptography and Data Security (FC 2021)*, ser. Lecture Notes in Computer Science, vol. 12675, 2021, pp. 279–298.

[28] D. Pham Minh, H. Vuong Huu, and D. A. Tran, "Efficient Data Availability Sampling via Coded Distributed Arrays," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2026)*, 2026, https://arxiv.org/abs/2606.16200.

[29] D. Karakostas and A. Kiayias, "Securing Proof-of-Work Ledgers via Checkpointing," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2021)*, 2021.

[30] E. N. Tas, D. Tse, F. Yu, and S. Kannan, "Babylon: Reusing Bitcoin Mining to Enhance Proof-of-Stake Security," arXiv:2201.07946, 2022, https://arxiv.org/abs/2201.07946.

[31] S. Azouvi and M. Vukolić, "Pikachu: Securing PoS Blockchains from Long-Range Attacks by Checkpointing into Bitcoin PoW using Taproot," in *Proceedings of the 2022 ACM Workshop on Developments in Consensus (ConsensusDay '22)*, 2022, pp. 53–65.

[32] E. N. Tas, D. Tse, F. Gai, S. Kannan, M. A. Maddah-Ali, and F. Yu, "Bitcoin-Enhanced Proof-of-Stake Security: Possibilities and Impossibilities," in *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, 2023, pp. 126–145.

[33] M. Capretto, M. Ceresa, A. Fernández Anta, P. Moreno-Sanchez, and C. Sánchez, "A Secure Sequencer and Data Availability Committee for Rollups," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025, pp. 2129–2143.

[34] ——, "A Decentralized Sequencer and Data Availability Committee for Rollups Using Set Consensus," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2026)*, 2026, pp. 1–9.

[35] T. Xie, J. Zhang, Z. Cheng, F. Zhang, Y. Zhang, Y. Jia, D. Boneh, and D. Song, "zkBridge: Trustless Cross-chain Bridges Made Practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 3003–3017.

[36] P. Vesely, K. Gurkan, M. Straka, A. Gabizon, P. Jovanovic, G. Konstantopoulos, A. Oines, M. Olszewski, and E. Tromer, "Plumo: An Ultralight Blockchain Client," in *Financial Cryptography and Data Security (FC 2022)*, 2022, pp. 597–614.

[37] J. Bonneau, I. Meckler, V. Rao, and E. Shapiro, "Coda: Decentralized Cryptocurrency at Scale," Cryptology ePrint Archive, Paper 2020/352, 2020, later renamed Mina.

[38] B. Bünz, L. Kiffer, L. Luu, and M. Zamani, "FlyClient: Super-Light Clients for Cryptocurrencies," in *Proceedings of the 2020 IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 928–946.

[39] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, "Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data," in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 111–120.

[40] S. Bowe, J. Grigg, and D. Hopwood, "Recursive Proof Composition without a Trusted Setup," Cryptology ePrint Archive, Paper 2019/1021, 2019.

[41] A. Kothapalli, S. Setty, and I. Tzialla, "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes," in *Advances in Cryptology – CRYPTO 2022*, ser. Lecture Notes in Computer Science, vol. 13510, 2022, pp. 359–388.

[42] I. Konnov, J. Kukovec, and T.-H. Tran, "TLA+ Model Checking Made Symbolic," *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, 2019.

[43] S. Braithwaite, E. Buchman, I. Konnov, Z. Milosevic, I. Stoilkovska, J. Widder, and A. Zamfir, "Formal Specification and Model Checking of the Tendermint Blockchain Synchronization Protocol (Short Paper)," in *2nd Workshop on Formal Methods for Blockchains (FMBC 2020)*, ser. OpenAccess Series in Informatics (OASIcs), vol. 84, 2020.

[44] O. Padon, K. L. McMillan, A. Panda, M. Sagiv, and S. Shoham, "Ivy: Safety Verification by Interactive Generalization," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2016, pp. 614–630.

[45] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill, "IronFleet: Proving Practical Distributed Systems Correct," in *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP)*, 2015, pp. 1–17.

[46] J. R. Wilcox, D. Woos, P. Panchekha, Z. Tatlock, X. Wang, M. D. Ernst, and T. Anderson, "Verdi: A Framework for Implementing and Formally Verifying Distributed Systems," in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2015, pp. 357–368.

[47] L. Qiu, Y. Kim, J.-Y. Shin, J. Kim, W. Honoré, and Z. Shao, "LiDO: Linearizable Byzantine Distributed Objects with Refinement-Based Liveness Proofs," *Proceedings of the ACM on Programming Languages*, vol. 8, no. PLDI, pp. 1140–1164, 2024.

[48] S. Bano, A. Sonnino, A. Chursin, D. Perelman, Z. Li, A. Ching, and D. Malkhi, "Twins: BFT Systems Made Robust," in *25th International Conference on Principles of Distributed Systems (OPODIS 2021)*, ser. LIPIcs, vol. 217, 2022, pp. 7:1–7:29.

[49] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos Engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[50] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99)*, 1999, pp. 173–186.

[51] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the Presence of Partial Synchrony," *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, 1988.

[52] M. Yin, D. Malkhi, M. K. Reiter, G. Golan-Gueta, and I. Abraham, "HotStuff: BFT Consensus with Linearity and Responsiveness," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, 2019, pp. 347–356.

[53] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, "Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus," in *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22)*, 2022, pp. 34–50.

[54] A. Spiegelman, N. Giridharan, A. Sonnino, and L. Kokoris-Kogias, "Bullshark: DAG BFT Protocols Made Practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 2705–2718.

[55] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and Efficient Asynchronous Broadcast Protocols," in *Advances in Cryptology – CRYPTO 2001*, ser. Lecture Notes in Computer Science, vol. 2139. Springer, 2001, pp. 524–541.

[56] P. Sheng, G. Wang, K. Nayak, S. Kannan, and P. Viswanath, "BFT Protocol Forensics," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021, pp. 1722–1743.

[57] W. Honoré, L. Qiu, Y. Kim, J.-Y. Shin, J. Kim, and Z. Shao, "AdoB: Bridging Benign and Byzantine Consensus with Atomic Distributed Objects," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 419–448, 2024.

[58] W. Honoré, J.-Y. Shin, J. Kim, and Z. Shao, "Adore: Atomic Distributed Objects with Certified Reconfiguration," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '22)*, 2022, pp. 379–394.

[59] S. Gilbert and N. Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.

[60] J. R. Douceur, "The Sybil Attack," in *Peer-to-Peer Systems (IPTPS 2002)*, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 251–260.

[61] E. Heilman, A. Kendler, A. Zohar, and S. Goldberg, "Eclipse Attacks on Bitcoin's Peer-to-Peer Network," in *Proceedings of the 24th USENIX Security Symposium*, 2015, pp. 129–144.

[62] Z. Rehman, M. A. Gregory, I. Gondal, H. Dong, and M. Ge, "Eclipse Attacks in Blockchain Networks: Detection, Prevention, and Future Directions," *IEEE Access*, vol. 13, pp. 25 918–25 933, 2025.

[63] A. Back, M. Corallo, L. Dashjr, M. Friedenbach, G. Maxwell, A. Miller, A. Poelstra, J. Timón, and P. Wuille, "Enabling Blockchain Innovations with Pegged Sidechains," Whitepaper, Blockstream, 2014.

[64] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, Private Smart Contracts," in *Proceedings of the 27th USENIX Security Symposium*, 2018, pp. 1353–1370.

[65] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry, and A. Gervais, "SoK: Layer-Two Blockchain Protocols," in *Financial Cryptography and Data Security (FC 2020)*, ser. Lecture Notes in Computer Science, vol. 12059, 2020, pp. 201–226.

[66] Offchain Labs, "Arbitrum Orbit AnyTrust Chains," Arbitrum blog, https://blog.arbitrum.io/arbitrum-orbit-anytrust-chains/, accessed 2026-10-02.

[67] S.-S. Lee, A. Murashkin, M. Derka, and J. Gorzny, "SoK: Not Quite Water Under the Bridge: Review of Cross-Chain Bridge Hacks," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2023)*, 2023, pp. 1–14, arXiv:2210.16209.

[68] E. Deirmentzoglou, G. Papakyriakopoulos, and C. Patsakis, "A Survey on Long-Range Attacks for Proof of Stake Protocols," *IEEE Access*, vol. 7, pp. 28 712–28 725, 2019.

[69] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge," Cryptology ePrint Archive, Paper 2019/953, 2019.

[70] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002, https://lamport.azurewebsites.net/tla/book.html.

[71] R. Lotan and S. Shoham, "Implicit Rankings for Verifying Liveness Properties in First-Order Logic," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2025)*, 2025, pp. 375–395.

[72] N. Bondarev, K. Ziborov, and Y. Yanovich, "Systematization of Knowledge: Formal Verification of Consensus Protocols," arXiv:2608.21935, 2026, https://arxiv.org/abs/2608.21935.

[73] J. Kwon and E. Buchman, "Cosmos: A Network of Distributed Ledgers," Whitepaper, 2016, https://github.com/cosmos/cosmos/blob/master/WHITEPAPER.md.

[74] Aztec Labs, "Noir: A Domain-Specific Language for Zero-Knowledge Proofs, and the Barretenberg Proving Backend," Software documentation, 2024, noir: https://noir-lang.org; Barretenberg: https://github.com/AztecProtocol/barretenberg.

Appendix

A. Related Work: Full Comparison

Table 16 lists the systems of Section 2 with the trigger, the decider, the behavior under failure and recovery, the assumption and the assurance each source reports.

Table 16: Failure handling in representative systems, from each system's primary publication or specification, except the baseline and Engram rows, which are designs evaluated in this paper. The assurance column does not rank the designs.

System	Trigger and observation	Decided by	Behavior under failure and recovery	Assumption	Assurance reported
Reference point					
In-band gate (this work)[a]	Missing receipt, read through vote extensions	Validity predicate	No proposal is valid until the receipts return; the chain does not resume in our implementation	External layers live	Measured here (RQ2, three runs per cell)
Receipt-gated baseline, out-of-band (this work)[a]	Missing receipt, read at each validator	Validity predicate	No proposal is valid until the receipts return; the chain resumes, validators may disagree	External layers live	Measured here (RQ2, three runs per cell)
Degradation inside consensus					
Ebb-and-flow [4]	Dynamic participation or network partition	Validator protocol	Ledger keeps growing and the finalized prefix lags; the prefix catches up	Partial synchrony (finalized ledger); synchrony (available ledger)[b]	Security proof; simulations
Casper FFG leak [16]	More than one third of validators stop voting	Validator protocol	Inactivity leak drains non-voting deposits; formula left open	PoW overlay; under 1/3 slashable stake (safety); 2/3 voting (liveness)	Accountable-safety and plausible-liveness proofs
External data in consensus					
Vote-extension oracle [9, 10]	External data update; signed vote extensions	Validator consensus pipeline	Stake-weighted median goes to the application; an unreachable oracle yields an empty vote extension that counts as valid, so the chain keeps running; the ADR prescribes no fallback	Signatures from at least two thirds of voting power	Design record; open-source code
Bitcoin anchoring					
Babylon [32]	Transaction censorship; a signed checkpoint whose block is unavailable	Clients and Bitcoin[c]	Bitcoin orders PoS block hashes (rollup mode); the chain later exits rollup mode; clients stop adding blocks that lack available data (emergency break)	Bitcoin safe and live	Security proofs; measured withdrawal delay and checkpoint cost
Rollup fallback					
Escape hatch [6]	No new state root within a time limit	Users	Users bypass the bridge and withdraw with a Merkle proof; a nullifier blocks a second escape	L1 live; users submit proofs	Design and use cases

Table 16 (continued)

System	Trigger and observation	Decided by	Behavior under failure and recovery	Assumption	Assurance reported
AnyTrust [7, 66]	Too few committee signatures	Sequencer	Full batch posted to the L1 with standard rollup delay; the committee path resumes	L1 live	Public documentation
OP Alt-DA [8]	External DA input unavailable; L1 challenge	L1 contract	Derivation stalls on missing data; the data is resolved on the L1, or the affected L2 blocks are reorged	L1 live; some party challenges	Specification of a beta feature, published in the experimental section
Secure arranger [33]	Safety violation by the arranger	Honest agent and L1 contract	Fraud proof settles on the L1; liveness violations cannot be proved there	L1 safe and live; one honest agent	Fraud-proof games mechanized in Lean 4
Engram	Bitcoin, DA, or peripheral-P2P health; signed validator observations	Validator BFT pipeline	Receipt rules relaxed for the failed layer and withdrawals locked; proof, DA backfill, and hysteresis restore[d]	CometBFT quorum connected; Byzantine power under 1/3	Per-layer bounded model checking; live fault injection (one host)

Notes. (a) Engram's binary with the FSM pinned to ANCHORED (Section 6.2); the two rows differ only in whether the receipts come through the vote extensions or from each validator's own reading. Both are constructs of this paper, not published systems. (b) The finalized ledger tolerates under 1/3 adversarial nodes, the available ledger under 1/2. (c) Clients force the switch to rollup mode and apply the emergency break. (d) The prototype enforces DA backfill; a heuristic can satisfy the proof condition unless proof-only recovery is on; the refinement is open (Sections 8.3 and 5).

B. Health Certificate: Adversarial Cases

Section 4.3 states Lemma 1 and summarizes how the adversarial cases reduce to it. This appendix gives each case.

Lemma 1 does not require honest observations to agree; it bounds what the adversary can do when they disagree. Each adversarial case reduces to it:

- *Stale or replayed observation.* A signature binds the height and round; an observation signed at another height or round fails (a) and is discarded.
- *Conflicting observations.* Two different signed observations by one validator count once by (b); Byzantine weight is below $W/3$ either way.
- *False observations.* Byzantine validators below $W/3$ reporting extreme values cannot move the aggregate outside the honest range (Lemma 1).
- *Proposer omission.* The proposer chooses S, and may omit honest observations. If S still meets (d), Lemma 1 still holds. The proposer's residual influence is choosing *which* honest value in the honest range is selected when honest validators genuinely disagree. With n equal-power validators, all honest, and a quorum of $q = \lfloor 2n/3 \rfloor + 1$ observations, the proposer can reach exactly the values from the $(\lfloor q/2 \rfloor + 1)$-th to the $(n - q + \lfloor q/2 \rfloor + 1)$-th smallest observation: a subset of $s \geq q$ observations has its median at rank $\lfloor s/2 \rfloor + 1$ within it, which is at least $\lfloor s/2 \rfloor + 1$ and at most $n - s + \lfloor s/2 \rfloor + 1$ among all n, and both extremes are reached at $s = q$. For n=7 and q=5 that is the third to the fifth smallest, the median of all seven being the fourth.

Table 17: Deployed circuit: constant-cost scaling (Noir+Honk, Poseidon2)

Interval count	Prove time	Verify time	Proof size
1	38.3 s	25 ms	14,656 B
16	41.4 s	23 ms	14,656 B
256 ($N_{\max}$)	40.4 s	24 ms	14,656 B

Only when the honest observations straddle a threshold does this choice change a decision, and a transition needs the hysteresis count over consecutive blocks while proposers rotate in proportion to voting power, so a Byzantine proposer influences the blocks it proposes. The same freedom exists one level down: CometBFT accepts any `LastCommit` that carries more than two thirds of the voting power and ignores absent signatures, so a proposer also chooses which precommits the block records. Requiring one observation for every commit vote of `LastCommit` would tie the two choices together and make an omission visible in the block, but it would not remove the choice (Section 8.5).

If omission drops S to weight at most $2W/3$, the certificate is not well-formed and honest validators reject the proposal. The only exception is the first height after vote extensions are enabled, whose predecessor carries no extensions; there the certificate is replaced by a maximally degraded metric vector, the conservative choice. In the live collusion suite (Section 7, case B3), honest validators accepted a proposer that kept exactly a quorum of certificates in 3 of 3 repeats and rejected one that kept fewer, 210 times in total across 3 repeats.

- *Missing or malformed observations.* CometBFT accepts a precommit's extension only if its signature verifies under the validator's consensus key, and the application's `VerifyVoteExtension` accepts only extensions that decode; signatures are checked again against the genesis keys when the certificate is aggregated. The extended commit for $h-1$ carries weight above $2W/3$, so an honest proposer can always build a well-formed certificate, and a Byzantine validator cannot reduce the usable weight by sending a decodable extension with a bad signature, because CometBFT would not have accepted that vote. This relies on CometBFT and the aggregation using the same key and sign bytes.
- *Divergent honest views.* When honest views straddle a threshold, different proposers may select certificates on different sides of it. Each resulting transition is legal and deterministic given its certificate; the FSM's hysteresis absorbs short alternations (Section 4.5).

C. Backend Trade-off: Noir+Honk vs. Plonky3

This appendix compares the deployed Noir/UltraHonk recovery circuit (Section 4.7) with a STARK-based alternative, Plonky3. The comparison is necessarily partial: boundary-signature verification, condition (e), dominates the deployed circuit's cost (1,097,274 opcodes with it versus 6,143 without, a factor of ∼180), and no Plonky3 configuration we built implements it.

Table 17 confirms that proving time, verification time, and proof size are independent of the number of real headers up to the compiled capacity $N_{\max}$=256. Verification by the validators' embedded WebAssembly verifier costs ∼26 ms per proof. An interval of L blocks requires $\lceil L/256 \rceil$ leaf proofs when submitted one by one, or a single aggregate proof up to 1024 blocks (Appendix D).

Table 18 makes the coverage gap explicit. Under a primitive-only comparison (256 chained Poseidon2 permutations, no FSM or signature logic, i.e. neither configuration implements condition (e)), Plonky3 produces a larger proof (1,278,939 B vs. 14,656 B, ∼87×) but proves

Table 18: Backend comparison: statement coverage

Config.	(a)	(b)	(c)	(d)	(e)
Noir+Honk (deployed)	✓	✓	✓	✓	✓
Plonky3, primitive-only	–	–	–	–	–
Plonky3, fuller statement	✓	✓	✓	✓	–

(a) header-chain continuity, (b) mode-label consistency (every recorded state is SOVEREIGN or RECOVERING), (c) lock-flag consistency (every recorded lock flag is set), (d) root binding (the first header links to the last anchored root and the last header hashes to the new root), (e) boundary quorum certification (ed25519 signatures of the first and last block).

faster (0.044 s vs. 0.684 s, ∼15.5×) with a transparent, post-quantum-secure setup versus the Noir/Honk backend's trusted KZG setup. Under a fuller-statement comparison that adds the mode-label, lock-flag and root-binding conditions (but still omits condition (e)), Plonky3 proves in 0.024–0.096 s with proof sizes of 84–124 KB across 2–256 headers. Neither comparison supports a general speed claim for Plonky3, because both omit the component that dominates the deployed circuit's cost. These numbers characterize primitive-level backend trade-offs only.

D. Chunk Aggregation

This appendix measures the aggregator of Section 4.7 against submitting the chunk proofs one by one. The measurements were taken offline on a 10-core, 32 GiB macOS workstation, not on Testbed A, with nargo 1.0.0-beta.22 and `bb` 5.0.0-nightly.20260522. The chain is synthetic: it has one deterministic header sequence with real ed25519 precommits and one validator set, so that consecutive chunks chain through their roots and heights. Aggregate proving and verification are averaged over five trials per interval length; each leaf proof was timed once. Absolute times are not comparable to Table 17, which was measured on a different host.

The aggregator verifies the chunk proofs in-circuit and checks that chunk i starts at the root and height where chunk $i-1$ ended, under the same validator-set hash. It exposes the same public inputs as a single chunk, so an aggregate proof stands in for a leaf proof, and its capacity $K=4$ is fixed at compile time. Unbounded incremental recursion would require the step circuit to receive its own verification-key hash as a public input, which we did not implement. Recursion is applied at the chunk level because in-circuit verification of one UltraHonk proof costs about 0.7 million gates, whereas hashing one header costs about 12 opcodes: recursion over per-header proofs measured 650–1,200× slower than direct hashing at matched sizes.

Table 44 (Section 4.7) shows that submitted bytes and verification time of the aggregate do not depend on L up to $K \cdot 256 = 1024$, while the one-by-one cost grows linearly. The price is one extra proof: the aggregator circuit has 3,002,299 gates and proves in 25.7–29.5 s using about 5.2 GB, against 3,234,226 gates, 30.0–31.5 s, and 5.7–5.9 GB for a leaf. Total proving time rises by 49% at $L=512$ and 22% at $L=1024$. Chunk proofs were produced sequentially; proving them in parallel was not measured. Every chunk boundary height needs a witness for the round in which that block committed, which the leaf circuit accepts for any round.

The aggregator rejects each of eleven malformed witnesses: reversed or repeated chunks, a wrong total count, final root, first height, or validator-set hash, a height gap between chunks, a different validator set in the second chunk, an extension through padding slots, and a chunk count outside $[1, K]$. The valid two-chunk witness is accepted, and a 512-header aggregate passes every `SubmitRecoveryProof` binding against fixture chain state. On the Docker testbed, aggregate proofs of three and two chunks (514 and 512 headers) were accepted by `SubmitRecoveryProof` on a live chain; whether the chain then leaves RECOVERING depends on the unproven-tail bound, as the Discussion reports. Aggregation bounds verification; the prover's time still grows with

the interval.

E. Formal Verification Details

This appendix holds the configurations and the negative or incomplete results that Section 5 summarizes. The raw logs and the counterexample traces are in `spec/results_formal/` and `spec/traces/` of the repository.

Refinement obligation C1

Splitting. To make C1 tractable we split it by which actions may fire and check each part with TLC under symmetry reduction over the three honest validators. With the consensus core frozen, the environment part (sensor updates, clock ticks, DA backfill) completed: 17,500 distinct states, depth 10, 10 m 35 s, no violation. With the environment frozen, the consensus-core part did not complete: the run ended after 1 h 41 m at 19,848,141 distinct states (depth 15, 13,542,792 states queued) without a violation. Neither part covers the interleaving of environment and consensus steps, and no composition argument is established, so C1 is not claimed.

Existence check. A completed run constrains a behaviour only if that behaviour is reachable, so a configuration is usable only if a round change, a Byzantine proposal and a decided block are all reachable in it. At N=4, f=1 with smaller timestamp and height bounds, a round change and a Byzantine proposal are reached in seconds in every configuration, but a decided block is not reached within the time budget: in the consensus-core part at three configurations (runs ended after 25 to 67 minutes at 7 to 30 million states), and at the smallest rung L0 of the monolithic ladder over the whole server layer (one round, one height, three timestamps), where the search ended after 6 h 47 min at 15.3 billion states generated (249 million distinct, depth 12) with its queue still growing. Restricting the same bound to the healthy path, and removing symmetry reduction at two rounds, also reaches no decision. Two honest prevotes for one proposal co-occur at depth 5 within a second, so the model is live through the prevote phase; whether a decision is reachable at this bound is undetermined. None of these configurations is reported as a result.

Target and mapping. The server layer carries two guards that stop it from certifying a timeout for a round an honest validator has already left, and one added abstract action for a round ended by a quorum of nil precommits. These are changes to the checked model, and the last weakens the target specification, so the target is an extension of the object of LiDO [47], not that object. The mapping from the server layer to it also simplifies four things: SUSPICIOUS is mapped to ANCHORED; a single fixed quorum, with local time, is distinguished in the abstract object; every validator has the same voting power; and the Bitcoin height is translated by $K_{\text{DEEP_FINALITY}}$. The obligation is a safety obligation only; refinement of the liveness properties is outside it. These choices bear on what a completed run of C1 would establish, so we state them here.

Tool limit. An inductive check of every server action would avoid the composition gap. It is set up as `MC_ServerRefinementStep_Apalache.tla`: the driver typechecks under Apalache 0.58.3, and the invariant-preservation base case holds for the initial states (no error, about four minutes). The check that the initial states map to abstract initial states stops in the SMT backend, which does not implement a set filter over a powerset, so the step obligation is not reached and the refinement rests on TLC alone.

Earlier ablation runs with Apalache. An earlier version of the specification (3 August) was also checked with Apalache's bounded SMT search, which reached the violations of the circuit breaker (depth 2, 66.7 s), hysteresis (depth 6, 117.0 s), the P2P gate (depth 3, 24.2 s) and DA consistency (depth 2, 34.5 s). On the current specification the SMT solver returns "unknown" at depth 2 for the hysteresis and P2P rows, so the five rows of Table 7 are TLC results.

Liveness properties and the status of their checks

Table 19 lists the properties, the hypotheses they need and the status of each check.

Table 19: Liveness properties, the hypotheses they need and the status of their check

Property	Layer	Hypotheses	Status
Circuit-breaker liveness; recovery initiation; complete re-anchoring	FSM	FSM progress	Checked with TLC (Table 6)
Consensus liveness after GST	Server	all four	Specified; check not completed (4 h 19 min, depth 7, about ten million states)
Proof generation; persistent-eclipse resolution; forced inclusion	Server	all four	Specified; not checked
Necessity of timeout progress (remove the $f+1$ fast-forward)	Server	(tests timeout progress)	Not completed: depth 12, 69,408,583 states generated, 828,673 distinct, no violation

Names in the specification: `CircuitBreakerLiveness`, `RecoveryAttemptLiveness`, `CompleteRecoveryLiveness` (FSM layer); `EventualDecisionUnderGSTLiveness`, `ZKProofGenerationLiveness`, `PersistentEclipseResolutionLiveness`, `ForcedInclusionLiveness` (server layer).

TLC run configurations

Table 20: TLC run configurations

Run	Constants and bounds
Tendermint (safety)	N=4, f=1 (three honest, one Byzantine node); MaxRound 2; MaxBTCHeight 10; MaxEngramHeight 10; MaxTimestamp 2; MaxIgnoreRounds 2; DAThreshold 2; BTCThreshold 4; symmetry reduction over the three honest nodes
FSM (safety, liveness)	SuspiciousThreshold 2; SovereignThreshold 4; DAThreshold 2; RecoveryHysteresisWait 2; SuspiciousHysteresisWait 2; MaxSuspiciousTime 2; DownHysteresisThreshold 2; MaxDownHysteresisThreshold 8; StableWindow 2; MinPeers 3; MinSubnetDiversity 2; MinAnchorPeers 1; MaxChurnRate 10; MinAvgTenure 100; MaxPeerLatency 50; the Bitcoin and Engram heights stay below 10 (safety run); no symmetry reduction

F. Experimental Setup: Details

This appendix holds the testbed configuration, the FSM parameters, the fault schedules and the synthetic workloads summarized in Section 6.

Testbed configuration and software versions

Table 21: Testbed configuration and software versions

Component	Testbed A
Host	Apple M1 Pro (10 cores: 8P + 2E), 32 GB RAM, 1 TB SSD; macOS 27.0
Container runtime	Docker 29.0.1; VM allocation 10 CPUs, 23.4 GiB RAM
Validators (7)	No CPU or memory limit per container
Consensus core	CometBFT v0.39.3 module path, replaced by the fork `engram-consensus-core` at commit `493a20121`
Application	Cosmos SDK v0.54.3; this repository at commit `1c34d994e7ad` for the prover and tail runs on the multi-host testbed and `3fe656dce021` for its composite, load and hung-RPC runs
Bitcoin	Bitcoin Core v25.0, regtest, two nodes (0.3 CPU, 512 MB each)
Celestia	celestia-app v1.11.0, private one-validator chain with a block every 11.0 s (0.5 CPU, 1 GB); seven celestia-node v0.14.1 bridges (0.3 CPU, 512 MB each)
Prover	nargo 1.0.0-beta.22, `bb` 5.0.0-nightly.20260522 (6 CPU, 8 GB limit)
Fault injection	Pumba (image tag `latest`) driving kernel `tc netem`
Formal tools	TLC 2026.04.18 (rev. c45c125) on OpenJDK 11.0.31; Apalache v0.58.3 on OpenJDK 21.0.2

CPU and memory figures are container limits. The fork's differences from upstream CometBFT are listed in Section 6.1.

Table 22: Testbed B: hosts and round trips (recorded on 11 October 2026)

Hosts	Region	Type (vCPU, RAM)
Validators 1–2	us-east-2	m5.large (2, 8 GB), Xeon Platinum
Validators 3–4	eu-west-1	m5.large (2, 8 GB), Xeon Platinum
Validators 5–7	ap-southeast-1	m5.large (2, 8 GB), Xeon Platinum
Bitcoin, celestia-app	us-west-2	c5.2xlarge (8, 16 GB), Xeon Platinum 8124M
Prover host	us-east-1	c7a.xlarge (4, 8 GB), EPYC 9R14
Proving workers (1 to 4)	us-east-1	c7a.2xlarge (8, 16 GB), EPYC 9R14

Round trip (TCP connect time, best of five; ICMP is not open)	ms
us-east-2 to eu-west-1	77
us-east-2 to ap-southeast-1	200–207
eu-west-1 to ap-southeast-1	163–173
Validators to the Bitcoin host: from us-east-2, eu-west-1, ap-southeast-1	46–47, 115–116, 172–173
Prover host to validator 1, to the Bitcoin host, to a worker	12, 57, under 1
Within a region	under 1

Clock offsets against NTP (`chrony`) were under 7 µs on every host. Software versions are those of Testbed A, in images for `linux/amd64`. The raw record is `scripts/e6_zk_reanchoring_benchmark/results_live/cloud_testbed.txt`.

The round trips were recorded on 11 October. The runs with background sensor reads were made on 16 and 17 October and the vanilla column of the load comparison on 12 October, on freshly created instances of the same types; the round trips were not recorded again.

FSM parameters

Table 23: FSM parameters used in the live experiments

Parameter	Value	Unit and role
`SuspiciousThreshold`	5	Bitcoin gap, blocks
`SovereignThreshold`	8	Bitcoin gap, blocks
`DAThreshold`	30	DA gap, blocks
`KDeepFinality`	2	Reorg depth, Bitcoin blocks
`MinPeers`	3	Clean peers
`MinSubnetDiversity`	2	Distinct subnets
`MinAnchorPeers`	2	Active anchor peers
`MaxChurnRate`	5	Peer churn events
`MinAvgTenure`	300	Mean peer tenure, s
`MaxPeerLatency`	200	Round-trip time, ms
`RecoveryHysteresisWait`	2	RECOVERING→ANCHORED, blocks
`DownHysteresisThreshold`	2	ANCHORED→SUSPICIOUS, blocks
`MaxDownHysteresisThreshold`	2	Cap on the backed-off threshold (equal to `DownHysteresisThreshold`: static)
`MaxUnprovenTailBlocks`	128	Tail after the proved checkpoint that still counts (default 8)
`ProofOnlyRecovery`	true	Recovery requires a verified proof (default false)
`StableWindow`	16	Healthy ANCHORED blocks that lower the flap level by one
`SuspiciousHysteresisWait`	2	SUSPICIOUS→ANCHORED, blocks
`MaxSuspiciousTime`	24	SUSPICIOUS escalation bound, blocks

Overrides: the Docker testbed runs the table's `DownHysteresisThreshold` of 2 with the cap equal to it, so the down threshold is static and the optional backoff is disabled, and sets `SuspiciousHysteresisWait` to 8, which is not adaptive. The hysteresis sweeps of Section 7.3 vary the down threshold, a static value being `MaxDownHysteresisThreshold` equal to `DownHysteresisThreshold`; the Monte Carlo study of Section 7.1 uses the P2P thresholds of Section L.

Fault schedules

Table 24: Fault schedules of the scripted runs

Step	Fault and mechanism	Duration
Composite run (standing WAN latency baseline)		
C1	Baseline, no fault	15 s
C2	Bitcoin fault: 500 ± 100 ms RPC delay on one `bitcoind` (profile length 2 min)	60 s
C3	All seven Celestia bridges stopped, added to C2	60 s
C4	Three cycles of 5% packet loss on validators 1 and 2 (WAN baseline suspended)	$3 \times$ (20 s on, 10 s off)
C5	Hold; bridges still stopped	30 s
C6	Heal: Bitcoin profile cleaned up, bridges restarted	30 s
C7	Wait for ANCHORED (limit 600 s)	until recovery
Seven-scenario sweep		
S1	Healthy baseline	15 s
S2	Checkpoint submission suspended on all validators	180 s
S3	All seven Celestia bridges stopped	60 s
S4	300 ± 50 ms delay on the six validator links of validator 1	120 s
S5	100% packet loss on validator 1	180 s
S6	One `bitcoind` and all seven bridges stopped	90 s
S7	Faults removed; wait for ANCHORED (limit 900 s)	until recovery

Synthetic workloads

Noise profile (in-process). The hysteresis sweeps drive the real state machine with sensor readings generated per block. Each block is disturbed independently with probability 0.2 and healthy otherwise, for 100 blocks, from a fixed pseudo-random seed (7 for the RECOVERING→ANCHORED sweep, 11 for the ANCHORED→SUSPICIOUS sweep), so every threshold value sees the same noise sequence. A disturbance sets sensors to a failing reading for one block. Critical disturbances set the Bitcoin gap to `SovereignThreshold`. Warning-level disturbances set the Bitcoin gap to `SuspiciousThreshold`, mark DA unavailable, or give the P2P sensor a snapshot with no clean peers, diversity, or tenure and only the minimum anchor count. The combined environments apply several at once.

Monte Carlo P2P study. A *snapshot* is one synthetic tuple of the six P2P sensor fields (subnet diversity, active anchors, clean peers, churn rate, mean tenure, latency). A benign snapshot draws each field inside its healthy range with a uniform random margin. An attack of *intensity* $x \in [0, 1]$ perturbs the fields it targets in proportion to x (for example, the relay attack adds x (`MaxPeerLatency`+150) ms of latency and leaves the other fields benign); $x = 0$ is benign and $x = 1$ is the full attack. Detectors are the six-bound profiler and the peer-count-only rule CleanPeers $\geq$ `MinPeers`. The study uses production-scale P2P thresholds: 8 subnets, 4 anchors, 20 clean peers, churn 5, tenure 300 s, latency 200 ms. Four measurements per cell follow, all from fixed seeds.

- *False-positive rate:* the share of 2,000 benign snapshots flagged unhealthy.
- *False-negative rate:* the share of 2,000 full-intensity snapshots flagged healthy.
- *Detection delay:* over 500 ramps, the number of snapshots until the first flag, with intensity rising $0.1, 0.2, \ldots, 1.0$ over 10 snapshots. The mean is taken over ramps that were detected.
- *Evasion threshold:* the largest intensity (on the 0–1 scale) at which a majority of 20 probes still read healthy, found by a 12-step binary search that queries the detector, so it bounds an adversary with oracle access.

The ramp-shape study uses 300 ramps per shape and the three shapes x (linear), $\sqrt{x}$ (front-loaded), and x^2 (slow). The upper 95% confidence bound on a rate observed as 0 in n trials is about $3/n$ (rule of three): 0.15% for $n = 2{,}000$ and 1% for $n = 300$. The detection delay is reported as the mean and the sample standard deviation over the 500 ramps of each cell. The study has three limits. The benign distribution, the attack directions and the intensity scale are our choices. Each attack perturbs only the fields it targets, along one direction. The evasion threshold varies one scalar, the intensity, by binary search against the detector, so the six dimensions are not searched jointly. The results therefore characterize the detectors' sensitivity within this generator.

Provenance of the reported results

Table 25 states, for each result of Section 7, what kind of measurement it is, what one replicate is, and which recovery mode the runs used. "Live" means a measurement on the seven-validator Docker deployment (Testbed A) against real Bitcoin regtest and Celestia processes; "in-process" means the real FSM code driven by a simulated sensor input; "model-based" means a conversion or a prediction computed from a model and not observed. "Proof-only" and "fallback" are the recovery modes of Section 4.7. Repeats of one case that share a chain are not independent deployments.

Table 25: Provenance of the evaluation results

Result	Kind	Replicate	Recovery mode
Single Byzantine proposer and nine single-actor attacks (RQ1, Appendix H)	Live	Attack runs on the seven-validator cluster; counts in Appendix H	Not the subject
Collusion suite A3–B7 (Table 38)	Live, with offline replay	Three repeats per case on one chain (33 repeats of A3–B7)	Not the subject
Threshold sweep (Table 39)	Live	Five runs per cell, each on a fresh chain	Not the subject
Weighted-aggregation checks	In-process (function level)	1,500 random quorum subsets per weight vector	Not applicable
P2P detector comparison	In-process, synthetic	Fixed seeds, 2,000 snapshots or 500 ramps per cell	Not applicable
P3 churn attack	Live	Three runs	Not the subject
P4 relay-latency attack	Live	Three runs, each on a fresh chain, 600 s recovery window; the figure shows an earlier single trial	Proof-only, bound 128 (runs); fallback (figure)
Baselines under peripheral faults (Table 8)	Live	Three trials per cell; baselines redeployed per trial	Proof-only, bound 128 (Engram, soft)
Composite fault, first run	Live	One trial	Fallback
Composite fault, repeats	Live	Five runs, each on a freshly deployed chain	Proof-only, bound 128
Prover runs with the code's default parameters, Testbed B	Live	One run each for one and two workers; superseded by the runs with the paper's parameters and not reported	Tail bound 8, heuristic allowed
Composite fault, Testbed B	Live, seven machines in three regions	Five runs, each on a freshly deployed chain, all started in ANCHORED	Proof-only, bound 128, two workers
Hung Bitcoin RPC, Testbed B	Live	One run per build, daemon frozen for 240 s	Not the subject
Freshness window of the snapshot rule, Testbed B	Live	One run per window (1, 2, 5 and 10 s), daemon frozen for 240 s	Not the subject
ExtendVote against added delay, Testbed B	Live	One 90 s window per delay (0, 100, 300 and 500 ms) and build, one chain per build	Not the subject
Recovery at boundary rounds 0, 1 and 2, Testbed B	Live, seven machines in three regions	Two recoveries per round, each set on one freshly deployed chain, with seven mutated proofs and a repeated submission in each, and one burst of 24 proofs per set	Proof-only, bound 128
Circuit tests of the boundary round and the quorum	In-process (nargo)	Eight cases of a script and ten unit tests, one run	Not applicable
Prover and unproven tail, Testbed B	Live	One run for each of one to four workers, 15 min in SOVEREIGN	Proof-only, bound 128
Load at 100 tx/s, Testbed B	Live	Three fresh deployments per configuration (vanilla, empty, sync, snapshot), one 60 s measurement each; an Engram chain is measured only after all seven validators report ANCHORED, and a deployment that did not reach it is not measured	Proof-only, bound 128 (Engram)
Seven-scenario sweep	Live	Seven runs, each on a freshly deployed chain (five on 8 October, two on 10 October with the sensor log kept, same binary)	Fallback
Hysteresis sweep (Table 45)	In-process, synthetic	One run per threshold, fixed seed	Not applicable
Hysteresis spot-checks and flapping runs	Live	Single runs per threshold; six three-phase runs; one stationary run	Not the subject

Result	Kind	Replicate	Recovery mode
What ends RECOVERING	Live	One run per setting; five runs with bound 128	Fallback and proof-only
Backfill of the SOVEREIGN interval (Table 40)	Live	Three runs per outage length, nine in all	Fallback
Withheld-data controls (Table 41)	Live	As in the table	Fallback
Overhead against vanilla (Table 10)	Live, paired	Twenty paired deployments on one host, 20 tx/s	Not applicable
Certificate size and `ProcessProposal` time	Microbenchmark	Repeated calls in a process	Not applicable
Load sensitivity (Table 43)	Live	Five repeats per rate on one host	Not applicable
Exposure in mainnet time	Model-based	Conversion by the thirty-fold Bitcoin acceleration	Not applicable
False escalations per day at mainnet cadence (Appendix K)	Model-based (simulation)	Simulated Bitcoin block intervals; not observed on the testbed	Not applicable
Proving and witness times (Appendices D and K)	Offline measurement	Prover run on a recorded header range, not on the live chain	Not applicable
Model checking (Section 5)	TLC and Apalache runs	One run per configuration, bounds in Table 20	Not applicable

G. Network Emulation Parameters

Table 26 lists every `tc netem` profile, applied through Pumba. No correlation or distribution option is set, so delay jitter is uniform and loss is independent per packet (the `netem` defaults). Profiles that name no interface use Pumba's default interface (`eth0`), which carries Bitcoin, Celestia, and RPC traffic rather than the dedicated pairwise validator links. Only the S4 scenario and the relay-latency attack (P4) of Section 7 target the validator links explicitly.

Table 26: Network emulation profiles (Testbed A)

Profile	Effect and target	Duration	Used by
`chaos-delay`	100 ms delay, 20 ms jitter, all seven validators	5 min	baseline
`chaos-loss`	5% loss, validators 1 and 2	2 min	composite
`chaos-eclipse`	100% loss, validator 1	3 min	S5
`chaos-btc-delay`	500 ms delay, 100 ms jitter, one `bitcoind`	2 min	composite
`chaos-wan-latency`	Per-validator delay and jitter, Table 27	10 min	WAN baseline
`chaos-wan-loss`	Per-validator loss, Table 27	10 min	WAN variant
S4 eclipse	300 ms delay, 50 ms jitter on the six validator links of validator 1	2 min	S4
P4 relay latency	350 ms delay, 50 ms jitter on all P2P interfaces of one validator	244 s	P4

Table 27: WAN baseline profiles per validator (one profile of seven emulated regions)

Validator	1	2	3	4	5	6	7
Delay (ms)	15	70	140	45	100	30	180
Jitter (ms)	3	15	25	10	20	6	30
Loss (%, loss variant)	1	3	6	2	4	1	5

Each validator receives one delay value on its egress, so the profile is a per-validator offset rather than a pairwise round-trip-time matrix. A pairwise matrix between regions is not measured on Testbed A. It is measured on Testbed B (Table 22).

H. Extended Attack-Resilience Results

Table 28 reports the ten-scenario single-actor attack suite, rerun for this paper on the seven-validator testbed (n=7) with three repeats per scenario. The repeats of a scenario share one chain, each starting from ANCHORED, so they are not independent deployments. A3 and A4 are the cases of the collusion suite (Section 7.1) with one Byzantine validator. The column *Byz. VP* gives the share of voting power held by the misbehaving validator: one of seven validators is 14.3%, and the attacker containers of A1 and A2 are non-validators (0%).

Table 28: Single-actor attack suite at n=7 (live; three repeats each; safety held and zero AppHash divergence in all 30 repeats)

#	Attack	Byz. VP	Attack-specific signal
A1	Eclipse (10 attackers, one subnet)	0%	Filter held in 3/3: peak 8 peers in the target subnet, equal to the limit of 8; nil-prevote ratio 0.0. The FSM left ANCHORED for SUSPICIOUS in 2/3 repeats
A2	Multi-subnet Sybil (12 attackers, 4 subnets)	0%	Filter held in 3/3 (peak 8 of 8). The FSM left ANCHORED for SUSPICIOUS in 2/3 repeats
A3	Data withholding	14.3%	Not rejected by honest validators; replay clean
A4	Forged Bitcoin receipt	14.3%	Rejected by honest validators (216 rejections over 3 repeats)
A5	Withdrawal during SOVEREIGN	—	Blocked in 3/3; the chain did not reach SOVEREIGN and the transaction never committed
A6	Malicious proposer (forged FSM state)	14.3%	Safety held in 3/3 (Section 7.1)
A7	Censorship	14.3%	Censoring observed in 3/3; liveness held; honest reject signals stayed at 0
A8	Combined (A6 + A1)	14.3%	Safety held under simultaneous overlap
A9	Double-signing	14.3%	6/6 honest validators logged real duplicate-vote evidence in 3/3 repeats, detected one block after the offense
A10	Timeout flood	14.3%	Cadence held: 0.754→0.785, 0.791→0.791 and 0.816→0.783 blocks/s (baseline→flood)

The SUSPICIOUS excursions in A1 and A2 occur in the attack phase and again in the recovery phase, after the attacker is gone, and the baseline phase was always ANCHORED. A control run of A1 without an attacker (three runs, the same phases and timing, seven validators) shows the same excursions: the validators were in SUSPICIOUS in 17% to 19% of the samples and never in SOVEREIGN. The excursions of A1 are therefore the sensor noise of the testbed, which the collusion suite's control c0 also shows, and not an effect of the attack. A2 has no control of its own. A3 and A4 were instrumented only for safety, divergence and rejections, without an attack-specific metric such as A5's blocked-withdrawal flag or A9's detection latency. No scenario drove block production toward zero.

Function-level checks of the weighted aggregation

The testbed and the TLA+ model give every validator the same power, where weight and count coincide, so a defect that confuses them would not appear in any run above. We therefore tested the aggregation function alone (`tests/weighted`) on three weight vectors that sum to 100, $[30, 20, 15, 10, 10, 10, 5]$, $[26, 19, 17, 13, 11, 8, 6]$ and $[34, 33, 33]$, and on seven equal weights. Against a reference weighted median (the element at position $\lfloor T/2 \rfloor$ of the multiset in which each value is repeated by its power) the function agreed on 1,500 random quorum subsets with random values for every vector, and a verified power of exactly $\lfloor 2W/3 \rfloor+1$ was accepted and one below it rejected for every subset. For every Byzantine coalition and every set of certificates the

proposer may omit while keeping a quorum, with the adversary reporting the lowest value, the highest, or a split, the aggregate stayed within the honest values and honest booleans were not flipped whenever the coalition's power was below $W/3$ (up to 30 of 100 for the first vector and 33 for the other two). A coalition of 35 (20 and 15) in the first vector, and of 34 in the other two, escaped the honest range by having the proposer omit enough honest certificates, so the bound is tight. This checks the implementation of the weighted median and the quorum, not the integration with the genesis power assignment, which a live run with unequal power would add.

Proposer influence. For n from 4 to 10 equal-power validators and 30 random sets of distinct honest values each, we ran the aggregation on every subset the proposer may choose (all subsets of at least $\lfloor 2n/3 \rfloor + 1$ observations). The smallest and largest aggregate matched the order statistics of Section 4.3 in every case.

P2P detector sensitivity

A P2P detector is useful for safety only if its signal cannot be abused by a minority. We compare the six-bound profiler (Section 4.2) with a peer-count-only detector in a Monte Carlo sensitivity study on synthetic attacks (2,000 benign and 2,000 attack snapshots plus 500 ramps per cell; Section L); the two most diagnostic attacks were then run live (Section 7.1).

Table 29: P2P detector accuracy: six-bound profiler against peer-count-only (synthetic sensitivity analysis; 2,000 benign and 2,000 attack snapshots, 500 ramps per cell)

	Six-bound profiler		**Peer-count-only**	
Attack	**Delay**	**Evasion**	**Delay**	**Evasion**
P1 Peer-slot exhaustion	2.23 ± 0.87	0.17	never	1.00
P2 Multi-subnet Sybil	1.00 ± 0.00	0.00	never	1.00
P3 Churn-based rotation	1.70 ± 0.61	0.12	never	1.00
P4 Relay-node latency	2.71 ± 1.19	0.25	never	1.00

Delay: mean ± sample standard deviation of the snapshots to first detection over the 500 ramps of 10 snapshots each ("never": no ramp was detected). Evasion: largest attack intensity on the 0–1 scale at which the detector still reads healthy. Both detectors have 0/2,000 false positives (95% upper bound 0.15%). The profiler has 0/2,000 false negatives at full intensity; the peer-count-only detector has 2,000/2,000, since each attack preserves the clean-peer count.

Table 29 is a sensitivity analysis of the two detectors against attack shapes that we define (Section L); it does not measure robustness against arbitrary eclipse or Sybil strategies. Within these four attacks the profiler flagged every full-intensity snapshot (0 misses in 2,000) and every ramp (300 of 300 in each cell under three ramp shapes, linear, front-loaded and slow; 95% upper bound on the miss rate 1%), regardless of concurrent Bitcoin or DA degradation. It reads attacks of low intensity as healthy: up to 0.25 for P4, 0.17 for P1 and 0.12 for P3. An attacker who keeps every field within its bound is not detected by the six bounds by construction (Section 4.2). The peer-count-only detector detected none of the 300 ramps in any cell.

I. Bitcoin Reorg Depth Reaction

Section L states a threat-model boundary – safety is modeled given Bitcoin reorganizations no deeper than a configurable finality depth $K_{\text{DEEP_FINALITY}}$ – as a precondition of the formal result. This appendix tests the implementation at and beyond that boundary with forced Bitcoin reorganizations, each confirmed from the resulting block hashes, at five depths in one seven-validator run: {1, 2, 3, 5, 15}, with $K_{\text{DEEP_FINALITY}}$=2 and a suspicious-state threshold of 5.

Figure 4 shows the state of the validators over time at each depth.

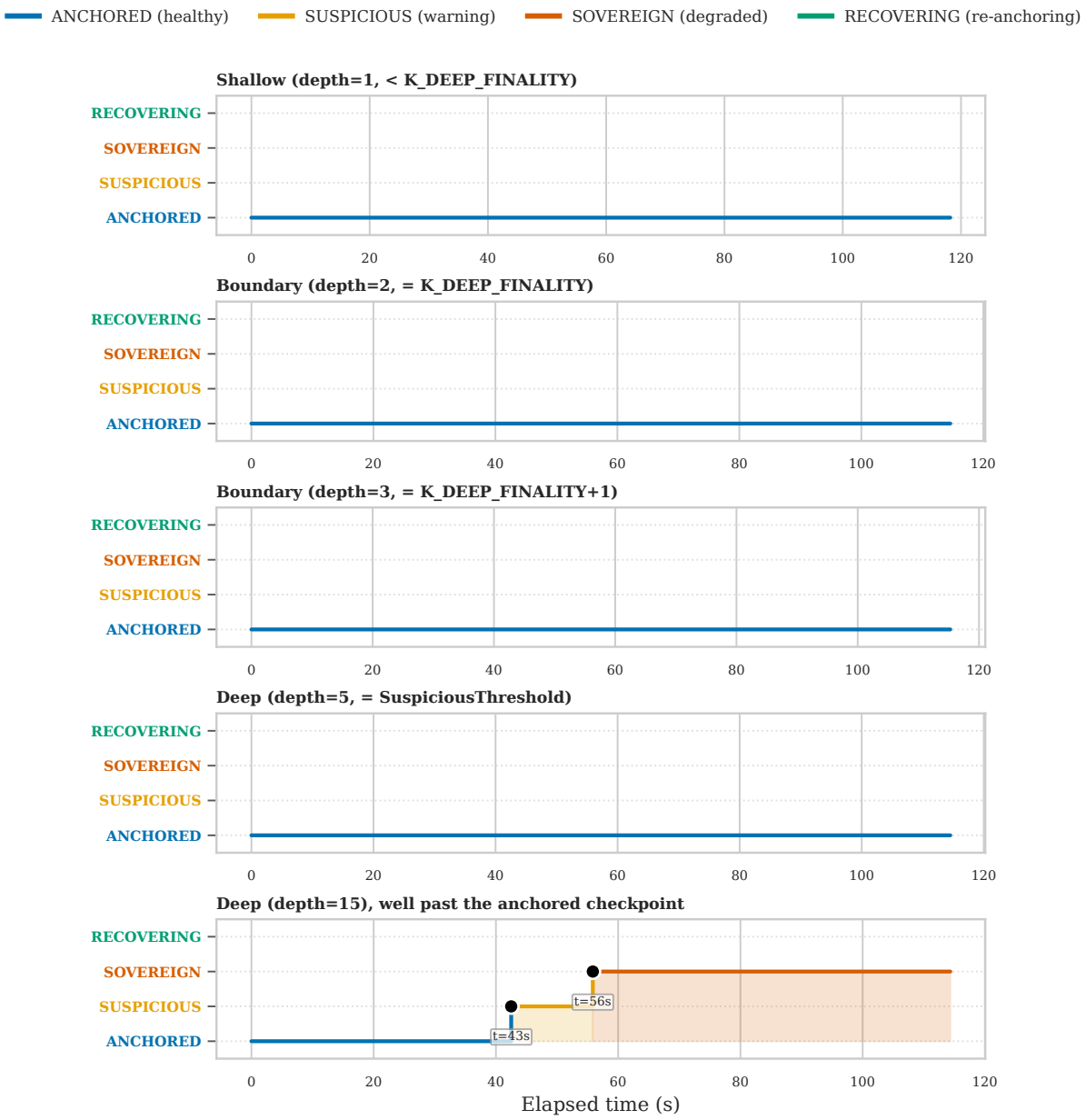


Figure 4: FSM reaction to Bitcoin reorgs at five depths, one trial per depth, seven-validator testbed (Testbed A). In each trial all 7 validators agree at every one of the 181 polled rounds (1,267 validator samples per trial, 0 RPC errors).

Table 30 summarizes the result. Depths 2–5 orphan real, independently-confirmed blocks (depth 1 is below the finality depth), yet none of them trips the FSM within the observation window: all 7 validators stay ANCHORED throughout. At depth 15, all 7 validators transition ANCHORED→SUSPICIOUS at t=43 s and →SOVEREIGN at t=56 s, in lockstep. This path rules out a direct critical-condition jump: the FSM's ANCHORED branch bypasses SUSPICIOUS entirely when the committed certificate's anchor-reverification flag is set, but the observed path instead passes through SUSPICIOUS, and the suspicious-duration counter reached 23 of the 24-block escalation bound (MaxSuspiciousTime) in the last sample before the SOVEREIGN transition. The depth-15 reaction is attributable to this duration-based escalation, not to anchor re-verification failing.

These runs measure the reaction to the reorganization only: the chain did not enter RECOVERING within the observation window, so the recovery mode does not affect them. The crossover between depth 5 (no reaction) and depth 15 (reaction) was not swept at finer granularity. Whether anchor re-verification ever set the anchor-reverification flag during this run is not established by the available logs – the per-validator sensor value was not captured, only the committed FSM state and suspicious-duration counter. The data that is available shows the depth-15 escalation came from the suspicious-duration bound, not from a direct critical-condition jump.

Table 30: FSM reaction to real Bitcoin reorganizations by depth

Depth	**Reorg confirmed?**	**FSM reaction**
1 (below K_{DEEP})	N/A (too shallow)	Stays ANCHORED
2 (= K_{DEEP})	Yes	Stays ANCHORED
3 (= K_{DEEP}+1)	Yes	Stays ANCHORED
5	Yes	Stays ANCHORED (within window)
15 (past checkpoint)	Yes	SUSPICIOUS (43s) → SOVEREIGN (56s)

J. Exploratory Extension: An Adaptive Down Threshold

This appendix supports Section 7.3. The extension is exploratory, and its results do not support claiming it as a contribution.

A static threshold trades reaction time against noise immunity, and the right point depends on network conditions that change over time. This appendix evaluates an extension that tries to follow them. We report it as exploratory and do not claim it as a contribution; the summary at its end gives the reason. We extend the backoff already used on RECOVERING to the down threshold $k_{\downarrow}$ of both demotion edges (Section 4.5). The effective threshold is $k_{\downarrow} = \min(k_{\max}, k_0 \cdot 2^c)$, where the flap level c rises by one at each such demotion and falls by one after T_{stable} healthy ANCHORED blocks. $k_{\downarrow}$ depends only on committed state, so it remains deterministic across validators. It starts at the responsive value k_0 and rises only when the network proves noisy, which also raises the cost of the repeated-trigger attack discussed in Section 8. The mechanism is implemented in the specification and in the prototype. We swept the down threshold live on the seven-validator cluster (Table 31): static $k \in \{2, 4, 6, 8\}$, each its own cap, and the adaptive threshold (k_0=2, $k_{\max}$=8), under wide-area latency jitter with no fault, under DA blips (all seven bridges stopped and restored in cycles of tens of seconds), and under a sustained DA outage (all bridges stopped) to measure detection. All 33 runs completed. A static value keeps no hidden state, so one genesis served all its trials and environments; the adaptive configuration was redeployed for every run because its flap level lives on the chain. Static k=2, k=6 and the adaptive threshold have three trials each; k=4 and k=8 have one.

Under jitter alone the static default of 2 flapped nine times per validator and spent 82% of the run in ANCHORED, and every static value from 4 up held ANCHORED without a flap. The adaptive threshold did not behave like the larger values: 8.3 flaps and 85% uptime, within the interval of k=2. Under DA blips no configuration stayed in ANCHORED (1–3% uptime), so the threshold did not matter there. Detection delay grew with the threshold, from 9.4 s at k=2 to 14.6 s at k=8 (one run), and the adaptive configuration sat between k=2 and k=6 at 11.7 s. In this sweep the adaptive threshold therefore bought no flap suppression over a static k=2, whereas a static k of 4 or more suppressed all flapping at a cost of under 2 s of detection delay for k=4; with one trial at k=4 and k=8 this is an observation, not an estimate. We did not measure the cost the adaptive threshold imposes on a repeated-trigger attacker, which is its stated purpose. A longer stable window is a candidate explanation for the weak suppression, since the flap level decays after 16 healthy blocks, about twenty seconds. We therefore repeated the adaptive configuration with a window of 64 blocks, under jitter and under the sustained outage only, three runs each, every run on a fresh genesis (last row of Table 31). Flaps fell from 8.3 to 3.0 per validator (all three runs gave 3), uptime rose from 85 to 93%, and the detection delay did not change (11.1 against 11.7 s). A window of 64 blocks recovers part of the gap, but not the full suppression of a static k of 4 or more; we did not test larger windows or the DA-blip environment with it.

Changing noise and a repeated-trigger adversary (simulation). The sweep above holds

Table 31: Live down-edge sweep on seven validators (300 s runs, mean ± 95% Student-t interval over the runs, none for a single run; flaps and uptime averaged over the seven validators of a trial). "Uptime" is the share of the run in ANCHORED; "Detection" is the delay from stopping the bridges of the sustained outage to the first non-ANCHORED state on one validator, and "To SOVEREIGN" the delay to SOVEREIGN.

Configuration	Runs	Flaps (WAN)	Uptime (WAN, %)	Uptime (DA blips, %)	Detection (s)	To Sovereign (s)
static k=2	3	9.0 ± 0.0	82 ± 3	1 ± 0	9.4 ± 0.4	38 ± 2
static k=4	1	0.0	100	1	11.1	41
static k=6	3	0.0 ± 0.0	100 ± 0	2 ± 2	12.8 ± 3.2	44 ± 5
static k=8	1	0.0	100	3	14.6	43
adaptive (2 → 8, T_{stable}=16)	3	8.3 ± 2.9	85 ± 7	1 ± 0	11.7 ± 0.9	42 ± 3
adaptive (2 → 8, T_{stable}=64)	3	3.0 ± 0.0	93 ± 2	–	11.1 ± 1.9	41 ± 3

Table 32: Static and adaptive down thresholds in the in-process simulation (blocks; mean ± 95% interval over 200 paired noise schedules). "Detect after noise" starts the outage as soon as the chain is ANCHORED again. Repeated trigger: demotions in 1200 blocks. Outage hidden: placements out of 35 of three Byzantine proposers that keep a real DA outage from being detected, with the chain quiet and with the flap level at its highest.

	Changing noise			Repeated trigger		Outage hidden
Threshold	**Detect, quiet**	**Flaps**	**Detect after noise**	**Naive**	**Patient**	**quiet / raised**
static k=2	2.0 ± 0.0	8.2 ± 0.5	2.0 ± 0.0	300	300	0 / 0
static k=4	4.0 ± 0.0	0.2 ± 0.1	3.9 ± 0.1	200	200	28 / 28
adaptive 2 → 8, T_{stable}=64	2.0 ± 0.0	3.4 ± 0.2	3.2 ± 0.2	1	18	0 / 35
adaptive 2 → 4, T_{stable}=64	2.0 ± 0.0	3.4 ± 0.2	3.1 ± 0.1	1	18	0 / 28

Table 33: Live three-phase runs: mean (range) over three runs per offset (the noisy phase pools six). Detection is the time from the bridge stop to the first non-ANCHORED state.

	Flaps	**Anchored share** noisy phase	**Detection, offset 0** s	**To Sovereign, offset 0** s	**Detection, offset 60** s
Static k=2	7.5 (7.0–9.0)	0.82 (0.80–0.84)	10.9 (10.7–11.4)	39.8 (38.7–41.3)	none: left ANCHORED in 3/3
Static k=4	0.0 (0.0–0.0)	1.00 (1.00–1.00)	13.7 (13.1–14.1)	43.7 (42.6–44.5)	11.5 (11.2–11.7)
Adaptive, window 16	6.8 (5.0–8.0)	0.84 (0.83–0.88)	12.7 (12.3–12.9)	41.3 (40.3–41.9)	none: left ANCHORED in 3/3
Adaptive, window 64	3.0 (3.0–3.0)	0.92 (0.91–0.92)	14.6 (13.2–15.8)	44.3 (43.1–45.4)	9.8 (9.7–10.0)
Adaptive, window 128	1.0 (1.0–1.0)	0.96 (0.95–0.96)	16.2 (13.1–20.6)	46.7 (43.0–51.2)	none: left ANCHORED in 3/3
Adaptive, cap 4, window 64	3.0 (3.0–3.0)	0.89 (0.80–0.92)	15.6 (14.1–17.5)	45.9 (44.8–46.6)	10.6 (10.3–10.8) [1/3 left ANCHORED]

the noise constant, the case in which a well-chosen static threshold needs no adaptation. The case for the adaptive threshold is noise that changes, and a party that triggers demotions on purpose. We tested both in-process with the keeper's own transition code. Noise is a sequence of bursts of one to three consecutive warning-level blocks, each starting with probability 0.03 per block; this reproduces the live demotion rate of a static threshold of 2 (4.6 demotions per 250 blocks, against 5.0 per 300 s live), and the same 200 noise schedules drive every configuration, so differences are paired. A run has 150 quiet blocks, 250 noisy blocks and a quiet phase, after which a sustained DA outage starts and we count the blocks until the first non-ANCHORED state. The adversary sends warning-level bursts for 1200 blocks and waits for ANCHORED between them: a naive one always sends k_0 blocks with one healthy block in between, a patient one waits for the flap level to return to zero first. The last column of Table 32 is the concealment simulation of Section 8.

Against a static threshold of 2, the adaptive one detects the outage as fast when the network is quiet (2 blocks), cuts the flaps of the noisy phase from 8.2 to 3.4, and absorbs the naive adversary (1 demotion against 300); a patient adversary obtains 18. Against a static threshold of 4 it does not win on flaps (3.4 against 0.2). What it keeps is the 2-block detection when quiet, about two seconds at the block rate here, and the resistance to concealment of the smallest threshold (Section 8). Its responsiveness returns slowly after noise: right after the noise the detection takes 3.1–3.2 blocks, and 2.0 after 96 quiet blocks. An adversary that reads the committed flap level pays 7.9 bad blocks per demotion with $k_{\max}$=8 and 4 with $k_{\max}$=4 (2 for a static threshold of 2). These are simulation results, calibrated to the live sweep above but not measured live.

Changing noise (live). We ran the same three phases on the seven-validator cluster: a quiet phase, a phase under the wide-area latency and jitter profile of the stationary sweep, and a quiet phase after which, once the chain was ANCHORED again and had stayed so for an offset of 0 or 60 s, all Celestia bridges were stopped. Every run used a fresh genesis, with three runs per configuration and offset (Table 33 and Fig. 5; the noisy-phase columns pool the six runs of a configuration).

A static threshold of 4 never flapped and stayed ANCHORED through the noisy phase of all six runs, at a detection delay of 13.7 s against 10.9 s for k=2, which flapped 7.5 times and spent 82% of the phase ANCHORED. The adaptive threshold suppressed more as its window grew, 6.8

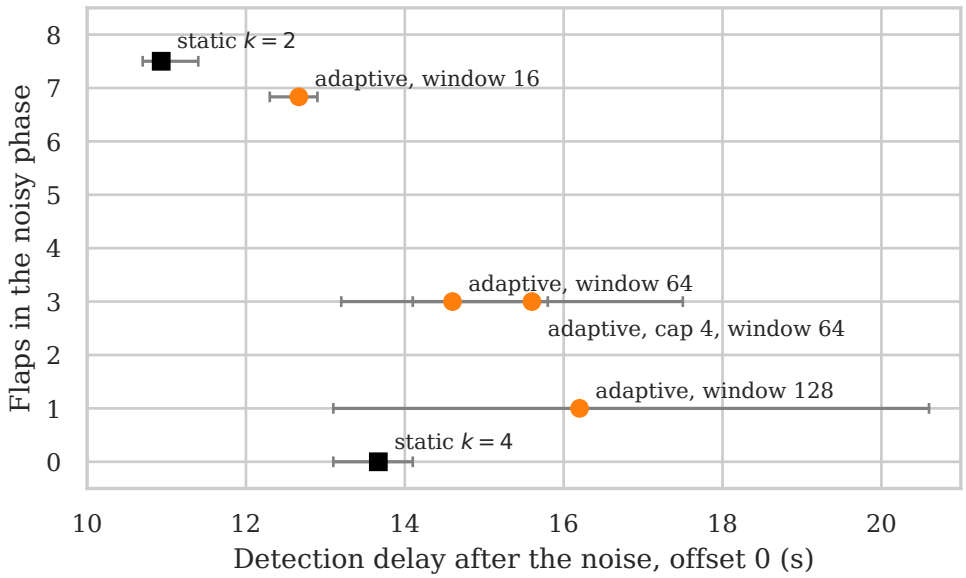


Figure 5: Flaps against detection delay in the live three-phase runs. Squares are static thresholds, circles the adaptive ones; the bars span the three runs at offset 0. Lower and to the left is better.

flaps for a window of 16 blocks, 3.0 for 64 and 1.0 for 128, which is close to the simulation for windows of 16 and 64 (6.1 and 3.4 flaps). It paid for that in detection at offset 0, 12.7, 14.6 and 16.2 s. No adaptive configuration reached the flap count of the static threshold of 4 at a shorter delay; the only one faster than it, with a window of 16, flapped 6.8 times. After an offset of 60 s the adaptive level had decayed: with a window of 64 the detection took 9.8 s against 11.5 s for the static threshold of 4, the ranges not overlapping, though with three runs each this is an observation, not an estimate. For k=2, a window of 16 and a window of 128 the chain left ANCHORED before the outage in all three runs at that offset, so no detection delay was measured; we did not identify the cause.

Repeated trigger (live). The live adversary stops every Celestia bridge for a nominal pulse of 3 s and restarts them, 21 to 23 times in about ten minutes (15 runs, three per configuration, each on a fresh genesis). Stopping and restarting take time: the bridges were down for 15–17 s per pulse on average, far longer than the nominal pulse and than any threshold, so a pulse is a full DA outage and not the short warning-level burst of the simulation. In all 15 runs, whatever the configuration, the chain left ANCHORED at the first pulse (one demotion), reached SOVEREIGN, and spent 0.2–2.2% of the run in ANCHORED. This experiment therefore does not separate the thresholds, and it gives no live evidence that the adaptive threshold absorbs short pulses: the simulation remains the only evidence for that case, and a live adversary needs a fault faster than stopping a bridge to produce pulses shorter than the threshold.

Summary of the extension. In the stationary live sweep the adaptive threshold behaved like k=2 (8.3 flaps, 85% uptime), and with a window of 64 blocks it flapped 3.0 times. In the live three-phase runs the flaps fell with the window (6.8, 3.0 and 1.0 for 16, 64 and 128 blocks) at detection delays of 12.7, 14.6 and 16.2 s, against no flap at 13.7 s for a static threshold of 4. The live repeated-trigger runs did not separate the thresholds, and the case the extension is meant for, absorbing short repeated pulses, rests on the simulation alone. We therefore treat a static threshold of 4 or more as the setting to use (Section 8) and the backoff as a result that did not hold up.

K. Security Envelope: Details

This appendix holds the analyses behind Section 8: the exposure before the lock at mainnet cadence, the choice of the down threshold, and the proving latency of the recovery proof.

Exposure before the lock at mainnet cadence

Section 8 converts the measured lock time to mainnet cadence by scaling. To check that conversion we drove the real FSM with Bitcoin blocks and a model of the checkpoint cycle of the anchor tracker: one checkpoint is pending at a time, it is included in the next Bitcoin block, it counts as confirmed with `KDeepFinality`+1 confirmations, the next one is submitted at once, and the committed anchored height follows the tracker after a lag of two to three Engram blocks, the one free parameter. At testbed cadence the model reproduces what we measured: the checkpoint pause reaches SOVEREIGN after 129 s (5th to 95th percentile 91 to 149 s; measured 125 s in RQ2 and 141.6 ± 13.8 s in S2), and the quiet phase of the live sweeps is 0.80–0.84 in ANCHORED at k=2 and 1.00 at k=4 (measured 0.79–0.87 and 1.00). At mainnet cadence the same fault reaches SOVEREIGN after 49.6 minutes with steady 600 s blocks (41 to 58) and 46.7 minutes with exponential block intervals (18 to 79), which is about five Bitcoin blocks and 2,300–2,500 Engram blocks. This is lower than the 62 minutes and 3,100 blocks of Table 47, which scales the measured 125 s by thirty: the table is the plain conversion and the model follows the checkpoint cycle, so the two bracket the lock time at mainnet cadence.

The healthy gap is not flat. It follows the checkpoint cycle, 2, 3, 4, 5 and back to 2 every three Bitcoin blocks, and in the first 18 s of the five composite runs, before any fault, it sat at 5, the `SuspiciousThreshold`, in 3–16% of the samples, for about two Engram blocks each time. This has two consequences. The threshold cannot simply be lowered, since the healthy cycle already touches it. And part of the flapping at k=2 in the live sweeps is the cycle, not the wide-area noise: the model gives one demotion per cycle, 1,440 per day at the testbed cadence and 48 per day at mainnet cadence, so the accelerated testbed exaggerates the baseline flapping thirty-fold.

The model also predicts a limit of the current parameters at mainnet cadence. When two Bitcoin blocks arrive within one polling interval, the next checkpoint skips a block and the gap stays at 5 for a full Bitcoin interval, much longer than `MaxSuspiciousTime` (24 Engram blocks, about 29 s, chosen for the 20 s cadence), so the FSM escalates to SOVEREIGN without a fault. With exponential block intervals a healthy chain does so about once in eight days (10 events in 77 days of exposure, the same for k=2 and k=4) and never with steady blocks or at the testbed cadence. This is a prediction of the model, not an observation. The timeout must therefore follow the Bitcoin cadence of the deployment: the live value of 24 blocks is 1.4 Bitcoin intervals at the testbed's 20 s, and a value of about 1.5 intervals (750 blocks, 15 min, at 600 s) removes most false escalations at the price of about 14 more minutes to reach SOVEREIGN in a real fault (Table 34). We keep 24 as the default, which the testbed experiments use, and treat the timeout as a deployment parameter. A live check at a 600 s mining interval (steady blocks, three runs on fresh chains) reached SOVEREIGN after 50.6, 50.9 and 60.7 min (5, 6 and 6 Bitcoin blocks; 2,451, 2,477 and 2,921 Engram blocks), 54.1 min on average. Two runs lie inside the range of the model (41 to 58 min) and the third above it, close to the plain conversion of Table 47.

Table 35 lists ways to bound the exposure and what each costs. We implement none of them; the first two bound the number of withdrawals without a lower threshold, and the last one shortens the window.

Hysteresis and concealment

Concealment against noise. A higher down threshold k tolerates longer bursts of noise, and it also tolerates a longer run of reports that a Byzantine proposer interleaves with honest ones.

Table 34: `MaxSuspiciousTime` at mainnet cadence (simulation; exponential Bitcoin block intervals of mean 600 s, k=4): false escalations of a healthy chain per day (30 runs of up to 72 h; the counts behind the small rates are 10, 3 and 1 events, so they indicate order of magnitude only) and the mean time from a checkpoint pause to SOVEREIGN (40 runs).

Blocks	Time	False escalations per day	Mean to Sovereign, min
24	29 s	0.13	53
500	10 min	0.04	63
750	15 min	0.01	67
1500	30 min	0.01	76

Table 35: Ways to bound the exposure before the lock.

Mechanism	Effect	Cost
Withdrawal quota per Bitcoin block, graded by the gap	$N \leq$ (Bitcoin blocks in the window) $\times$ quota, independent of the Engram block rate and of the sender	Lower withdrawal throughput also when healthy
Release funds only after the block is anchored K deep	Exposure close to zero	Every withdrawal waits about three Bitcoin blocks; needs a real bridge
Sensor on the age of the pending checkpoint against the expected cycle	Window shortens from about six to three or four Bitcoin blocks (our estimate)	Changes the FSM input, so every experiment on the Bitcoin gap would be rerun

Within the assumption the choice of k does not matter: for each of the 21 placements of two Byzantine proposers in the seven-slot rotation, a real DA outage was detected after exactly k blocks ($k \in \{2, 3, 4\}$, simulation). With three Byzantine validators, outside the assumption, a Byzantine proposer can keep a certificate that reports a healthy DA layer, and an honest proposer then reports the outage in the blocks it proposes; the outage is detected only if enough honest proposers follow one another to reach k. In the simulation it was detected for all 35 placements at k=2, missed for 14 at k=3 and for 28 at k=4, and with four Byzantine validators it was missed for every placement. The live check at k=2 agrees: with three Byzantine validators the outage was detected in 5 of 5 runs. The adaptive threshold exposes the same weakness only while its flap level is high (0 of 35 placements hidden at level 0, 28 at level 1, all 35 at level 2), whereas a static threshold of 4 or more is exposed permanently.

A window rule. A rule that demotes when at least m of the last w blocks are bad, instead of after a consecutive run, is not cut by interleaved reports. In a reduced model of the ANCHORED↔SUSPICIOUS edge, validated against the keeper-based simulation above (static thresholds and the adaptive $2 \to 4$ reproduce its flaps, detection delays and hidden placements), 4 of 8 hid the outage in 0 of 35 placements, against 28 for a static threshold of 4, at 0.8 flaps against 0.2 and the same 4-block detection. It does not remove the cost of being fast: its detection is as slow as the static threshold, and an adversary that sends bursts of m blocks still obtains 200 demotions in 1200 blocks, where the adaptive threshold allows 1. Combining the two (an adaptive window) gave 4.6 flaps and 172 demotions, so no single rule won on all four criteria. We did not implement the window rule.

Live check of concealment. The live check at k=4 (three Byzantine validators with biased certificates, all seven bridges stopped for 120 s, a fresh chain per run) agrees with the simulation for adjacent placements and not for interleaved ones. With three consecutive Byzantine slots the outage was detected in 3 of 3 runs, as predicted. With slots 0, 2 and 4 of the rotation, which the simulation predicts would hide it, the chain also left ANCHORED and reached SOVEREIGN in 3 of 3 runs, with 6 to 9 blocks outside the honest range per run (6 to 12 over all six). We have not isolated why the prediction failed for the interleaved placement, so the concealment at k=4

rests on the simulation alone.

Choosing the down threshold. The experiments ran with `DownHysteresisThreshold` of 2, and the live runs show that value to be too low for a wide-area network: under the three-phase noise a static k=2 flapped 7.5 times and spent 82% of the noisy phase in ANCHORED, a static k=4 did not flap and stayed ANCHORED throughout, at a detection delay of 13.7 s against 10.9 s (an earlier single-run spot-check at k=4 did flap, Section 7.3). At the testbed's accelerated Bitcoin cadence the healthy checkpoint cycle reaches the suspicious threshold in 3–16% of the samples, and k=2 spent only 0.79–0.87 of the quiet phase in ANCHORED, against 1.00 for k=4; the model of the cycle accounts for this, though we did not isolate it live. We therefore recommend $k_{\downarrow} \geq 4$ as a deployment setting, set from the noise of the environment like `MaxSuspiciousTime` is set from the Bitcoin cadence, and keep 2 as the value of the reported experiments. The price is the concealment of the preceding paragraph: an outage is detected only if k consecutive proposers are honest, so the larger the threshold, the easier it is to hide one.

Proving latency of the recovery proof

Measured proving latency. In the first live deployment with chunk aggregation, the prover built a 512-header aggregate from two chunks in about 4.3 minutes and a 514-header aggregate from three in about 6.6 minutes; the tip then stood 211 and 321 blocks beyond the last covered header, close to the product of latency and block rate. Both proofs were accepted by `SubmitRecoveryProof`, but with the bound at 128 neither made the recovery flag hold, and the chain stayed in RECOVERING. A single 256-header proof takes about two minutes, about 110 blocks, and fits, but with little margin: at a block rate about 17% higher, or a slower prover, it would not, so the recovery times of the proof-only runs hold for this testbed's block rate and proving speed. With the bound raised to 512 blocks and everything else unchanged, the chain left RECOVERING through an aggregate proof in both of two deployments. In the first the prover built a 514-header proof from three chunks in 6.5 minutes, and the tip stood 309 blocks (about 6.4 minutes) beyond the last covered header when it was submitted; in the second a 512-header proof from two chunks took 4.6 minutes, with the tip 224 blocks (about 4.7 minutes) beyond. All seven validators were then in ANCHORED, 1,023 and 888 seconds after the chain started. Proof-only recovery was on, so only the proof could have restored it. The interval proved in these runs is the one since genesis, not one following an outage injected into a healthy chain, and the runs with the bound at 128 are a single deployment.

Aggregation bounds the verifier's cost, not the prover's. The aggregate is one more proof on top of the chunk proofs, so building it takes longer than building the chunks alone (49% longer at 512 headers and 22% at 1024 in Appendix D). The chunk proofs are independent given the chunk boundary roots, so they can be built concurrently and joined by the aggregator circuit.

Proving latency on dedicated hosts. Table 36 reports the proving step (`nargo execute` and `bb prove` of one segment, in the published prover image) on three `amd64` instance types, each alone on its host with no chain running. The witness generation is single-threaded and takes 43 s on the two newer types and 74 s on the older one; `bb` uses all eight cores. We chose the `c7a.2xlarge` for the proving workers of Testbed B from this table, that is, for its speed, before running the recovery experiments; the first run on each host, 4–7 s slower (cold cache), is left out of the means. On the multi-host chain the latency from the start of a proof to its accepted submission was 77 s with one worker and 76 s with two (73–87 s per proof): the proving step as seen by the prover host, 69–75 s including the transfer to the worker, and 2–17 s of witness preparation, publication of the audit trail and submission.

Table 37 reports the cost on the ten-core host with the testbed stopped, at 0.8 blocks per second. Proving and aggregation alone fit inside the bound of 128 blocks, with 65 blocks for 512 headers and 109 for 1024. The witnesses, which are prepared one chunk after another, take 108 s and

Table 36: The proving step of one segment on one host, mean of the runs after the first (two runs on the `c7a.2xlarge` and `c7i.2xlarge`, three on the `c5.2xlarge`; one host per type).

Instance	CPU	`nargo execute` (s)	`bb prove` (s)	Step (s)
c7a.2xlarge	AMD EPYC 9R14	43.2	21.4	64.7
c7i.2xlarge	Intel Xeon Platinum 8488C	43.0	27.7	70.7
c5.2xlarge	Intel Xeon Platinum 8124M	74.0	37.1	111.0

Table 37: Latency of building the recovery proof on one ten-core host (testbed stopped). Proving is the chunk proofs plus the aggregation; the tail is the proving time at 0.8 blocks per second. Sequential and parallel are means of two runs, except the parallel run at 1024 headers, which is one run.

		Sequential		Two provers	
Headers	Witnesses (s)	Proving (s)	Tail	Proving (s)	Tail
512	108	81.3	65	72.6	58
1024	213	136.5	109	118.9	95

213 s and are not part of that figure. With them the latency from the end of the interval to an accepted proof is about 189 s (151 blocks) for 512 headers and 349 s (280 blocks) for 1024, above the bound. Two concurrent provers with five threads each shorten the proving step by 11% and 13% because they share the cores; four concurrent provers did not start, since the host lacks the memory. A single leaf proof takes 28 s with all ten cores, 36 s with five and 60 s with two.

Whether the prover keeps up with block production is therefore not established. It needs a prover with cores of its own, and witnesses prepared chunk by chunk while the chain is still in Sovereign, so that only the last chunk and the aggregation remain on the critical path; we measured neither. The 4.3–6.6 minutes of the live deployment were measured while the validators ran on the same cores, so the latencies depend on the load of the host. The validators verify the submitted proof during block execution, so the verification cost does not change with how the proof was built.

Restart after a long chain

The publisher's contiguous height is kept in memory. A validator that restarts therefore republishes the chain from height 0 and reports DA as failed until it has caught up. In one run, four validators restarted on a chain of about 7,400 heights; the certified DA health stayed failed and the chain remained in Sovereign for more than half an hour. Persisting the contiguous height would remove the dependence on chain length; the prototype does not.

L. Extended Evaluation and Design Details

This appendix holds material that the main text refers to and that is not needed to follow its argument: tables, figures and analyses moved from the main sections, unchanged.

Safety evaluation: additional tables and the peripheral quorum gate

The suite fixes the number of Byzantine validators; the threshold itself is measured in a second sweep (Table 39) with $b \in \{0, 1, 2, 3\}$ validators and two adversary goals: forcing Sovereign through false degraded observations, and hiding a real DA outage by reporting it healthy. Each cell is five runs, each on a fresh chain started from Anchored. For $b \leq 2$ no block lay outside the honest range and the adversary reached neither goal (30 of 30 runs). For b=3 the aggregate left the honest range in 40–42 blocks and the adversary forced Sovereign in 5 of 5 runs, which is the boundary Lemma 1 predicts. Hiding an outage did not succeed at b=3 either: honest

Table 38: Collusion suite at n=7 (A3 and A4 are the single-actor cases it reruns; B8 is three runs of an honest split view with no adversary; c0 is the control)

Case	Byz.	Behaviour	Rejected by honest validators	Repeats	Same AppHash	Replay clean
a3	1	claims a DA attestation one height ahead	none	3	3/3	3/3
a4	1	forged Bitcoin checkpoint hash	btc verify receipt 216	3	3/3	3/3
b1	2	coordinated false degraded observations	none	3	3/3	3/3
b2	2	false observations; proposer biases the certificate	none	3	3/3	3/3
b3a	1	proposer keeps exactly a quorum of certificates	none	3	3/3	3/3
b3b	1	proposer keeps fewer than a quorum	sub quorum certificate 210	3	3/3	3/3
b4	2	double-signing at the same height	none	3	3/3	3/3
b5	2	withdrawal in a block of a locked state	withdrawal locked 320	3	3/3	3/3
b6	2	stale DA receipt; forged Bitcoin hash	btc verify receipt 155, da verify receipt 165	3	3/3	3/3
b7a	2	false observations with two honest validators eclipsed	none	3	3/3	3/3
b7b	2	false observations with three honest validators eclipsed	none	3	3/3	3/3
b8	0	honest split view: 3 then 4 of 7 eclipsed	none	3	3/3	3/3
c0	0	control: node04 restarted, no behaviour	none	3	3/3	3/3

Table 39: Threshold sweep: Byzantine validators b at n=7 (f=2)

Goal of the adversary	Byz.	Runs	Blocks outside honest range	Same AppHash	Adversary's goal reached
force SOVEREIGN	0	5	0	5/5	0/5
force SOVEREIGN	1	5	0	5/5	0/5
force SOVEREIGN	2	5	0	5/5	0/5
force SOVEREIGN	3	5	40–42	5/5	5/5
hide a DA outage	0	5	0	5/5	0/5
hide a DA outage	1	5	0	5/5	0/5
hide a DA outage	2	5	0	5/5	0/5
hide a DA outage	3	5	5–12	5/5	0/5

proposers' blocks still reported it (5–12 blocks outside the range). An outage can be hidden only when every run of consecutive honest proposers is shorter than the hysteresis threshold, and these placements did not produce that.

Peripheral P2P quorum gate. A P2P detector is useful for safety only if its signal cannot be abused by a minority. A Monte Carlo sensitivity study on synthetic attacks that we define shows that the six-bound profiler flags every full-intensity snapshot and every ramp, reads attacks of low intensity (up to 0.25) as healthy, and that a peer-count-only detector detects none of them (Appendix H); it does not measure robustness against arbitrary eclipse or Sybil strategies. We then ran the two most diagnostic attacks live. In P3 (churn-based rotation: eight restart cycles on two of seven validators, 29% of voting power), no FSM transition was committed in any of three runs (in the first, the churn metric on both attacked validators crossed its threshold): two validators' signed observations cannot move a certified aggregate backed by a quorum. In P4 (a sustained delay of 350±50 ms on all of one validator's P2P interfaces, which all seven validators observe), the chain escalated in each of three runs on fresh chains: all validators entered SUSPICIOUS 30–35 s and SOVEREIGN 60–66 s after the delay began. After the delay cleared, every run returned to ANCHORED at all seven validators, on average 279 s later (95% interval ±26 s, Student t). The return is paced by the recovery proof that proof-only recovery with an unproven tail of 128 blocks requires (Section 7.3), not by the hysteresis waits; an earlier single trial with the heuristic fallback returned within seconds (Fig. 6). Each run also showed a brief fall-back from RECOVERING to SOVEREIGN (Section 8.3).

P3 and P4 are two outcomes of one mechanism. The same kind of sensor-confirmed degradation is ignored when a minority observes it and drives a bounded cascade when all validators do, as Lemma 1 predicts: a minority cannot move the certified aggregate, whereas a degradation seen by every honest validator must move it.

Liveness and recovery: seven-scenario sweep, data availability and the adaptive threshold

Seven-scenario sweep. The composite trace does not isolate which failure mode drives which behavior. A second run therefore walks the chain through seven sequential scenarios (Table 24):

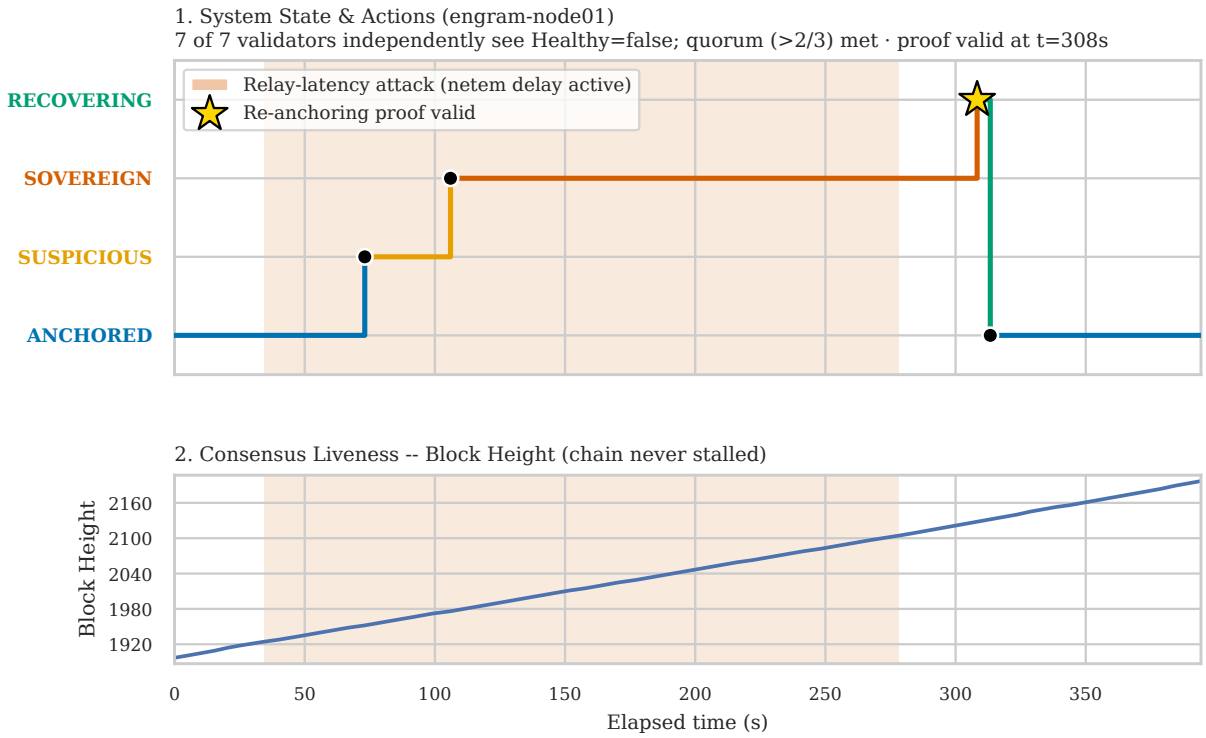


Figure 6: FSM cascade under a real, quorum-visible P2P relay-latency attack (P4): all 7 validators independently sense the degradation, the transition commits once quorum is met, and block production (bottom panel) never stalls through the cascade or the recovery. The shaded band is the delay window and the star marks the verification of the recovery proof. Single earlier trial with the heuristic fallback enabled; the three runs reported in the text used proof-only recovery.

S1 healthy baseline, S2 Bitcoin congestion (checkpoint submission suspended), S3 DA outage (bridges stopped), S4 partial P2P eclipse, S5 anchor-peer isolation of one validator, S6 combined Bitcoin and DA failure, and S7 recovery. Each scenario starts from the state the previous one left, and the chain is given up to 600 s to return to ANCHORED after S2–S5 and up to 900 s in S7, with the heuristic fallback enabled. We ran the sequence seven times, each on a freshly deployed chain (Fig. 7 shows the first run).

The chain kept committing blocks in every scenario of every run. S2 and S3 escalated through SUSPICIOUS to SOVEREIGN in all seven runs: S2 after 143.5 s on average (95% interval ±6.3 s, Student t, range 133.8–153.3 s) and S3 after 30.5 s (±1.6 s, range 27.5–33.5 s). S4 reached SOVEREIGN in all seven runs, after 72.8–110.7 s in the six runs that entered it from ANCHORED. S5 produced only a transient SUSPICIOUS excursion absorbed by hysteresis in all six runs that entered it from ANCHORED: isolating one of seven validators from its anchor peers leaves the certified active-anchor-peer count above zero, so the critical condition never holds. S4 and S5 are therefore distinct failure classes rather than two intensities of one signal. S6 entered SOVEREIGN in every run.

The return to ANCHORED after S7 is where the runs differ. The chain entered RECOVERING 16.6–19.6 s after the faults were removed in six runs and then fell back from RECOVERING to SOVEREIGN between one and six times before returning to ANCHORED at all seven validators. From entering RECOVERING to that return it took 331, 332, 334, 334 and 347 s in five runs and 645 s in the sixth, the run with six fall-backs (median 334 s; mean 387 s with a 95% interval of ±133 s, which the one long run dominates). The seventh run did not return: after S4 it stayed in SOVEREIGN through S5, S6 and the 898 s of S7, with all seven validators agreeing on it and the chain still committing blocks (Section 8.3). The statement that the chain returns to ANCHORED after the faults are removed therefore holds in six of the seven runs, and the recovery time is

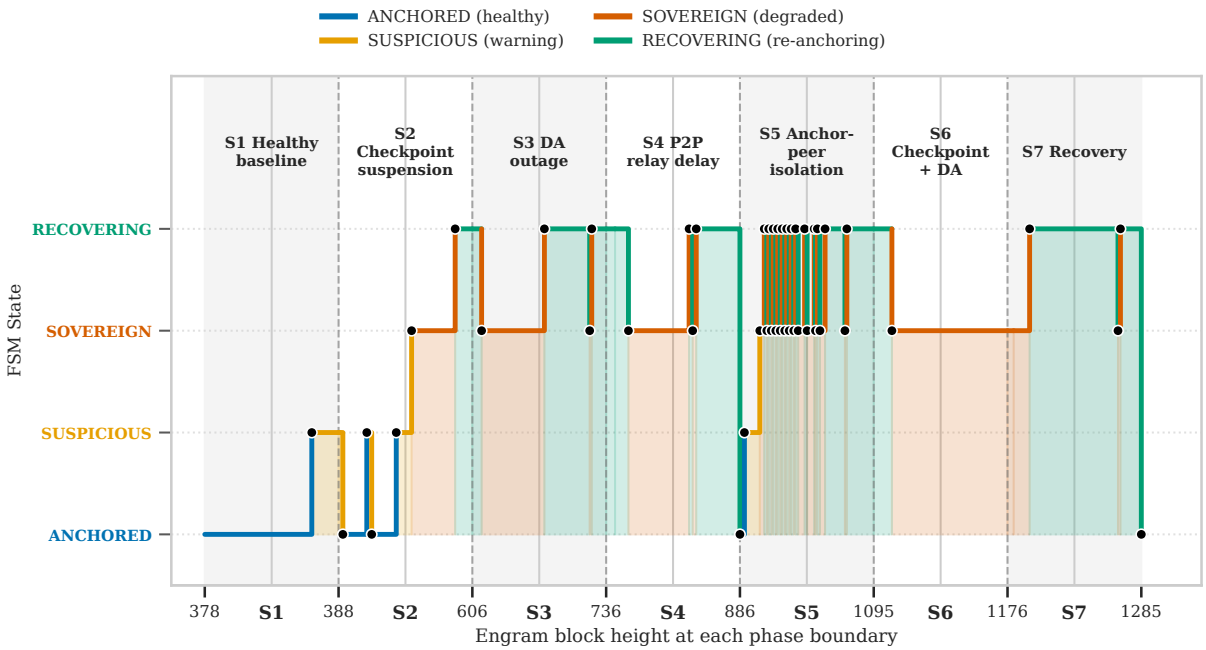


Figure 7: Continuous FSM-state timeline across seven sequential fault-injection scenarios (S1–S7), true block-height ranges, live 7-validator testbed. Each scenario begins where the previous scenario's recovery ended. Dots mark state changes; the lower axis gives the block height at each phase boundary, and every phase has the same width. First of seven runs, heuristic fallback enabled; the chain returned to ANCHORED at the end of S7.

long and variable under the heuristic fallback.

Data availability of the Sovereign interval. The exit condition of Section 4.7 requires the interval's block data to be republished before RECOVERING→ANCHORED. We stopped all seven Celestia bridges for 120, 300 and 600 s, restarted them, and after the chain returned to ANCHORED re-fetched every height of the interval from Celestia through `blob.GetAll`, comparing the bytes with the Engram block. Table 40 shows that all 2,291 heights over nine runs were found and identical. The chain entered SOVEREIGN 46–49 s after the bridges stopped, and the time in RECOVERING did not grow with the interval: 37–61 s for 75–77 blocks and 49–61 s for 465–475 blocks. The check is external to the chain: the gate itself rests on the quorum-reported contiguous height, and these runs show that height was truthful when all validators were honest. They cover a total DA outage on one host with three runs per length. All runs use one bridge blob limit; the 600 s outage is reported only at that limit, since the bridges did not survive it at 512 MiB. These runs kept the heuristic fallback of Section 4.7 enabled (proof-only recovery off), so the time in RECOVERING does not isolate the cost of the recovery proof; the data-availability results are independent of that choice. A gate that always opens would pass the runs above, so we also withheld data. A test hook makes a validator publish only head markers: its DA liveness stays healthy, so the chain can reach RECOVERING, but its contiguous height never advances. The certified height is a weighted median over the certificates, so the gate should hold when four or more of the seven validators withhold data and open when three or fewer do. Table 41 shows that it did in all seven runs. In a third control, two validators signed a contiguous height of 10^{12} while the five honest ones withheld data; the median stayed at zero and the gate held, as it should against $f{=}2$ misreporting validators. These controls use two to three runs per setting, each on a freshly deployed cluster, and the withholding validators are a test configuration, not a modelled adversary. The reported third control ran with all five withholding validators.

Exploratory extension: an adaptive down threshold. We also evaluated an extension that raises $k_{\downarrow}$ after each flap and lowers it after T_{stable} healthy blocks (Section 4.5). The result is

Table 40: Republication of the SOVEREIGN interval after a total DA outage (three runs per outage length; range over runs; heuristic fallback enabled)

Outage (s)	Interval (blocks)	Time in Recovering (s)	Heights found, identical
120	75–77	36.6–61.1	229 / 229
300	217–219	60.9–67.3	655 / 655
600	465–475	49.0–61.2	1407 / 1407

Table 41: Backfill-gate controls after a 120 s DA outage (validators withholding block data; the gate must hold for 300 s or open; heuristic fallback enabled)

Setting	Runs	Interval (blocks)	Outcome
4 of 7 withhold data	3	201–208	held in RECOVERING 3 / 3
3 of 7 withhold data	2	67–72	left RECOVERING after 34 and 43 s; data identical
2 inflated claims + 5 withhold	2	202–208	held in RECOVERING 2 / 2

negative: in live runs it did not outperform a static threshold of 4 (no flap at a detection delay of 13.7 s), and its absorption of repeated short pulses rests on simulation alone (Appendix J). We therefore recommend a static threshold of 4 or more (Section 8) and treat the backoff as an extension, not a contribution.

Appendix I complements these results by measuring how deep a real Bitcoin reorganization must be before the FSM reacts.

Cost: certificate scaling and load sensitivity

Where the cost is not. Each proposal carries the health certificate as its first transaction: one signed vote extension per validator in the quorum plus the leader's claimed aggregation. At seven validators it is 894 bytes on average (live, ANCHORED; 889–926 across states, 300 consecutive blocks), which splits into a fixed 70 bytes and 118 per validator (22 for the address, 28 for the extension, 66 for the signature). The signatures are a copy of the vote extensions, kept in the proposal because `ProcessProposal` receives none under ABCI 2.0; they are separate from the signatures in the block's last commit, so the cost is paid twice. The one proof-carrying block observed was 14.9 KB, consistent with the 14.7 KB proof. Validating an accepted proposal, including the ed25519 verification of one signature per validator, costs 301 μs per block at seven validators (mean of ten benchmark runs on the final commit with the machine idle, standard deviation 13 μs). Size and validation time are linear in the number of validators (Table 42); we measured the block interval only at seven validators, so the larger rows are a microbenchmark of the certificate, not a measurement of a larger network. Verifying a recovery proof, a one-off event, takes 26 ms and is not included. `ExtendVote`, which reads the live Bitcoin and Celestia sensors before a validator can precommit, takes 16.6 ms on average (p95 51 ms, five repeats) and does not depend on the number of validators, since each validator reads only its own sensors. None of these is large enough to explain a lower block rate at a ∼1 s block interval at seven validators.

Where the cost is. Against a single local process, Engram's blocks arrive 0.23 s later (1.262 against 1.034 s). Table 10 shows that this gap is not Engram's. Seven vanilla validators on the same hosts, network, genesis and load produce a block every 1.233 s with vote extensions disabled and every 1.229 s with empty extensions, within 0.03 s of Engram (95% CIs of 0.015–0.023 s). Hence the 0.2 s is the price of running a seven-validator BFT network on one host, and vote extensions as such add no measurable time. The paired design isolates Engram's own addition. It alternates the two arms on fresh deployments of the same hosts, genesis and load, with the arm that runs first alternating, so that drift of the host cancels within a pair. Over 20 pairs Engram's

Table 42: Certificate size and `ProcessProposal` time against the number of validators (microbenchmark of the shipped code with real ed25519 signatures, one core, ten runs; sizes are from the benchmark fixture, whose extensions are about 15% smaller than the live ones, so the live size at n=7 is 894 against 776 bytes).

Validators	Certificate, bytes	`ProcessProposal`, μs (sd)
4	473	181 (7)
7	776	301 (13)
16	1,685	638 (21)
32	3,301	1,242 (22)
64	6,533	2,466 (58)

Table 43: Load sensitivity under open-loop offered load, seven validators on one host (60 s per repeat, five repeats; mean ± 95% Student-t interval). The ANCHORED share is the fraction of two-second samples of one validator's FSM state spent in ANCHORED and was recorded for the third arm only.

Offered (tx/s)	Arm	Committed (tx/s)	p95 latency (s)	Peak CPU (%)	Anchored share
20	Engram	20 ± 0	1.6 ± 0.1	10 ± 2	–
20	Vanilla	20 ± 0	1.6 ± 0.1	8 ± 3	–
50	Engram	50 ± 0	1.7 ± 0.0	15 ± 8	–
50	Vanilla	50 ± 0	1.6 ± 0.0	12 ± 2	–
100	Engram	100 ± 0	2.2 ± 0.4	29 ± 6	–
100	Engram, peer-latency bound 5000 ms	100 ± 0	1.7 ± 0.1	21 ± 5	0.83 ± 0.09
100	Vanilla	100 ± 0	1.6 ± 0.1	19 ± 11	–
200	Engram	200 ± 0	3.8 ± 0.8	81 ± 11	–
200	Engram, peer-latency bound 5000 ms	200 ± 0	1.9 ± 0.5	39 ± 23	0.13 ± 0.37
200	Vanilla	200 ± 0	1.5 ± 0.1	23 ± 5	–
500	Engram	500 ± 0	6.5 ± 0.8	83 ± 12	–
500	Engram, peer-latency bound 5000 ms	500 ± 0	4.6 ± 1.1	73 ± 11	0.00 ± 0.00
500	Vanilla	500 ± 0	1.9 ± 0.4	64 ± 30	–

block interval exceeds the vanilla one by 0.022 s per height (1.8%; 1.246 s against 1.225 s). The 95% interval is 0.005 to 0.038 s from the Student t distribution on the pair differences and 0.008 to 0.038 s from a bootstrap over pairs; it excludes zero, so Engram adds a small cost, of the order of the 16.6 ms `ExtendVote` time. One pair (+0.144 s) is an outlier; without it the estimate is 0.015 s (0.005 to 0.026 s). We planned to stop when the half-width fell below 0.012 s or at 20 pairs; at 20 pairs it was 0.017 s, so the interval is wider than planned. The unpaired deployments of Table 10 agree: the three Engram deployments average 1.2707, 1.2423 and 1.2620 s and the two seven-validator vanilla deployments 1.2292 and 1.2331 s, a difference of 0.027 s with a 95% interval of −0.008 to 0.062 s from the pooled between-deployment spread (0.012 s, three degrees of freedom). The repeats of one arm share a chain, so the unit of independence is the deployment. Those arms ran on separate deployments except the first Engram and the extensions-on arm, which shared one chain (Engram first); the Engram arm was repeated on a fresh deployment to check for an order effect and moved by 0.02 s.

Load sensitivity. Table 10 is measured at 20 tx/s. Both systems accepted every submitted transaction into the mempool across an earlier 5–100 tx/s ramp, but mempool acceptance is not throughput. To see how the two systems respond to load we offered 20, 50, 100, 200 and 500 tx/s for 60 s with five repeats per rate and drained the mempool after each (Table 43). We did not look for a throughput ceiling, which one host cannot give: vanilla CometBFT and Engram committed every offered transaction at every rate, so neither saturated, and the limit we hit first was CPU, with the busiest validator at 64–83% at 500 tx/s on a host that runs all seven. What separates the systems is latency: the p95 stays at 1.5–1.9 s for vanilla, while Engram's rises from 2.2 s at 100 tx/s to 6.5 s at 500 tx/s.

At 20 tx/s the addition of Engram is a few percent of the block interval (Table 10); as the load grows, latency grows and the FSM leaves ANCHORED for part of the time, already at 100 tx/s,

as follows. The Engram figures from 100 tx/s upward are not a steady ANCHORED state. The peer-latency sensor read round-trip times of up to 1.2 s between validators under this load in the first Engram arm, above the 200 ms bound, so we repeated the sweep with the bound raised to 5000 ms. The chain still left ANCHORED: the share was 0.83 at 100 tx/s, 0.13 at 200 tx/s and 0 at 500 tx/s, so the peer-latency bound is not the only reason. A second candidate is the DA check: under load its RPC may exceed its timeout and read as a failed layer. We repeated the runs at 200 and 500 tx/s on fresh deployments, three each, with the DA RPC timeout raised to 90 s and the DA check timeout to 2.5 s, against three runs with the defaults. The share of samples in ANCHORED rose from 0.68 to 0.75 at 200 tx/s and from 0.36 to 0.80 at 500 tx/s, where the default runs also reached SOVEREIGN and the relaxed ones did not; the relaxed runs still left ANCHORED, so the DA timeouts explain part of the departure and not all of it. In the relaxed runs at 500 tx/s the p95 latency was 3.1–4.6 s and the peak validator CPU was 42–46% (three runs). That arm had lower latency than the default one (p95 1.9 against 3.8 s at 200 tx/s) and was still above vanilla. We therefore cannot separate the cost of the consensus extension from the effect of a demoted FSM above 100 tx/s on this host, and report only that Engram's latency grows with load where vanilla's does not. All seven validators share the host's cores, so absolute values do not transfer to separate machines, and one host does not support a claim about production throughput or scalability. The 5000 ms bound was set through the environment of that deployment, whose genesis we did not keep; the first Engram arm did not record the FSM state.

Discussion: additional security analyses and the testbed gap

Repeated triggering of Suspicious. An adversary that repeatedly pushes the chain into SUSPICIOUS is limited only by the bounded forced-transaction queue and the down threshold. An optional backoff of the down threshold, which we did not evaluate on the RECOVERING→SOVEREIGN edge, was meant to raise the price of such repetition. On the ANCHORED→SUSPICIOUS edge it did not beat a static threshold of 4 in live runs, and its absorption of short pulses has no live evidence (Appendix J).

Hysteresis: noise against concealment. A higher down threshold k tolerates longer bursts of noise, and it also tolerates a longer run of reports that a Byzantine proposer interleaves with honest ones. Within the assumption the choice of k does not matter for detection: for each of the 21 placements of two Byzantine proposers in the seven-slot rotation, a real DA outage was detected after exactly k blocks ($k \in \{2, 3, 4\}$, simulation). With three Byzantine validators, outside the assumption, a larger k makes concealment easier: in the simulation the outage was missed for 14 of 35 placements at k=3 and for 28 at k=4. The live runs at k=2 detected it in 5 of 5 runs; at k=4 the live runs did not reproduce the simulation's concealment for interleaved placements, so concealment at k=4 rests on the simulation alone. The experiments ran with $k_{\downarrow}$=2, which the live noise sweeps show to be too low for a wide-area network (flapping, 82% of the noisy phase in ANCHORED); we recommend $k_{\downarrow} \geq 4$ as a deployment setting, set from the noise of the environment, at the price of easier concealment. Appendix K gives the detail, including a window rule that is not cut by interleaving and that we did not implement.

Testbed gap. Four properties of the testbed limit how its results transfer to a production deployment.

- *Single physical host.* All Testbed A containers share one machine's CPU and memory, so resource contention can affect every validator at once; wide-area conditions are emulated with `tc netem` rather than real geographic distribution. Testbed B removes both limitations for the experiments repeated on it (so far the proving-latency runs; its composite and load runs are pending).
- *Accelerated Bitcoin.* Regtest blocks arrive about 30× faster than on mainnet. Height-based

thresholds are unaffected, but wall-clock durations scale accordingly.

- *DA payload.* The DA publisher submits the transactions of committed blocks in height-ordered batches of at most 1.5 MB and tracks the highest contiguous published height; the exit gate of Section 4.7 requires that height to cover the SOVEREIGN interval. The gate rests on a quorum-reported contiguous height, not on a cryptographic proof of retrievability. Section 7.3 reports the republication of the interval after a live DA outage.
- *Single-bridge trust.* Each validator trusts its own bridge node, which downloads and verifies full Celestia blocks, rather than sampling across independent peers; our DA-outage results measure bridge failures, not sampling-detected partial withholding.

Protocol: sensor definitions and long intervals

Bitcoin finality gap. $\Delta H_{\mathrm{BTC}} = H_{\mathrm{current}} - \min(H_{\mathrm{submitted}}, H_{\mathrm{anchored}})$ measures how far the latest checkpoint lags Bitcoin's tip; it grows under congestion or checkpoint censorship. Because a delay metric cannot detect a checkpoint that was reorganized away, each validator also re-verifies, against its own Bitcoin full node, that the checkpoint transaction is still included at depth $\geq K_{\mathrm{DEEP_FINALITY}}$. A failed re-verification is a critical condition.

DA gap. $\Delta H_{\mathrm{DA}} = H_{\mathrm{local}} - H_{\mathrm{verified}}$ measures the lag between the chain head and the highest height whose DA submission the validator has confirmed as retrievable from its own Celestia bridge node. This is bridge-level retrievability, not a data-availability-sampling proof; Section 8.4 describes what the prototype publishes.

Units of the gaps. The two gaps count different things because the two layers guarantee different things. The Bitcoin gap counts Bitcoin blocks, since a checkpoint's guarantee is its confirmation depth; it grows only with the Bitcoin blocks that arrive while no checkpoint is anchored, so a slow Bitcoin chain adds no gap. The DA gap counts Engram heights, since the guarantee of the DA layer is that every Engram height is retrievable. Each threshold is read in the unit of its own layer and converts to time at that layer's cadence: a Bitcoin block takes 600 s on mainnet and 20 s on the testbed, an Engram block about 1.2 s on both. A time bound that must cover a Bitcoin fault, such as the SUSPICIOUS timeout T_{sus} (Section 4.5), therefore follows the Bitcoin cadence of the deployment (Section 8).

P2P health. Peer count alone cannot distinguish a long-lived honest peer from a fresh Sybil, and eclipse attacks combine vectors no single metric captures [61, 62]. The profiler combines six bounds – subnet diversity, active anchor peers, clean peers, churn rate, mean peer tenure, and latency – applied to the Engram P2P interface, the Bitcoin client, and the Celestia client. These bounds characterize peripheral and partial P2P degradation; a partition of the core consensus network below a quorum is outside what any FSM can handle (Section 1).

Longer intervals. An interval longer than 256 headers is covered by a sequence of proofs: each accepted proof advances the on-chain rt_{last} to its rt_{new}, and the next proof must start from that root. This costs $\lceil L/256 \rceil$ proofs for L blocks. An aggregator circuit can fold up to four chunk proofs into one proof of the same size, which makes verification cost independent of L up to 1024 headers at the price of one extra proof; proving time still grows with L (Table 44, Appendix D).

The aggregate has a constant size and verification time, and costs more to prove than the chunks alone; the prover's chunks were built one after another, and proving them concurrently was not measured.

Table 44: Cost of the recovery proof by interval length L (offline workstation, synthetic chain; one by one against aggregate, from Appendix D)

L	Submission	Prove	Bytes	Verify
256	single proof	31.5 s	14,656	19.8 ms
512	aggregate of 2	89.9 s	14,656	20.2 ms
512	one by one	60.5 s	29,312	39.5 ms
1024	aggregate of 4	146.7 s	14,656	20.2 ms
1024	one by one	120.1 s	58,624	79.1 ms

Verify is the validators' embedded WebAssembly verifier, per proof (30 iterations, three runs); the one-by-one rows are n times the leaf cost, and the aggregate's proving time includes its leaf proofs. Aggregate proving and verification are averaged over five trials per length, each leaf proof was timed once, and the chain is synthetic (Appendix D).

Related-work taxonomy and fault schedules

Network-level faults use kernel traffic control (`tc netem` via Pumba): uniform delay and jitter, packet loss, single-node isolation, and a heterogeneous per-validator wide-area profile approximating seven regions. Appendix G lists the parameters of every profile. The *WAN latency baseline* is the per-validator delay and jitter profile of Table 27, held for the whole run unless a step in Table 24 suspends it.

Application-level faults reproduce mechanisms that delay alone cannot, and each simulates the consequence of a failure rather than its cause.

- *Bitcoin congestion* suspends the submission of new checkpoints (a pending one still confirms), so the finality gap grows directly. It does not congest the Bitcoin mempool.
- *DA outage* stops one or all seven Celestia bridge containers. It does not emulate a data-availability-sampling failure.
- *Byzantine behavior* is injected through attacker processes or through a validator configured to forge proposal fields, omit transactions, double-sign, or flood timeout messages.

Every fault targets a peripheral layer or a minority of validators; none partitions the core consensus network below a quorum, consistent with the scope defined in Section 1.

Table 24 gives the onset, mechanism, and duration of each fault in the two scripted runs of Section 7.2. Times are relative to the start of each run, durations are the length of the polling window the script holds the fault for, and each of S2–S5 is followed by a wait of up to 600 s for the validators to return to ANCHORED before the next scenario starts.

The table is in Appendix F: the composite run holds a Bitcoin fault, adds a DA outage, cycles packet loss on two validators, heals, and waits for recovery; the seven-scenario sweep applies one fault at a time (S2–S6) between a healthy baseline and a recovery wait.

Protocol: lemma limits, certificate binding and the intended verifier

What the lemma does not give. It bounds each metric separately. The aggregate vector need not equal the observation of any honest validator, so a condition on several metrics can hold of $\mathsf{Agg}(C^h)$ when it holds of no honest observation. For example, with three equally weighted honest validators and three metrics, each validator seeing a different metric beyond its threshold, every aggregated metric is within its threshold although every honest validator sees a degradation. The lemma also does not say which honest value the aggregate takes when honest values differ (see the proposer discussion below).

Adversarial cases. The lemma does not require honest observations to agree; it bounds what

the adversary can do when they disagree, and each adversarial case reduces to it (Appendix B). A stale or replayed observation fails the height and round binding, two observations by one validator count once, and false observations below $W/3$ cannot leave the honest range. The proposer chooses the set S and may omit honest observations; if S still meets (d) the lemma holds, and if not, the certificate is not well-formed and honest validators reject the proposal. The proposer's residual influence is choosing *which* honest value in the honest range is selected when honest validators genuinely disagree: with n=7 equal-power validators and a quorum of q=5 it reaches the third to the fifth smallest observation, the median of all seven being the fourth. Only when honest observations straddle a threshold does this choice change a decision, and a transition needs the hysteresis count over consecutive blocks while proposers rotate, so a Byzantine proposer influences the blocks it proposes. In the live collusion suite (Section 7, case B3), honest validators accepted a proposer that kept exactly a quorum of certificates in 3 of 3 repeats and rejected one that kept fewer, 210 times in total across the 3 repeats.

Height and round binding. The certificate committed at height h describes observations signed at height $h-1$, one block of lag. A signature binds $(chainID, h-1, r, m_i)$, and a well-formed certificate takes observations only from the commit round $r^\star$ and only from validators whose precommit is in that commit. This prevents replay of an observation from another height or round into C^h. The binding is a property of the consensus instance, not of the measurement: a validator can still sign a cached Bitcoin tip or stale peer metrics (Section 4.2).

Intended verifier. A CometBFT quorum certificate shows that a quorum signed a block. The recovery proof adds a statement about the headers that the chain recorded between two signed blocks: each records SOVEREIGN or RECOVERING with the lock flag set. The circuit does not tie the header chain to the two signed blocks; the public inputs do, and a verifier must compare them with its own record of the roots, the height of the first block and the hashes of the two boundary blocks. This matters to a verifier that does not replay the chain, such as a bridge or light client on another chain, a node syncing from genesis, or a Bitcoin observer (Section 3.2). For validators, which already hold the headers, it is a consistency check. The target verifier is an external light client that accepts a re-anchoring checkpoint only with a valid proof; the prototype does not include such a client. As a check that the proof can be verified without the node, a separate process on the same host read a committed block over RPC, took the proof (14,656 bytes) and the 224-byte public inputs from its recovery-proof transaction, and verified them with `bb` against the verification key shipped with the repository; this was one proof on one chain.

Intervals longer than 256 headers and the proving cost by length (Table 44) are in Appendix L.

Formal analysis: certificates in the model and the refinement

Health certificates in the model. The server layer models each validator's observation as three Booleans (healthy, warning, critical) and the Bitcoin and DA gaps clipped to 0..3; the aggregate is a per-field majority for the Booleans and a weighted median for the gaps, and `IsValidProposal` rejects a certificate below a quorum. A Byzantine validator may sign any observation; an honest validator with an eclipsed P2P layer may report a degraded one, never a healthier one, which is a sensor-soundness assumption of the model. Every validator has the same voting power, and the proposer may choose any member set that forms a quorum.

Lemma 1 is checked as `AggWithinHonestRange` with Apalache, with the observation vector left symbolic so that one check covers every vector. For N=4 and f=1 it reported no violation (22 s). With two Byzantine validators in the same network it reports a violation (25 s), so the bound on Byzantine members is necessary. Signatures, height and round binding, and the individual P2P counters are not modelled. The check covers the aggregate as a function; the certificate's role inside `IsValidProposal` is covered only by the open refinement obligation.

Why the refinement is not claimed. We split the obligation by the actions that may fire and checked each part with TLC. The environment part completed without violation; the consensus-core part did not complete; and neither covers the interleaving of environment and consensus steps. A completed run constrains a behaviour only if that behaviour is reachable, so we required of every configuration that a round change, a Byzantine proposal and a decided block all be reachable. In the monolithic bound L0 (one round, one height, three timestamps) a round change and a Byzantine proposal are reachable within seconds, and two honest prevotes for one proposal co-occur at depth 5. A lock, a decided block and the FSM transition that only a decision commits were not reached: the full L0 space was explored to 15.3 billion states generated (249 million distinct, depth 12) without a violation and without terminating, and probes restricted to the healthy path reached no decision either. None of these configurations is reported as a result (Appendix E).

The refinement target is an extension of the object of LiDO [47] with one added action and two guards on timeout certification, and the added action weakens the target. The mapping simplifies four things, which the appendix lists, and the obligation concerns safety only. An inductive check with Apalache over the whole `ServerNext` would avoid the composition gap. It is set up as `MC_ServerRefinementStep_Apalache.tla`: the driver typechecks and its base case, invariant preservation by the initial states, holds, but the check that the initial states map to abstract initial states stops in Apalache's SMT backend, which does not implement a set filter over a powerset. The refinement therefore rests on TLC alone.

Evaluation: colluding validators and the hysteresis sweep

Colluding Byzantine validators (f=2). The threat model allows f=2 colluding validators at n=7. We therefore define a collusion suite in which two validators act jointly: (B1) both report coordinated false health observations; (B2) one reports false observations while the other, as proposer, builds its certificate to favor them; (B3) the proposer omits honest observations, both down to the smallest quorum (five of seven validators) and below it; (B4) both double-sign at the same height; (B5) both attempt to include a withdrawal while in SOVEREIGN; and (B6) the proposer attaches a stale or forged Bitcoin or DA receipt. AppHash agreement is necessary but not sufficient evidence of safety, since honest validators could agree on an illegal transition. For every committed height we therefore also replay $\delta(s^{h-1}, \mathsf{Agg}(C^h))$ offline from the committed certificate and check that it equals the committed s^h, that the aggregate lies within the honest range of Lemma 1, and that no withdrawal commits while $s^h \in \{\text{SOVEREIGN}, \text{RECOVERING}\}$. We repeated each case three times in succession on the 7-validator cluster, every repeat starting from ANCHORED (B5 starts from SOVEREIGN by design). The repeats of a case share one chain, so they are not independent deployments. Table 38 lists the cases. In all 33 repeats of cases A3–B7, the honest validators committed identical AppHashes and the offline replay found no block whose committed state differs from $\delta(s^{h-1}, \mathsf{Agg}(C^h))$, no aggregate outside the honest range, and no withdrawal committed in a locked state. Honest validators rejected the forged Bitcoin hash (A4: 216 rejections in total), the stale or forged receipts (B6), the withdrawals in a locked state (B5: 320), and the proposer that kept fewer than a quorum of certificates (B3b: 210); B3a, where the proposer keeps exactly a quorum, was accepted. In B4 the honest validators produced double-signing evidence for both validators, within two blocks of the second signature at the longest. B7 repeats B1 while honest validators are actually eclipsed, so that part of the honest observations are themselves degraded: with two eclipsed (B7a) the chain behaved as in the other cases, and with three (B7b) it reached SOVEREIGN in all three repeats, with every aggregate still inside the honest range.

Not every case is rejected, and safety does not rest on rejection in the others. In B1, B2, B3a and B7 the blocks are well-formed and the reports they carry are opinions rather than checkable facts; there safety rests on the aggregation. With at most two colluding validators the weighted

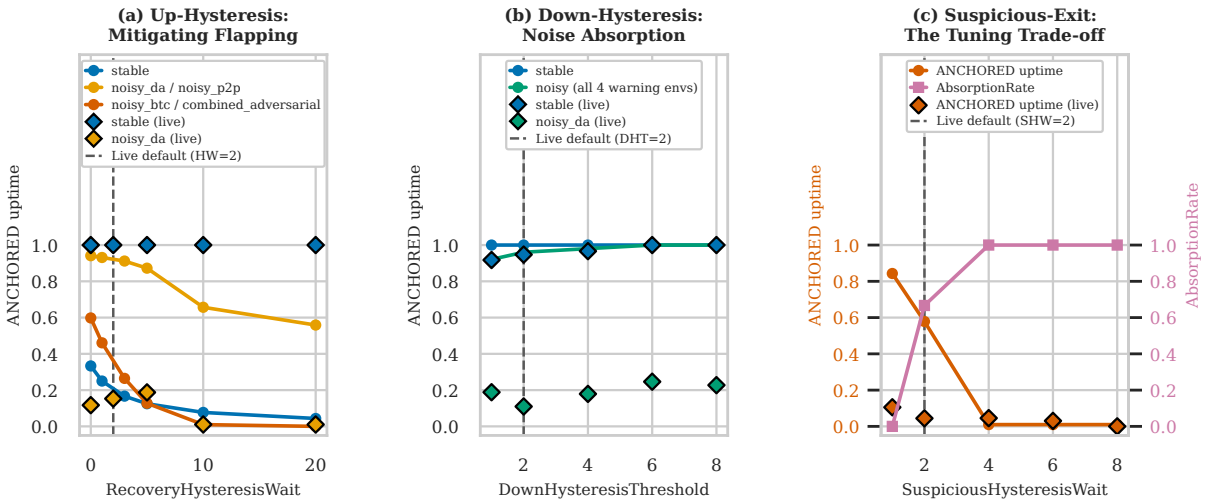


Figure 8: Hysteresis sweeps on the three gated edges: (a) RECOVERING→ANCHORED (wait H), (b) ANCHORED→SUSPICIOUS (down threshold k), (c) SUSPICIOUS→ANCHORED (wait; right axis: absorption rate). ANCHORED uptime is the fraction of blocks spent in ANCHORED. Lines: in-process runs under fixed-seed noise profiles; diamonds: live spot-checks on the seven-validator cluster (Testbed A), where divergence from the in-process line reflects network behavior; dashed line: the prototype default.

Table 45: Down-hysteresis sweep (ANCHORED→SUSPICIOUS), in-process (101 blocks, noise probability 0.2, seed 11, one run per threshold)

Threshold	1	2	4	6	8
Flapping (count)	5	1	1	0	0
Anchored uptime (%)	92.08	96.04	98.02	100.00	100.00
Absorption rate (%)	84.21	94.74	95.24	100.00	100.00

median cannot leave the range of the honest observations (Lemma 1), and the replay found no block outside that range. The sweep of Table 39 shows where this stops: with b=3 the aggregate leaves the range and the adversary forces SOVEREIGN. A3 is accepted because a DA attestation one height ahead lies inside the tolerance of the receipt check, a limit of the DA trust assumption rather than of the aggregation. B4 is not rejected either, since each vote is valid by itself; the honest validators record the double signature as evidence, and with two double-signers the repeats committed identical AppHashes and replayed cleanly.

State transitions cannot be attributed to the adversary in this setting. The control c0, in which no validator misbehaves and one is only restarted, also moved from ANCHORED to SUSPICIOUS in 3 of 3 repeats, because of ordinary sensor noise on the testbed. We therefore judge the cases by replay, AppHash agreement, and the honest-range check, not by whether a transition occurred.

Tables 38 and 39 are in Appendix L.

On the ANCHORED→SUSPICIOUS edge (Table 45), raising the down-hysteresis threshold from 1 to 4 reduces flapping from 5 events to 1 and raises anchored uptime from 92.1% to 98.0%, and the absorption rate rises from 84.2% to 95.2% and reaches 100% at threshold 6. The live spot-check changes the conclusion. Under wide-area jitter alone, with no injected fault, thresholds 1, 2, and 4 still flapped, and only thresholds ≥ 6 were clean (0 flaps, 100% uptime). The later live runs do not repeat this for k=4: it did not flap in the stationary sweep (one run, Table 31) nor in the six three-phase runs (Table 33), whereas the spot-check is a single run on one noise period, so we weigh the later runs more. The default of 2 used in the other experiments is therefore too low for WAN conditions; the other experiments did not count flaps and cannot be taken as evidence to the contrary. A static default must trade reaction time (low k) against noise immunity (high k); the adaptive alternative is the exploratory extension at the end of this section.

The other two edges show the opposite side of the trade-off. On the RECOVERING→ANCHORED edge, a larger hysteresis wait strictly reduces anchored uptime under sustained critical-level noise (59.8%→0.0% as the threshold goes from 0 to 20), under the critical-level noise of Section L. This follows from the design. In RECOVERING, a critical reading returns the chain directly to SOVEREIGN (the RECOVERING branch of `CalculateNextState`), so the run of healthy blocks must start again, and a longer wait is less likely to complete between two critical readings. On the SUSPICIOUS→ANCHORED edge, absorption rate rises from 0% to 100% as the threshold goes from 1 to 4, at a cost: the SUSPICIOUS timeout keeps running while the counter accumulates, so only thresholds 1–2 reached ANCHORED before the timeout forced SOVEREIGN. Any adaptive threshold must therefore stay below the SUSPICIOUS timeout, which bounds $k_{\max}$.

Discussion: griefing, history divergence and future work

Cost of griefing. An adversary that degrades the peripheral view of the honest validators – for example by delaying their Bitcoin connectivity – can drive the chain into SOVEREIGN and freeze withdrawals for as long as the degradation lasts. In P4 (Section 7.1) a sustained delay on the P2P interfaces of one validator, observed by all seven, was enough in all three runs; we did not establish whether the cause is the sensor design or a threshold crossing, so Lemma 1 does not prevent it. This is a new attack surface relative to a chain that ignores its periphery, but not relative to a strict receipt-gated chain: the same attacker stalls the strict receipt-gated chain, whereas Engram freezes only withdrawals. Lemma 1 prevents Byzantine validators from causing the freeze alone: honest validators whose peripheral view is degraded must supply the rest of the majority. Together with the Byzantine validators they must hold more than half of the certificate weight, and the proposer's choice of quorum subset sets that weight. At n=7 with two colluders, an honest proposer that includes all seven observations requires two degraded honest validators, and a Byzantine proposer that selects a minimal quorum of five requires one. In a design without a quorum-backed health input, one node's view suffices. The freeze is the intended response to an unverifiable periphery, not a safety violation. Its economic cost to users and the attacker's cost of sustaining it are not quantified.

The analysis of repeated triggering of SUSPICIOUS is in Appendix L.

History divergence at re-anchoring. During SOVEREIGN, the ordering of blocks rests on the validator quorum alone. A coalition exceeding f could produce a SOVEREIGN history that differs from one it would have produced under Bitcoin timestamping; the recovery proof certifies the history's continuity, its recorded mode and lock labels, and the boundary quorum signatures. It does not certify its uniqueness beyond the f bound, that every transition followed the transition function, or that no withdrawal executed (the lock is enforced when a block executes; the proof covers the flag recorded for it), and it does not cover the unproven tail. The withdrawal lock limits what such a history can extract.

Data availability of the Sovereign interval. During SOVEREIGN the interval's block data is held only by the validators' own block stores. The backfill gate of Section 4.7 makes its publication a precondition for returning to ANCHORED.

Additional figures and tables

Methodology: baseline definitions, workloads and the Celestia setup

Celestia. The DA layer is Celestia software, not the Celestia network. A private `celestia-app` chain with one validator produces a block every 11.0 s (median 11.0 s, standard deviation 0.04 s over 80 consecutive blocks; no timing parameter is set, so this is the image default), about nine Engram blocks, and the seven bridges download and verify its full blocks. There is no public network, no light node or data availability sampling, and no independent operator, so a DA fault

Table 46: Hysteresis waits of the FSM (defaults of the prototype; Section 4.5).

Wait	Applies to	Default	Adaptive
Down wait $k_{\downarrow}$	ANCHORED→SUSPICIOUS, RECOVERING→SOVEREIGN	2; with the optional backoff, doubles per flap level up to a cap (equal to the threshold on the testbed, so the threshold is static)	Optional
Suspicious exit $k_{\uparrow}$	SUSPICIOUS→ANCHORED	2 (8 on the testbed)	No
Recovery H	RECOVERING→ANCHORED	2	No

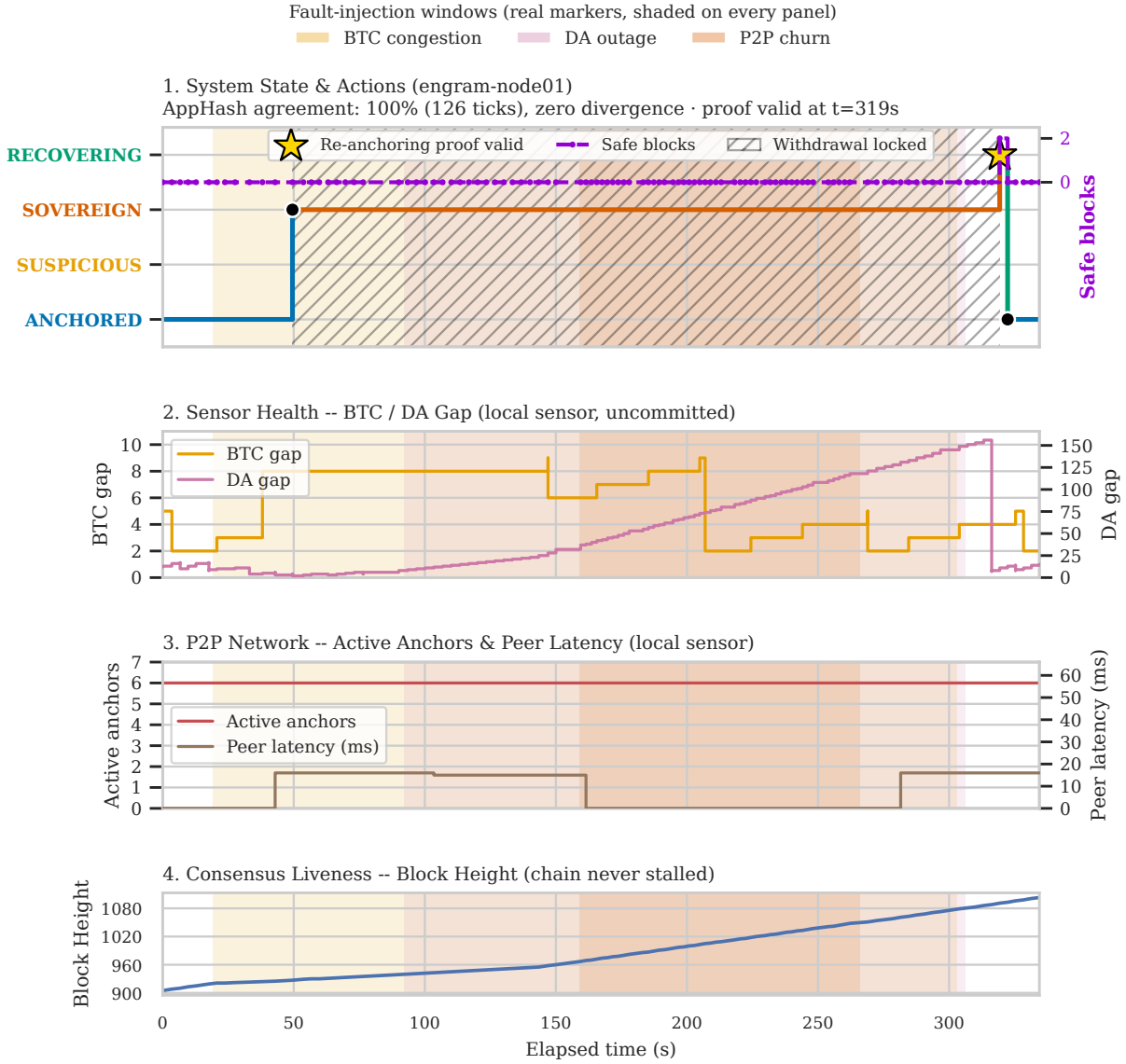


Figure 9: Composite-fault timeline of one run: (1) FSM state of one validator with the lock (hatched) and the safe-block counter; (2) Bitcoin and DA gaps; (3) active anchors and peer latency; (4) block height. Panels 2 and 3 are the validator's local, uncommitted sensor readings. Shaded bands are the fault windows (Bitcoin congestion, DA outage, P2P packet loss; schedule in Table 24). First composite run: one trial, heuristic fallback enabled.

Table 47: Time from the start of a fault to the lock of withdrawals, as measured on the testbed (Bitcoin block every 20 s) and converted to mainnet cadence (every 600 s). Engram blocks are 1.2 s on both.

Fault	Testbed	Bitcoin blocks	Engram blocks	Mainnet
Checkpoint pause	125 s	6.3	104	62 min, 3,100 blocks
DA outage	40 s	—	33	40 s, 33 blocks

is the loss of the bridges and not the partial withholding of data. The DA gap counts Engram heights (Section 4.2): its threshold of 30 blocks, about 36 s, spans about three Celestia blocks, and a healthy chain sits below it: in the first 15 s of seven composite runs the gap averaged 12.3 blocks (95th percentile 17, maximum 18; 781 samples). The lock time after a DA outage does not scale with the Celestia cadence. The DA results therefore show what Engram does when the DA layer is lost, not that it detects partial withholding.

Out-of-band receipt-gated baseline (RQ2). The same predicate and the same pinned ANCHORED state, but the receipts are not read through the vote extensions: in `PrepareProposal` and `ProcessProposal` each validator reads Bitcoin and the DA layer itself and checks the proposal against its own reading (`ENGRAM_MODE=hard-gated-oob`). A rejected height therefore cannot starve the next one of fresh receipts, which removes both dependencies on block commits of the in-band baseline (Section 7.2). The price is that two honest validators may read different values and disagree on the validity of a proposal, which the certificate of Section 4.3 exists to prevent. It is a second construct of this paper, built to show whether the in-band baseline's failure to resume is a property of where the receipts are observed rather than of gating validity on them. In what follows the first baseline is called in-band and this one out-of-band observation; both are receipt-gated and neither locks withdrawals.

What the predicate implies, and what RQ2 measures. If, over an interval, some receipt predicate holds for no proposal at any height, then no value satisfying V_h can be decided in that interval. This is the external-validity property of Byzantine agreement [55], and in CometBFT an application rejection in `ProcessProposal` yields a nil prevote. It is a consequence of the predicate, not a result of this paper. RQ2 therefore asks different questions: how many blocks the implementation commits while the predicate is nearly unsatisfiable (the receipt tolerance widens with the round), whether it resumes after the fault, and what Engram commits instead. The first two depend on the implementation and are reported separately for the two variants.

Vanilla CometBFT (control, RQ4). The same application with the health-aware ABCI++ hooks disabled. It does not depend on Bitcoin or Celestia and therefore never stalls when they fail; it serves as the cost reference for RQ4 and as a control showing that measured stalls or fallbacks are not artifacts of the harness.

Ablation baselines. A peer-count-only P2P detector isolates the contribution of the six-bound profiler (RQ1), and each hysteresis edge with its threshold set to its floor isolates the contribution of hysteresis (RQ3).

Synthetic workloads. Two synthetic workloads feed the in-process studies; both use fixed seeds, and Appendix F gives the generators and their limits. The hysteresis sweeps drive the real state machine with sensor readings in which each block is disturbed independently with probability 0.2, for 100 blocks, so every threshold value sees the same noise sequence. The Monte Carlo P2P study compares the six-bound profiler with a peer-count-only rule on 2,000 benign and 2,000 full-intensity synthetic snapshots, and on ramps of rising intensity, at production-scale thresholds; it measures false-positive and false-negative rates, detection delay and an evasion threshold. The benign distribution, the attack directions and the intensity scale are our choices, so the results characterize the detectors' sensitivity within this generator.

Protocol: who verifies what

Who verifies what. The recovery proof is submitted as a message and verified by *every Engram validator* during block execution, using an embedded, deterministic WebAssembly build of the UltraHonk verifier. The message handler binds the proof's public inputs to chain state:

- rt_{last} to the committed last anchored root, and rt_{new} to the accumulator of a header the chain recorded;
- the first height to the last anchored height plus one, and the header count to the distance between the two checkpoints;
- the first and last block hashes to the CometBFT hashes the chain recorded at those heights, and the validator-set hash to that of the genesis set.

A verified height is latched so a proof cannot be replayed. Interior headers are bound only through the accumulator, not through per-block signatures. Bitcoin only timestamps the checkpoint; it does not verify the proof. After recovery, validators resume writing `OP_RETURN` checkpoints of 44 bytes (a 4-byte tag, the Engram height, and the CometBFT block hash at that height), and the Bitcoin gap sensor requires them to reach depth $K_{\text{DEEP_FINALITY}}$. Anchor re-verification compares the hash with the local block hash. The proof's rt_{new} is a Poseidon2 accumulator, not a block hash, and is not written to Bitcoin.

Formal analysis: model-checked properties and attack resilience

Six safety properties are specified. The first five are checked in the FSM safety run (the first as an invariant, the others as action or temporal properties). The sixth, Monotonicity Safety, is defined in the server layer and appears only in its refinement drivers, whose runs did not complete.

- *Circuit Breaker Isolation:* withdrawals are locked if and only if the state is SOVEREIGN or RECOVERING.
- *Hysteresis Integrity:* RECOVERING→ANCHORED requires the full hysteresis wait and a valid recovery proof.
- *Suspicious-Exit Hysteresis Integrity:* the corresponding guard on SUSPICIOUS→ANCHORED.
- *Strict FSM Transition Safety:* only transitions drawn in Fig. 3 occur.
- *Sovereign-Data Publication:* every RECOVERING→ANCHORED transition occurs with the DA backfill condition true.
- *Monotonicity Safety:* heights and time never decrease.

The Tendermint driver checks a separate invariant covering, among others, agreement on the decided value, external validity, accountability and the absence of double spending across modes. Five attack-specific properties are enforced by the validity predicate: resistance to data withholding; resistance to forged or reorganized anchors through anchor re-verification; a deterministic bound on Byzantine timeout flooding; resistance to eclipse-driven recovery through the six-bound P2P gate; and accountability, i.e. any safety violation implies double-signing evidence. Accountability is inherited from CometBFT's equivocation evidence pool rather than proved anew. The ablation study below tests two of them, data withholding and eclipse-driven recovery, by removing the mechanism that enforces each; the others are design properties of the validity predicate that Section 7 exercises with fault and attack injection. All of these properties are conditional on the Bitcoin reorg-depth bound of Section 3.1.

How the server layer models the health certificates is described in Appendix L.

Formal analysis: safety versus liveness

Safety versus liveness. We follow the separation of concerns of LiDO [47], which verifies Jolteon in Coq; our tools and our liveness encoding differ. In LiDO, safety is a property of an abstract object that transfers to a protocol through a refinement mapping. The premise that makes the abstract object safe is *QuorumOverlap*: any two valid quorums share at least one honest validator. We call it the linking invariant; with honest validators' refusal to sign conflicting votes it prevents two conflicting decisions, and with equal voting power it follows from $n \geq 3f+1$. It is a *safety* premise and does not establish liveness. That the composed server layer refines the abstract object, i.e. that the mapping preserves the invariant at every step, is the open obligation C1 of Section 5.5.

Liveness is argued separately. LiDO obtains it by adding a pacemaker to the abstract object and stating the liveness requirements as properties of bounded time segments, under a single GST. Our specification expresses the same kinds of requirement as TLA+ fairness hypotheses that TLC takes as given. It models GST as in partial synchrony: a predicate `AfterGST` on the model's real time, which never decreases, so that once GST has passed it stays passed. The predicate concerns the network only; it does not mention the FSM state or the peripheral layers, so the specified liveness property covers every FSM state, SOVEREIGN included. Four hypotheses carry the argument: the consensus layer relies on the first three, which correspond to LiDO's pacemaker and leader requirements, and the fourth, FSM progress, concerns the FSM layer and has no counterpart in LiDO. They are:

- *Partial synchrony:* behaviours reach GST (`(<>AfterGST) ~> decided`) and messages between honest validators are delivered (weak fairness on `Push` and `Pull`).
- *Timeout progress:* weak fairness on round-timeout expiry and on the $f+1$ round-skip (fast-forward) action.
- *Proposer progress:* weak fairness on each honest node's invocation step, so every honest proposer whose round is reached after GST eventually proposes.
- *FSM progress:* weak fairness on the FSM transition and on environment updates, so a persistently healthy (or unhealthy) certificate eventually drives the corresponding transition.

Only the FSM-layer liveness properties (circuit-breaker liveness, recovery initiation, complete re-anchoring) are checked; that layer has no network, so they need FSM progress alone and no GST. Consensus liveness after GST is specified on the server layer, but its check did not complete (Appendix E), so the GST model states our assumptions and is not a verified result for the composed system.

Methodology: terms, metrics and statistics

Terms. n is the number of validators (n=7 on the main testbed). A *run* is one execution of a scenario; unless stated otherwise it starts from ANCHORED on a freshly deployed chain, so runs are independent deployments, and intervals over runs are intervals over deployments. A *repeat* is a further measurement on the chain of an earlier one (the repeats of Table 10 share a deployment, so their interval describes one chain). An *arm* is one system configuration that a table compares with the others. The number of runs is written out wherever it matters. Every 95% interval over runs uses the Student t critical value for the number of runs (for example 2.78 for five), not 1.96.

Let a run record the FSM state s_b of every block b in a window W.

- *Anchored uptime* $= |\{b \in W : s_b = \text{ANCHORED}\}|/|W|$. In-process, W is the whole recorded timeline: for the noisy environments, the 100 noisy blocks plus the setup blocks that

precede them (two for the RECOVERING→ANCHORED sweep, one for the others). The no-noise control of that sweep ends at the first ANCHORED block, so its window is only the setup blocks plus the wait, and its uptime falls as the wait grows. Live, W is the sequence of RPC samples of one validator over a 300 s window.

- *Flapping count* is the number of transitions that immediately reverse the preceding transition, for example ANCHORED→SUSPICIOUS followed by SUSPICIOUS→ANCHORED, over the same W. Live runs detect reversals between consecutive samples, so a reversal faster than the polling interval is not seen.
- *Absorption rate* is $a/(a+d)$, where a is the number of disturbed blocks that begin in ANCHORED and leave the state unchanged, and d is the number that move it to SUSPICIOUS. It is defined only on the ANCHORED→SUSPICIOUS and SUSPICIOUS→ANCHORED sweeps.
- *Blocks/s* $= (h_{\max} - h_{\min})/T$ over a measurement window of length T (60 s after an 8 s warm-up in Section 7.4). *Steady blocks/s* $= 1/\bar{\imath}$, where $\bar{\imath}$ is the mean block interval. *Gap fraction* $= 1 - n\bar{\imath}/T$, where n is the number of observed intervals: the share of the window not accounted for by intervals at the steady pace, that is, time spent in heights that took longer than one interval.

Discussion and evaluation: unexpected results, internal validity and function-level checks

Function-level checks. The testbed and the TLA+ model give every validator the same power, so a defect that confuses weight with count would not appear in any live run. We therefore tested the weighted aggregation alone on three unequal weight vectors and on equal weights: it agreed with a reference weighted median on 1,500 random quorum subsets each, kept the aggregate within the honest values whenever the coalition's power was below $W/3$, and escaped it at 34 to 35 of 100 by omission of honest certificates, so the bound is tight. For n from 4 to 10, the smallest and largest aggregate over every subset the proposer may choose matched the order statistics of Section 4.3. This checks the implementation of the median and the quorum, not the integration with a genesis power assignment, which a live run with unequal power would add (Appendix H).

The peripheral P2P quorum gate and its cascade figure (Fig. 6) are in Appendix L.

Unexpected results. Three results need a causal account.

- *Block rate.* Against a single local vanilla process, Engram's block interval is longer. Seven vanilla validators on the same hosts give the same interval, with vote extensions on or off, so the gap is the cost of the seven-validator network and Engram's own addition is 1.8% (95% interval 0.4 to 3.1% over 20 paired deployments; Section 7.4).
- *Hysteresis threshold.* A static threshold of 2 flapped under wide-area jitter (9.0 flaps per validator), and thresholds of 4, 6 and 8 did not (Section 7.3; three runs for 2 and 6, one for 4 and 8). We did not isolate which sensor crosses its threshold; a run with the jitter profile removed, and per-predicate logging, would show it.
- *Reorg depth.* Reorgs of depth 3–5 orphaned confirmed blocks without a reaction, while depth 15 escalated through the suspicious-duration bound (Appendix I). Whether the anchor-reverification flag was ever set is not established, because the per-validator sensor value was not captured; logging it and sweeping depths 6–14 would locate the boundary.

Internal validity. Host sharing can confound cost measurements. Repeated measurements with confidence intervals, and a vanilla comparison on the same host under the same load, mitigate this; the host still affects absolute values.

The four properties by which the testbed limits the transfer of its results are in Appendix L.